جزوهٔ درسی · دانشکدهٔ مهندسی کامپیوتر

ساختمان داده‌ها و الگوریتم‌ها

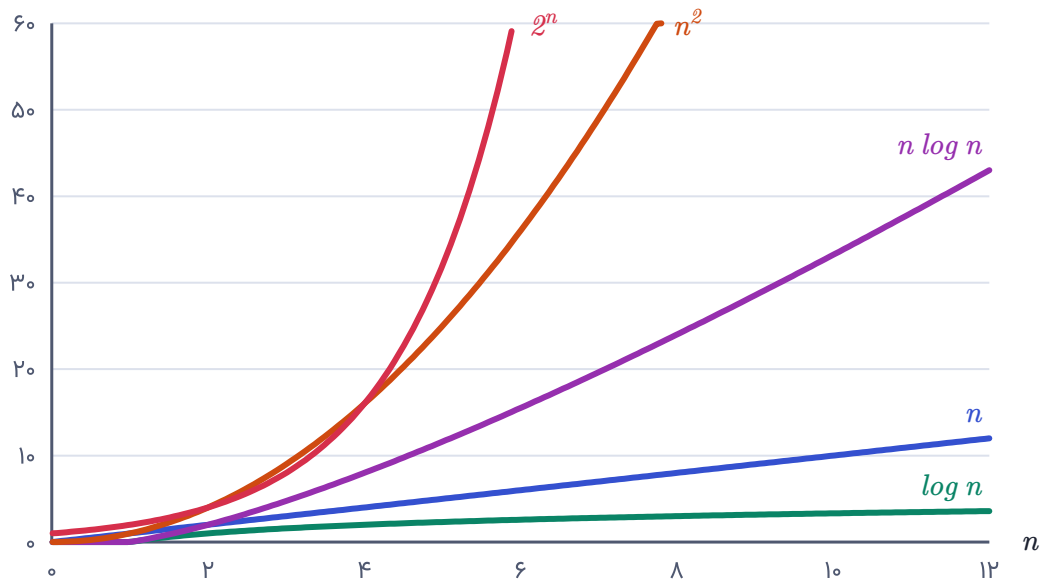
از تحلیل زمان اجرا و نمادهای مجانبی تا تقسیم و حل، الگوریتم‌های تصادفی و تحلیل سرشکنی؛ با توضیح کامل هر مفهوم، اثبات‌ها، مثال‌های حل‌شده و پاسخ «چرا»هایی که معمولاً بی‌جواب می‌مانند.

۱ طراحی و تحلیل الگوریتم

۲ تقسیم و حل

۳ الگوریتم‌های تصادفی

۴ تحلیل سرشکنی



چرا تحلیل الگوریتم مهم است؟ با بزرگ‌شدن n، فاصلهٔ این منحنی‌ها آن‌قدر زیاد می‌شود که هیچ سخت‌افزار سریع‌تری جبرانش نمی‌کند.

پیش از شروع: این جزوه را چطور بخوانیم؟

اسلایدهای درس معمولاً فقط تیترها و فرمول‌ها را دارند و جای خالیِ «چرا این‌طور است؟» و «این را از کجا آوردیم؟» را به کلاس می‌سپارند. این جزوه همان جاهای خالی را پر می‌کند. ترتیب مطالب همان ترتیب اسلایدهاست، اما هر جا اسلاید از یک گام پرید، گام را کامل نوشته‌ایم. مطالبی هم که در اسلاید نبود ولی برای فهم عمیق‌تر یا امتحان لازم است، در کادرهای صورتیِ «تکمیلی» آمده‌اند تا مرز میان «اسلاید» و «افزوده» همیشه روشن باشد.

هر نوع کادر رنگ و معنای ثابتی دارد:

تعریف	تعریف دقیق یک مفهوم	**قضیه / لم**	حکمی که باید ثابت شود
اثبات	استدلال گام‌به‌گام	**مثال**	مثال حل‌شده با عدد
شهود	تصویر ذهنی پیش از فرمول	**چرا؟**	پاسخ به پرسش‌های رایج دانشجو
نکته	جزئیاتی که نباید از قلم بیفتد	**اشتباه رایج**	دام‌های امتحانی
تکمیلی	فراتر از اسلاید، برای فهم عمیق‌تر	**خودآزمایی**	سؤال تمرینی همراه پاسخ

نکته · قرارداد نمادها

در سراسر جزوه، $\log n$ بدون پایه یعنی $\log_2 n$ و $\ln n$ یعنی لگاریتم طبیعی. آرایه‌ها از اندیس ۱ شروع می‌شوند (مثل $A[1..n]$) مگر خلافش گفته شود. عبارت «زمان اجرا» یعنی تعداد عملیات پایه‌ای که الگوریتم انجام می‌دهد، نه ثانیه. شبه‌کدها چپ‌به‌راست نوشته شده‌اند و شمارهٔ خط دارند تا بتوانیم در متن به آن‌ها ارجاع دهیم.

فهرست مطالب

فصل ۱ طراحی و تحلیل الگوریتم‌ها

۱.۱ حل مسئله با الگوریتم
۱.۲ الگوهای طراحی و کاربردها
۱.۳ تحلیل الگوریتم چیست؟
۱.۴ چه چیزی زمان اجرا را تعیین می‌کند؟
۱.۵ چطور تحلیل کنیم؟ شمارش عملیات پایه
۱.۶ مسئلهٔ مرتب‌سازی و مرتب‌سازی درجی
۱.۷ انواع تحلیل: بدترین، میانگین، بهترین
۱.۸ تحلیل دقیق مرتب‌سازی درجی
۱.۹ مقایسهٔ توابع با نرخ رشد
۱.۱۰ نمادهای مجانبی O، Ω و Θ
۱.۱۱ نکته‌های نمادگذاری و ویژگی‌ها
۱.۱۲ کران‌های مجانبی توابع رایج
۱.۱۳ زمان اجرای دستورهای مختلف
۱.۱۴ الگوریتم‌های چندجمله‌ای و اهمیت آن‌ها
۱.۱۵ مثال اول: اعداد فیبوناچی
۱.۱۶ مثال دوم: زیرآرایه با بیشترین مجموع
۱.۱۷ مروری بر زمان‌های اجرای رایج
۱.۱۸ خودآزمایی فصل ۱

فصل ۲ تقسیم و حل

۲.۱ ایدهٔ تقسیم و حل
۲.۲ مرتب‌سازی ادغامی
۲.۳ رابطهٔ بازگشتی و درخت بازگشت
۲.۴ درخت بازگشت در حالت کلی
۲.۵ قضیهٔ اصلی
۲.۶ اگر آرایه را به بیش از دو بخش تقسیم کنیم؟
۲.۷ زیرآرایهٔ بیشینه با تقسیم و حل
۲.۸ جمع و ضرب اعداد صحیح
۲.۹ ضرب کاراتسوبا
۲.۱۰ روش حدس و استقرا
۲.۱۱ خودآزمایی فصل ۲

فصل ۳ الگوریتم‌های تصادفی

۳.۱ تصادف در طراحی الگوریتم
۳.۲ فضای احتمال متناهی
۳.۳ متغیر تصادفی
۳.۴ امید ریاضی و خطی بودن آن
۳.۵ متغیر نشانگر و جایگشت تصادفی
۳.۶ میانهٔ تقریبی تصادفی
۳.۷ زمان اجرای الگوریتم‌های قطعی و تصادفی
۳.۸ مونت‌کارلو در برابر لاس‌وگاس

۳.۹ مسئلهٔ استخدام
۳.۱۰ انتخاب تصادفی
۳.۱۱ انتخاب قطعی: میانهٔ میانه‌ها
۳.۱۲ کوئیک‌سورت تصادفی
۳.۱۳ خودآزمایی فصل ۳

فصل ۴ تحلیل سرشکنی

۴.۱ سه نگاه به هزینه
۴.۲ مثال ۱: پشته با عمل multipop
۴.۳ مثال ۲: شمارندهٔ دودویی
۴.۴ روش‌های تحلیل سرشکنی
۴.۵ روش انبوهه
۴.۶ روش حساب‌داری
۴.۷ روش تابع پتانسیل
۴.۸ مثال تکمیلی: آرایهٔ پویا
۴.۹ مقایسهٔ سه روش
۴.۱۰ خودآزمایی فصل ۴

پیوست پیوست: برگهٔ خلاصه، واژه‌نامه و منابع

پ.۱ فرمول‌های پرکاربرد
پ.۲ جدول الگوریتم‌های جزوه
پ.۳ واژه‌نامهٔ فارسی–انگلیسی
پ.۴ منابع

فصل ۱

طراحی و تحلیل الگوریتم‌ها

یاد می‌گیریم چطور سرعت یک الگوریتم را مستقل از کامپیوتر و زبان برنامه‌نویسی بسنجیم، و چرا یک ایدهٔ بهتر از هزار برابر سخت‌افزار سریع‌تر ارزش دارد.

مقدمه | اعداد فیبوناچی | زیرآرایه با بیشترین مجموع | مرتبهٔ رشد مجانبی | زمان‌های اجرای رایج

۱.۱ حل مسئله با الگوریتم

هر بار که با کامپیوتر مسئله‌ای را حل می‌کنیم، چهار مرحله را پشت سر می‌گذاریم، حتی اگر حواسمان به آن‌ها نباشد:

۱. **مسئله:** دقیقاً مشخص می‌کنیم ورودی چیست و خروجی باید چه ویژگی‌ای داشته باشد. مسئلهٔ مبهم، الگوریتم درست ندارد.

۲. **طراحی الگوریتم:** یک دستورالعمل گام‌به‌گام، متناهی و بی‌ابهام می‌نویسیم که برای **هر** ورودی مجاز، خروجی درست را تولید کند.

۳. **تحلیل الگوریتم:** ثابت می‌کنیم الگوریتم درست کار می‌کند و می‌سنجیم چقدر منابع (زمان و حافظه) مصرف می‌کند.

۴. **برنامه‌نویسی:** الگوریتم را به زبان یک ماشین واقعی ترجمه می‌کنیم.

تمرکز این درس روی مرحلهٔ دوم و سوم است. شاید بپرسید چرا مستقیم سراغ کدنویسی نرویم و بعد برنامه را اجرا نکنیم تا ببینیم سریع است یا نه؟ پاسخ در کادر زیر است.

چرا؟ · چرا قبل از کدنویسی تحلیل کنیم؟

اول، اجرای برنامه فقط دربارهٔ ورودی‌هایی که امتحان کرده‌ایم چیزی می‌گوید؛ ممکن است ورودی بدی وجود داشته باشد که آن را نیازموده‌ایم. دوم، نوشتن و اشکال‌زدایی برنامه گران است؛ اگر از روی کاغذ بفهمیم ایده‌ای برای ورودی‌های بزرگ چند هزار سال طول می‌کشد، از همان اول سراغش نمی‌رویم. سوم، تحلیل به ما می‌گوید **کجای** الگوریتم گلوگاه است و کجا ارزش بهینه‌سازی دارد. در این فصل با چند مثال می‌بینیم که تغییر ایده (نه تغییر سخت‌افزار) زمان را از «میلیون‌ها سال» به «چند میکروثانیه» می‌رساند.

۱.۲ الگوهای طراحی و کاربردها

الگوریتم‌ها هزاران شکل دارند، اما بیشترشان از چند **الگوی طراحی** (algorithmic paradigms) پیروی می‌کنند. الگو یعنی یک راهبرد کلی که برای دستهٔ بزرگی از مسائل جواب می‌دهد. شناختن این الگوها یعنی وقتی با مسئلهٔ تازه‌ای روبه‌رو می‌شوید، با دست خالی شروع نمی‌کنید:

الگو	ایدهٔ اصلی	نمونه
حریصانه (Greedy)	در هر گام بهترین انتخابِ همان لحظه را می‌کنیم و هرگز به عقب برنمی‌گردیم.	درخت پوشای کمینه
تقسیم و حل (Divide & Conquer)	مسئله را به چند زیرمسئلهٔ کوچک‌تر می‌شکنیم، هر کدام را بازگشتی حل می‌کنیم و جواب‌ها را ترکیب می‌کنیم.	مرتب‌سازی ادغامی (فصل ۲)
برنامه‌ریزی پویا (Dynamic Programming)	جواب زیرمسئله‌های هم‌پوشان را یک بار حساب و ذخیره می‌کنیم تا دوباره‌کاری نشود.	فیبوناچی با حلقه (همین فصل)
شارش شبکه (Network Flow)	مسئله را به عبور «جریان» در یک گراف با ظرفیت‌های محدود تبدیل می‌کنیم.	تطابق بیشینه
تصادفی (Randomized)	الگوریتم خودش سکه می‌اندازد تا از ورودی‌های بد فرار کند یا ساده‌تر شود.	کوئیک‌سورت تصادفی (فصل ۳)

دامنهٔ کاربرد الگوریتم‌ها تقریباً بی‌انتهاست: حافظهٔ نهان (caching)، کامپایلرها، پایگاه‌های داده، زمان‌بندی، شبکه، تحلیل داده، پردازش سیگنال، گرافیک کامپیوتری، محاسبات علمی، تحقیق در عملیات، هوش مصنوعی و زیست‌شناسی محاسباتی، تنها بخشی از آن‌اند. در همهٔ این حوزه‌ها، تفاوت میان الگوریتم خوب و بد، تفاوت میان «قابل استفاده» و «غیرقابل استفاده» است.

۱.۳ تحلیل الگوریتم چیست؟

تعریف · تحلیل الگوریتم

تحلیل به مجموعهٔ روش‌های ریاضی گفته می‌شود که با آن‌ها دو چیز را ثابت می‌کنیم: **درستی** (correctness) یعنی الگوریتم برای هر ورودی مجاز خروجی درست می‌دهد، و **کارایی** (efficiency) یعنی الگوریتم چه مقدار منبع (معمولاً زمان) مصرف می‌کند.

دربارهٔ کارایی یک الگوریتم A معمولاً دو پرسش داریم:

- **زمان اجرای A چقدر است؟** این را با یک تابع از اندازهٔ ورودی بیان می‌کنیم.
- **آیا A کاراترین الگوریتم ممکن برای این مسئله است؟** برای پاسخ باید یک **کران پایین** پیدا کنیم: ثابت کنیم **هیچ** الگوریتمی نمی‌تواند این مسئله را سریع‌تر از فلان مقدار حل کند. اگر زمان A با این کران پایین برابر باشد، A بهینه است. مثلاً در فصل ۲

می‌بینیم که جمع دو عدد n بیتی حداقل $\Omega(n)$ زمان می‌خواهد (چون باید همهٔ بیت‌ها را دست‌کم یک بار دید)، پس روش جمع دبستانی بهینه است.

۱.۴ چه چیزی زمان اجرا را تعیین می‌کند؟

زمان اجرای یک برنامه به عوامل زیادی بستگی دارد:

نکته · عوامل بیرونی

سخت‌افزار (سرعت پردازنده، حافظهٔ نهان) و **کامپایلر** (کیفیت بهینه‌سازی). این‌ها ربطی به ایدهٔ الگوریتم ندارند و معمولاً زمان را فقط در یک **ضریب ثابت** ضرب می‌کنند.

نکته · عوامل مربوط به ورودی

اندازهٔ ورودی (مثلاً تعداد عناصر آرایه) و **چیدمان ورودی** (مثلاً آرایه از قبل مرتب است یا برعکس). این‌ها همان چیزی‌اند که تحلیل ما باید دربارهٔ آن‌ها حرف بزند.

تعریف · تابع زمان اجرا

$T(n)$ یعنی زمان اجرای الگوریتم وقتی اندازهٔ ورودی n است. گاهی زمان به بیش از یک متغیر وابسته است؛ مثلاً برای گرافی با n رأس و m یال، زمان را با $T(n, m)$ نشان می‌دهیم.

نکته · «اندازهٔ ورودی» یعنی چه؟

اندازهٔ ورودی تعداد «واحد»های اطلاعاتی ورودی است و به مسئله بستگی دارد: در مرتب‌سازی تعداد عناصر، در ضرب دو عدد بزرگ **تعداد بیت‌ها** (نه خود عدد!)، و در گراف تعداد رأس‌ها و یال‌ها. این تمایز در فصل ۲ (ضرب اعداد صحیح) اهمیت پیدا می‌کند.

۱.۵ چطور تحلیل کنیم؟ شمارش عملیات پایه

برای اندازه‌گیری کارایی دو راه به ذهن می‌رسد:

اشتباه رایج · راه اول: زمان‌سنجی

برنامه را برای اندازه‌های مختلف ورودی اجرا کنیم و با ساعت زمان بگیریم. **خوب نیست**، چون عدد به‌دست‌آمده مخصوص همان کامپیوتر و همان کامپایلر است و دربارهٔ ورودی‌هایی که امتحان نکرده‌ایم چیزی نمی‌گوید.

مثال · راه دوم: شمارش عملیات

تعداد **عملیات پایه** (primitive operations) را بشماریم: جمع $(A = B + C)$، ضرب، افزایش $(A = A + 1)$، انتساب، مقایسه $(A < B)$، عملیات منطقی و ... . **خوب است**، ولی هر عمل پایه زمان متفاوتی دارد.

برای ساده شدن کار، راه دوم را با یک فرض انتخاب می‌کنیم:

تعریف · فرض هزینهٔ واحد

هر عمل پایه دقیقاً ۱ واحد زمان می‌برد. با این فرض، وابستگی به سرعت کامپیوتر حذف می‌شود؛ بدون آن، مقایسه و حتی وارسی ادعاهای ما ناممکن بود.

چرا؟ · این فرض که آشکارا غلط است! چرا مجاز است؟

درست است که ضرب از جمع کندتر است. اما اگر کندترین عمل پایه k برابر سریع‌ترین باشد، زمان واقعی حداکثر k برابر عدد شمرده‌شده است. k یک **ثابت** است و به n بستگی ندارد. در ادامه می‌بینیم که در تحلیل مجانبی ضرایب ثابت را عمداً نادیده می‌گیریم، پس این فرض هیچ چیزِ مهمی را خراب نمی‌کند. آنچه مهم است **شکل رشد** تعداد عملیات با n است.

این فرض فقط وقتی می‌شکند که اعداد بی‌اندازه بزرگ شوند (مثلاً جمع دو عدد هزاررقمی دیگر «یک عمل» نیست). در چنین جاهایی، مثل ضرب اعداد صحیح در فصل ۲، به‌جای عمل روی کل عدد، **عمل روی بیت‌ها** را می‌شماریم.

۱.۶ مسئلهٔ مرتب‌سازی و مرتب‌سازی درجی

تعریف · مسئلهٔ مرتب‌سازی

ورودی: دنبالهٔ $\langle a_1, a_2, \ldots, a_n \rangle$ از اعداد.

خروجی: جایگشتی $\langle a'_1, a'_2, \ldots, a'_n \rangle$ از همان اعداد به‌طوری که $a'_1 \le a'_2 \le \cdots \le a'_n$.

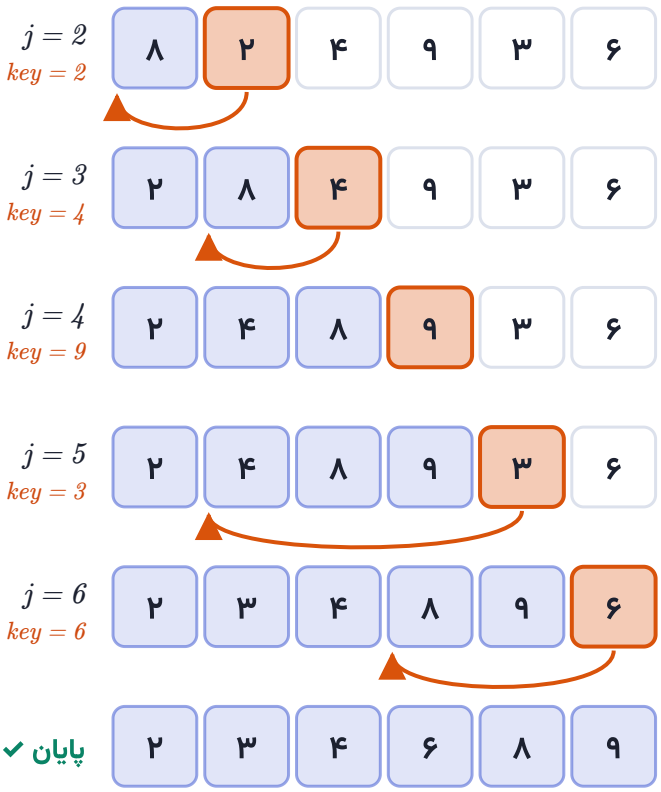
مثال: ورودی $6\ 3\ 9\ 4\ 2\ 8$ و خروجی $2\ 3\ 4\ 6\ 8\ 9$.

شهود · مرتب کردن ورق‌های بازی

مرتب‌سازی درجی (Insertion Sort) همان کاری است که موقع مرتب کردن ورق‌های بازی در دست انجام می‌دهید: ورق‌های دست چپ همیشه مرتب‌اند؛ ورق تازه را برمی‌دارید، از راست به چپ با ورق‌های دستتان مقایسه می‌کنید، ورق‌های بزرگ‌تر را یکی‌یکی کنار می‌زنید و ورق تازه را در جای درستش **درج** می‌کنید.

```
INSERTION-SORT(A, n) ▷ A[1..n]
1  for j = 2 to n
2      key = A[j]                      // ورق تازه
3      i = j - 1
4      while i > 0 and A[i] > key      // بزرگ‌ترها را یکی به راست ببر
5          A[i+1] = A[i]
6          i = i - 1
7      A[i+1] = key                    // ورق را در جای خالی بگذار
```

در هر دور حلقهٔ بیرونی، بخش $A[1..j-1]$ مرتب است و $A[j]$ (یعنی key) باید در آن درج شود. حلقهٔ `while` از انتهای بخش مرتب به سمت چپ حرکت می‌کند و هر عنصرِ بزرگ‌تر از key را یک خانه به راست می‌برد. وقتی به عنصری کوچک‌تر یا مساوی رسید (یا به ابتدای آرایه)، خانهٔ $i+1$ خالی شده است و key آنجا می‌نشیند.



شکل ۱-۱ اجرای مرتب‌سازی درجی روی 6 3 9 4 2 8. خانه‌های آبی بخش مرتب‌شده‌اند، خانهٔ نارنجی key است و پیکان نشان می‌دهد key کجا درج می‌شود. در گام $j = 4$ عدد ۹ از همهٔ قبلی‌ها بزرگ‌تر است و جابه‌جا نمی‌شود.

چرا این الگوریتم درست است؟ ناوردای حلقه

برای اثبات درستی الگوریتم‌های حلقه‌ای از ابزاری به نام **ناوردای حلقه** (loop invariant) استفاده می‌کنیم: گزاره‌ای که در شروع هر دور حلقه برقرار است.

قضیه · ناوردای مرتب‌سازی درجی

در شروع هر دور حلقهٔ `for` با شمارندهٔ j، زیرآرایهٔ $A[1..j-1]$ شامل همان عناصری است که در ابتدا در $A[1..j-1]$ بودند، ولی به ترتیب صعودی.

آغاز: وقتی $j = 2$ است، $A[1..1]$ فقط یک عنصر دارد و آرایهٔ یک‌عنصری خودبه‌خود مرتب است.

نگهداری: فرض کنید ناوردا در شروع دور j برقرار باشد. حلقهٔ `while` عناصر $A[j-1], A[j-2], \dots$ را که از key بزرگ‌ترند یک خانه به راست می‌برد و key را درست بعد از اولین عنصرِ نابزرگ‌تر قرار می‌دهد. پس $A[1..j]$ همان عناصر قبلی به‌اضافهٔ key است و مرتب است؛ یعنی ناوردا در شروع دور $j+1$ برقرار است.

پایان: حلقه وقتی تمام می‌شود که $j = n+1$. با جای‌گذاری در ناوردا، $A[1..n]$ مرتب است؛ یعنی کل آرایه مرتب شده است.

∎

نکته · ناوردای حلقه = استقرای ریاضی

«آغاز» همان پایهٔ استقراست، «نگهداری» همان گام استقرا، و «پایان» جایی است که از ناوردا نتیجهٔ دلخواه را بیرون می‌کشیم. تفاوتش با استقرای معمولی این است که استقرا تا بی‌نهایت ادامه دارد ولی حلقه یک جا متوقف می‌شود.

۱.۷ انواع تحلیل: بدترین، میانگین، بهترین

زمان مرتب‌سازی درجی فقط به n بستگی ندارد؛ به چیدمان ورودی هم وابسته است. اگر آرایه از قبل مرتب باشد حلقهٔ `while` هرگز اجرا نمی‌شود، اما اگر برعکس مرتب باشد هر عنصر باید تا ابتدای آرایه برود. پس باید مشخص کنیم «زمان اجرا» دقیقاً کدام عدد است:

بدترین حالت $T(n)$ = **بیشترین** زمان الگوریتم روی **هر** ورودی به اندازهٔ n. معمولاً همین را می‌خواهیم

حالت میانگین $T(n)$ = **امید ریاضی** زمان روی همهٔ ورودی‌های اندازهٔ n. نیازمند فرضی دربارهٔ **توزیع آماری** ورودی‌هاست. گاهی

بهترین حالت $T(n)$ = **کمترین** زمان روی ورودی‌های اندازهٔ n. تقریباً هرگز

چرا؟ · چرا بدترین حالت؟ و چرا بهترین حالت «تقلب» است؟

بدترین حالت یک **تضمین** است: مهم نیست ورودی چه باشد، الگوریتم از این کندتر نمی‌شود. برای سیستمی که باید همیشه پاسخ بدهد (مثلاً کنترل ترمز خودرو) همین تضمین ارزش دارد. به‌علاوه، در بسیاری از الگوریتم‌ها بدترین حالت زیاد پیش می‌آید (مثلاً جست‌وجوی چیزی که در پایگاه داده نیست).

حالت میانگین مفید است، اما «میانگین روی چه توزیعی؟» اگر فرض کنیم همهٔ ورودی‌ها هم‌احتمال‌اند و در عمل ورودی‌ها اغلب تقریباً مرتب باشند، نتیجهٔ ما بی‌معناست.

بهترین حالت تقریباً هیچ چیز نمی‌گوید: هر الگوریتم کُندی را می‌توان طوری دستکاری کرد که روی **یک** ورودی خاص سریع باشد (مثلاً اول بررسی کند «آیا ورودی همان آرایهٔ مرتبِ معروف است؟» و اگر بود فوراً جواب بدهد). این یعنی با بهترین حالت می‌شود سرِ هر کسی کلاه گذاشت.

۱.۸ تحلیل دقیق مرتب‌سازی درجی

حالا با فرض هزینهٔ واحد، تعداد اجرای هر خط را می‌شماریم. فرض کنید t_j تعداد دفعاتی باشد که **شرطِ** `while` برای مقدار j بررسی می‌شود (یک بار بیشتر از تعداد جابه‌جایی‌ها، چون آخرین بررسی شکست می‌خورد).

خط	دستور	هزینه	تعداد اجرا
۱	`for j = 2 to n`	c_1	n
۲	`key = A[j]`	c_2	$n-1$
۳	`i = j - 1`	c_3	$n-1$
۴	`while ...`	c_4	$\sum_{j=2}^{n} t_j$
۵	`A[i+1] = A[i]`	c_5	$\sum_{j=2}^{n} (t_j - 1)$
۶	`i = i - 1`	c_6	$\sum_{j=2}^{n} (t_j - 1)$
۷	`A[i+1] = key`	c_7	$n-1$

(خط `for` یک بار بیشتر از بدنه اجرا می‌شود، چون آخرین بار شرطش بررسی و رد می‌شود.) پس:

$$T(n) = c_1 n + (c_2 + c_3 + c_7)(n-1) + c_4 \sum_{j=2}^{n} t_j + (c_5 + c_6) \sum_{j=2}^{n} (t_j - 1).$$

بهترین حالت: آرایهٔ مرتب

اگر آرایه از قبل مرتب باشد، برای هر j اولین مقایسهٔ $A[j-1] > key$ نادرست است، پس $t_j = 1$ و جملهٔ $\sum (t_j - 1)$ صفر می‌شود:

$$T(n) = c_1 n + (c_2 + c_3 + c_4 + c_7)(n-1) = an + b \;=\; \Theta(n).$$

یعنی زمان **خطی** است. (همان $\sum_{j=2}^{n} 1$ که در اسلاید آمده.)

بدترین حالت: آرایهٔ معکوس

اگر آرایه نزولی باشد، key از همهٔ عناصر $A[1..j-1]$ کوچک‌تر است و باید با همه مقایسه شود، پس $t_j = j$. با فرمول جمع حسابی:

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1, \qquad \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}.$$

با جای‌گذاری، $T(n) = an^2 + bn + c$ برای ثابت‌های a, b, c به دست می‌آید، یعنی $T(n) = \Theta(n^2)$: زمان **درجهٔ دو**.

حالت میانگین: همهٔ جایگشت‌ها هم‌احتمال

اگر همهٔ $n!$ جایگشت ورودی هم‌احتمال باشند، key به‌طور متوسط از نیمی از عناصر $A[1..j-1]$ کوچک‌تر است؛ پس به‌طور متوسط $t_j \approx j/2$. جمع $\sum_{j=2}^{n} j/2 \approx n^2/4$ است که باز هم $\Theta(n^2)$ است. یعنی **حالت میانگین فقط حدود نصفِ بدترین حالت است و مرتبه‌اش عوض نمی‌شود.**

جمع‌بندی · مرتب‌سازی درجی

بدترین حالت $\Theta(n^2)$ · حالت میانگین $\Theta(n^2)$ · بهترین حالت $\Theta(n)$. برای nهای کوچک یا آرایه‌های تقریباً مرتب سریع است، ولی برای nهای بزرگ نه.

نکته · از ثابت‌ها خلاص شدیم!

در محاسبهٔ بالا ثابت‌های $c_1, \dots, c_7$ را نمی‌دانستیم، اما نتیجهٔ نهایی ($\Theta(n^2)$) به آن‌ها بستگی نداشت. این دقیقاً همان چیزی است که می‌خواستیم: نتیجه‌ای مستقل از سخت‌افزار. ابزار رسمی این «دور ریختن ثابت‌ها» نمادهای مجانبی‌اند که در ادامه می‌آیند.

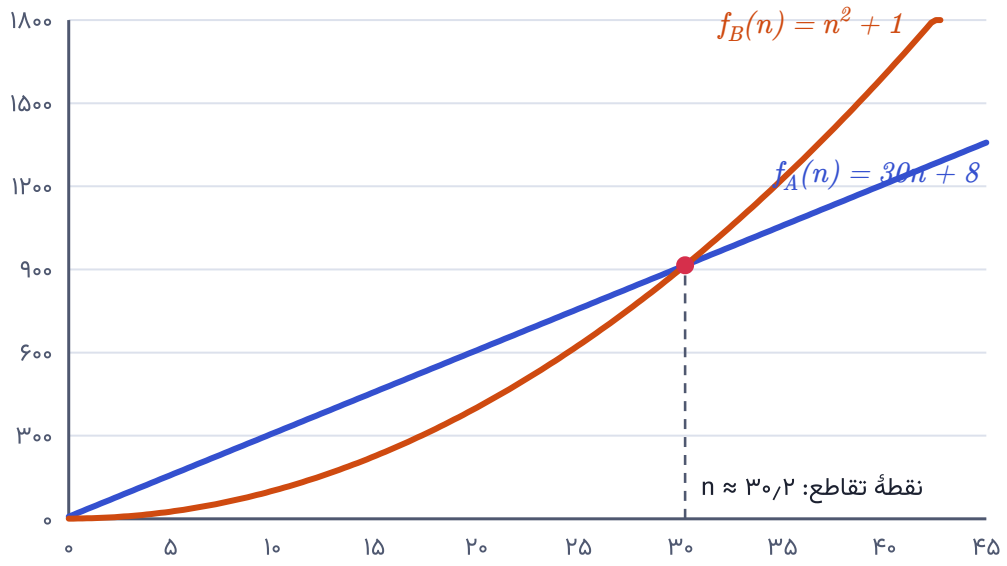
۱.۹ مقایسهٔ توابع با نرخ رشد

وقتی می‌گوییم «تابع f از g سریع‌تر رشد می‌کند»، منظورمان **رفتار مجانبی** است: برای nهای به‌اندازهٔ کافی بزرگ، $f(n)$ سرانجام از $g(n)$ بیشتر می‌شود و دیگر هرگز کمتر نمی‌شود، هر قدر هم که در ابتدا کوچک‌تر بوده باشد. به همین دلیل به این نوع مقایسه **تحلیل مجانبی** (asymptotic analysis) می‌گوییم.

مثال · کدام برنامه را انتخاب کنیم؟

فرض کنید سایتی برای پردازش داده‌های کاربران (مثلاً سوابق مالی) طراحی می‌کنید. برنامهٔ A برای پردازش n رکورد $f_A(n) = 30n + 8$ میکروثانیه و برنامهٔ B $f_B(n) = n^2 + 1$ میکروثانیه زمان می‌برد. اگر بدانید روزی میلیون‌ها کاربر خواهید داشت، کدام را انتخاب می‌کنید؟

برای nهای کوچک B سریع‌تر است (مثلاً $n = 10$: $f_A = 308$ ولی $f_B = 101$). اما $f_B(n) > f_A(n)$ یعنی $n^2 - 30n - 7 > 0$، که با حل معادلهٔ درجه دو برای $n \geq 31$ برقرار است. از آنجا به بعد B همیشه کندتر است و فاصله مدام بیشتر می‌شود: برای یک میلیون رکورد، A حدود ۳۰ ثانیه و B حدود 10^{12} میکروثانیه، یعنی بیش از ۱۱ روز طول می‌کشد. **پس A را انتخاب می‌کنیم.**



شکل ۱-۲ تابعی که سریع‌تر رشد می‌کند، دیر یا زود از دیگری بالاتر می‌رود. ضریب ۳۰ فقط محل تقاطع را جابه‌جا می‌کند، نه نتیجهٔ نهایی را.

می‌گوییم $f_A(n) = 30n + 8$ «از مرتبهٔ n» است و می‌نویسیم $O(n)$: حداکثر تقریباً متناسب با n رشد می‌کند. همین‌طور $f_B(n) = n^2 + 1$ از مرتبهٔ n^2 یعنی $O(n^2)$ است. به‌طور کلی، یک الگوریتم $O(n^2)$ برای ورودی‌های بزرگ از یک الگوریتم $O(n)$ کندتر است.

۱.۱۰ نمادهای مجانبی O، Ω و Θ

حالا همین ایده‌ها را دقیق می‌کنیم. در همهٔ تعریف‌ها f و g توابعی از اعداد طبیعی به اعداد نامنفی‌اند (زمان اجرا هرگز منفی نیست).

تعریف · کران بالا: O بزرگ

$f(n)$ از $O(g(n))$ است اگر ثابت‌های $c > 0$ و $n_0 \geq 0$ وجود داشته باشند به‌طوری که

$$\forall n \geq n_0 : \quad f(n) \leq c \cdot g(n).$$

تعریف · کران پایین: امگای بزرگ

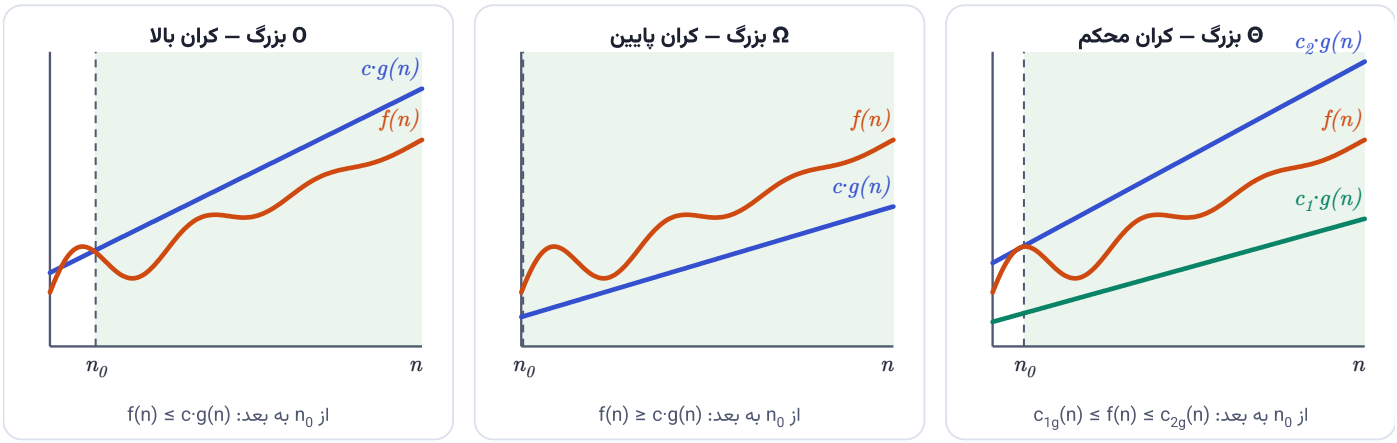
$f(n)$ از $\Omega(g(n))$ است اگر ثابت‌های $c > 0$ و $n_0 \geq 0$ وجود داشته باشند به‌طوری که

$$\forall n \geq n_0 : \quad f(n) \geq c \cdot g(n).$$

تعریف · کران محکم: تتای بزرگ

$f(n)$ از $\Theta(g(n))$ است اگر هم $O(g(n))$ باشد و هم $\Omega(g(n))$. معادلش: ثابت‌های $c_1, c_2 > 0$ و n_0 وجود دارند به‌طوری که

$$\forall n \geq n_0 : \quad c_1\, g(n) \leq f(n) \leq c_2\, g(n).$$



شکل ۱-۳ ناحیهٔ سبز جایی است که تعریف باید برقرار باشد. قبل از n_0 هر اتفاقی ممکن است بیفتد و برای ما مهم نیست.

چرا؟ · چرا c و چرا n_0؟

ثابت c همان ابزار «دور ریختن ضرایب» است: $30n$ و n باید هم‌مرتبه حساب شوند، پس اجازه می‌دهیم g را در هر ثابتی که لازم است ضرب کنیم. **آستانهٔ** n_0 ابزار «مجانبی بودن» است: رفتار تابع برای nهای کوچک اهمیت ندارد، پس می‌گوییم «از جایی به بعد». بدون n_0، مثلاً $n^2 = O(n^2 - 5)$ نادرست می‌شد، چون برای $n = 2$ سمت راست منفی است.

مثال · $f(n) = 32n^2 + 17n + 32$

ادعا: $f(n) = O(n^2)$. برای $n \geq 1$ داریم $17n \leq 17n^2$ و $32 \leq 32n^2$، پس

$$f(n) \leq 32n^2 + 17n^2 + 32n^2 = 81\, n^2.$$

یعنی $c = 81$ و $n_0 = 1$ کار می‌کند. **ادعا:** $f(n) = \Omega(n^2)$. برای همهٔ n داریم $f(n) \geq 32n^2$، پس $c = 32$ کافی است. در نتیجه $f(n) = \Theta(n^2)$.

از همین‌ها نتیجه می‌شود:

- $f(n)$ از $O(n^2)$، $O(n^3)$، $\Omega(n^2)$، $\Omega(n)$ و $\Theta(n^2)$ **است** (چون $n^2 \le n^3$ و $n^2 \ge n$ برای $n \ge 1$).
- $f(n)$ از $O(n)$، $\Omega(n^3)$، $\Theta(n)$ و $\Theta(n^3)$ **نیست**.

مثلاً چرا $f(n) \neq O(n)$؟ فرض کنید c و n_0 وجود داشته باشند که $32n^2 + 17n + 32 \le cn$. آنگاه $32n^2 \le cn$، یعنی $n \le c/32$ برای **همهٔ** $n \ge n_0$، که ممکن نیست چون n می‌تواند هر قدر بخواهد بزرگ شود. تناقض.

تکمیلی (فراتر از اسلاید) · آزمون حد و نمادهای o و ω کوچک

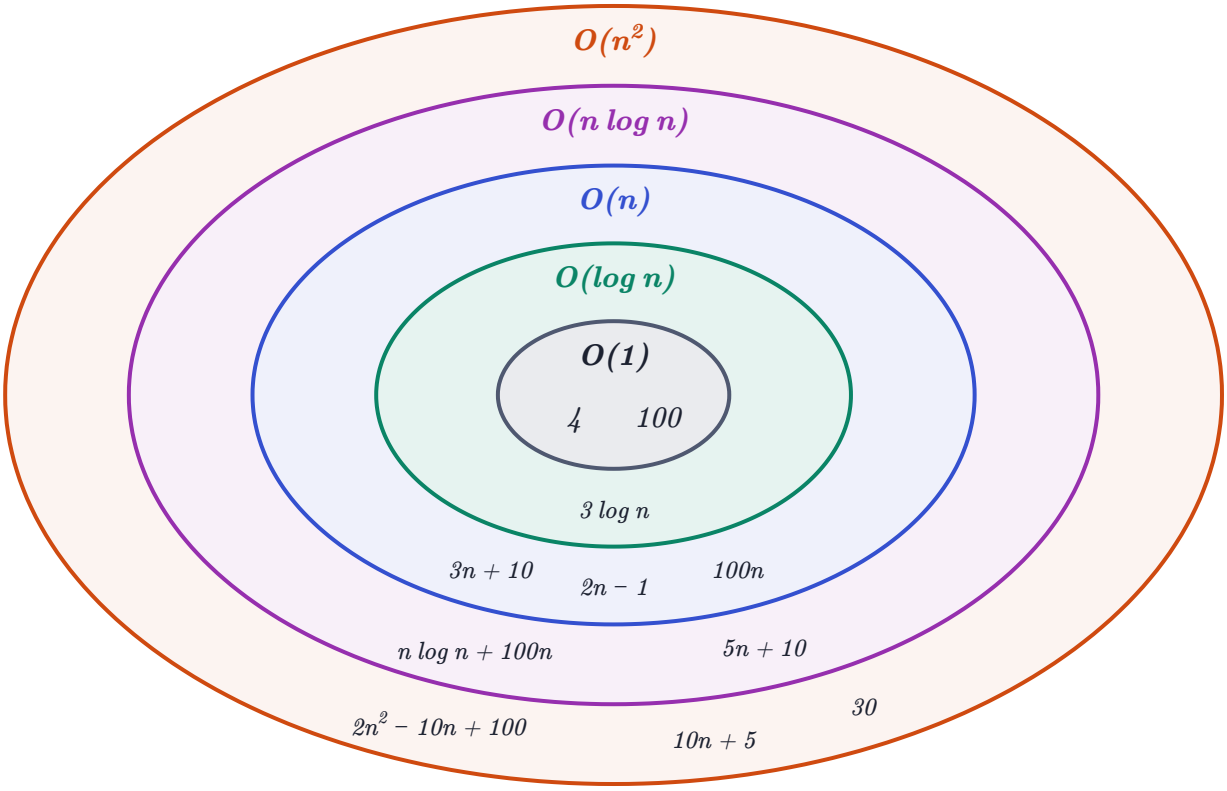
در عمل، ساده‌ترین راه مقایسهٔ دو تابع محاسبهٔ حد نسبت آن‌هاست. اگر $L = \lim_{n\to\infty} f(n)/g(n)$ وجود داشته باشد:

- اگر $0 < L < \infty$، آنگاه $f = \Theta(g)$.
- اگر $L = 0$، آنگاه $f = O(g)$ ولی $f \neq \Omega(g)$. در این حالت می‌نویسیم $f = o(g)$ (o کوچک: «اکیداً کندتر رشد می‌کند»).
- اگر $L = \infty$، آنگاه $f = \Omega(g)$ ولی $f \neq O(g)$. می‌نویسیم $f = \omega(g)$.

مثلاً $\lim \frac{32n^2+17n+32}{n^2} = 32$، پس $\Theta(n^2)$. برای حدهای سخت‌تر از قاعدهٔ هوپیتال استفاده می‌کنیم.

نماد O یک «مجموعه» است

دقیق‌ترین برداشت از $O(g(n))$ این است: **مجموعهٔ همهٔ توابعی که از g سریع‌تر رشد نمی‌کنند.** مثلاً $3n + 10$ شامل $O(n)$، $2n - 1$ و $100n$ است؛ ولی شامل ثابت‌هایی مثل 3 یا 100 هم هست، چون ثابت‌ها هم از n سریع‌تر رشد نمی‌کنند. به همین ترتیب $10n + 5$ و $100n$ و حتی عدد 30 عضو $O(n^2)$ هستند. نتیجه این‌که این مجموعه‌ها تودرتو هستند:



شکل ۱-۴ $O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2)$

. هر تابع داخل یک حلقه، عضو همهٔ حلقه‌های بیرونی آن هم هست.

اشتباه رایج · «O یعنی دقیقاً همین مرتبه»؟ نه!

O فقط کران بالا است. جملهٔ «زمان اجرای مرتب‌سازی درجی $O(n^3)$ است» **درست** است، فقط دقیق نیست. اگر بخواهید بگویید مرتبه دقیقاً همین است، باید از Θ استفاده کنید. در امتحان، وقتی سؤال «مرتبهٔ زمانی» را می‌خواهد، بهترین پاسخ Θ است.

۱.۱۱ نکته‌های نمادگذاری و ویژگی‌ها

سوءاستفادهٔ مجاز از علامت مساوی

همه می‌نویسیم $f(n) = O(g(n))$، اما این «مساوی» مساوی واقعی نیست؛ **یک‌طرفه** است. مثال اسلاید: اگر $f(n) = 5n^3$ و $g(n) = 3n^2$، آنگاه $f(n) = O(n^3)$ و $g(n) = O(n^3)$؛ اگر این‌ها مساوی واقعی بودند، با تعدی نتیجه می‌گرفتیم $f(n) = g(n)$، که غلط است. نمادگذاری دقیق‌تر $f(n) \in O(g(n))$ است، و $=$ را باید «عضوِ» خواند.

اشتباه رایج · جملهٔ بی‌معنا

«هر الگوریتم مرتب‌سازیِ مبتنی بر مقایسه **حداقل** $O(n \log n)$ مقایسه لازم دارد.» این جمله بی‌معناست، چون O کران بالاست و «حداقلِ یک کران بالا» هیچ اطلاعاتی نمی‌دهد (هر تابعی، حتی صفر، از «حداقل O» کوچک‌تر نیست!). شکل درست: «... به $\Omega(n \log n)$ مقایسه نیاز دارد.» **برای کران پایین از Ω استفاده کنید.**

ویژگی‌ها

قضیه · تعدی و جمع‌پذیری

تعدی: اگر $f = O(g)$ و $g = O(h)$ آنگاه $f = O(h)$. همین برای Ω و Θ هم برقرار است.

جمع‌پذیری: اگر $f = O(h)$ و $g = O(h)$ آنگاه $f + g = O(h)$. اگر $f = \Omega(h)$ و $g = \Omega(h)$ آنگاه $f + g = \Omega(h)$. و اگر $f = \Theta(h)$ و $g = O(h)$ آنگاه $f + g = \Theta(h)$.

تعدی: ثابت‌های c_1, n_1 و c_2, n_2 وجود دارند که برای $n \geq n_1$، $f(n) \leq c_1 g(n)$ و برای $n \geq n_2$، $g(n) \leq c_2 h(n)$. پس برای $n \geq \max(n_1, n_2)$ داریم $f(n) \leq c_1 c_2\, h(n)$. ثابت $c = c_1 c_2$ و آستانهٔ $n_0 = \max(n_1, n_2)$ کار می‌کنند.

جمع‌پذیری: برای $n \geq \max(n_1, n_2)$، $f(n) + g(n) \leq c_1 h(n) + c_2 h(n) = (c_1 + c_2)\, h(n)$.

حالت سوم: کران بالا از قسمت قبل می‌آید. برای کران پایین، چون $g \geq 0$، داریم $f + g \geq f \geq c\, h$.

∎

نکته · کاربرد عملی جمع‌پذیری

اگر الگوریتمی از دو مرحلهٔ پشت سر هم با زمان‌های $\Theta(n^2)$ و $O(n \log n)$ تشکیل شده باشد، کل زمان $\Theta(n^2)$ است: **کندترین مرحله تعیین‌کننده است.**

۱.۱۲ کران‌های مجانبی توابع رایج

قضیه · چندجمله‌ای‌ها

$a_0 + a_1 n + \cdots + a_d n^d$ با $a_d > 0$ از $\Theta(n^d)$ است. یعنی فقط جملهٔ با بالاترین توان مهم است.

کران بالا: برای $n \geq 1$ هر $|a_i| n^i \leq |a_i| n^d$، پس مجموع حداکثر $\left(\sum_i |a_i|\right) n^d$ است.

کران پایین: مجموع دست‌کم $a_d n^d - \sum_{i<d} |a_i| n^{d-1}$ است. اگر $S = \sum_{i<d} |a_i|$ و $n \geq 2S/a_d$، آنگاه $S\, n^{d-1} \leq \frac{a_d}{2} n^d$، پس مجموع دست‌کم $\frac{a_d}{2} n^d$ است.

■

زمان چندجمله‌ای: زمان اجرا $O(n^d)$ برای ثابتی مثل d که به اندازهٔ ورودی n بستگی ندارد.

قضیه · لگاریتم‌ها

(۱) برای هر دو ثابت $a, b > 1$: $O(\log_a n) = O(\log_b n)$. پس **می‌توان پایهٔ لگاریتم را ننوشت.**

(۲) برای هر $x > 0$: $\log n = O(n^x)$. یعنی لگاریتم از **هر** چندجمله‌ای، حتی $n^{0.001}$، کندتر رشد می‌کند.

(۱) با قاعدهٔ تغییر پایه، $\log_a n = \dfrac{\log_b n}{\log_b a}$. مخرج یک ثابت مثبت است، پس دو لگاریتم فقط در یک ضریب ثابت تفاوت دارند.

(۲) با هوپیتال: $\lim_{n\to\infty} \dfrac{\ln n}{n^x} = \lim_{n\to\infty} \dfrac{1/n}{x\, n^{x-1}} = \lim_{n\to\infty} \dfrac{1}{x\, n^x} = 0.$

■

قضیه · نمایی‌ها

برای هر $r > 1$ و هر $d > 0$: $n^d = O(r^n)$. یعنی **هر تابع نمایی از هر چندجمله‌ای سریع‌تر رشد می‌کند.**

نسبت جمله‌های متوالیِ $a_n = n^d / r^n$ برابر $\frac{a_{n+1}}{a_n} = \left(1 + \frac{1}{n}\right)^d \cdot \frac{1}{r}$ است که با بزرگ شدن n به $\frac{1}{r} < 1$ میل می‌کند. پس از جایی به بعد دنباله مثل یک دنبالهٔ هندسی با قدرنسبت کمتر از یک کوچک می‌شود و به صفر میل می‌کند؛ در نتیجه کران‌دار است و $n^d = O(r^n)$.

■

جمع‌بندی · نردبان رشد

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

علامت $f \prec g$ یعنی $f = o(g)$. هر چه به راست برویم، الگوریتم برای ورودی‌های بزرگ کندتر است.

۱.۱۳ زمان اجرای دستورهای مختلف

برای تحلیل سریع یک شبه‌کد، این قواعد را از ساختارهای کنترلی (شکل اسلاید ۲۸) به خاطر بسپارید:

ساختار	قاعده	زمان
دنباله‌ای از دستورها (بلوک)	زمان‌ها جمع می‌شوند؛ چون تعداد دستورها ثابت است، کندترین دستور غالب است.	$O(f_1 + \cdots + f_k) = O(\max_i f_i)$
شرط `if-else`	بررسی شرط $O(1)$ است و یکی از دو شاخه اجرا می‌شود؛ در بدترین حالت شاخهٔ کندتر.	$O(\max(f(n), g(n)))$
حلقهٔ `while`	اگر حداکثر $g(n)$ بار تکرار شود و هر بار بدنه $O(f(n))$ و آزمون شرط $O(1)$ باشد.	$O(g(n) \cdot f(n))$
حلقهٔ `for`	مقداردهی اولیه، آزمون و به‌روزرسانی هر کدام $O(1)$؛ بدنه $O(f(n))$؛ حداکثر $g(n)$ دور.	$O(g(n) \cdot f(n))$

اشتباه رایج · ضربِ «تعداد دور × بدترین بدنه» فقط کران بالاست

اگر هزینهٔ بدنه در دورهای مختلف تغییر کند، ضرب ممکن است کران شُل بدهد. برای کران دقیق باید هزینهٔ هر دور را جداگانه جمع بزنیم. مثال:

```
i = 1
while i ⩽ n
    for j = 1 to i
        x = x + 1
    i = 2 * i
```

حلقهٔ بیرونی $\log n + 1$ بار اجرا می‌شود و بدنه در بدترین دور n واحد کار دارد؛ قاعدهٔ ضرب $O(n \log n)$ می‌دهد. اما جمع دقیق $1 + 2 + 4 + \cdots + n = 2n - 1$ است، پس زمان واقعی $\Theta(n)$ است!

مثال · دو حلقهٔ تودرتوی وابسته

```
for i = 1 to n
    for j = i to n
        x = x + 1
```

دور بیرونی iام، $n - i + 1$ بار بدنه را اجرا می‌کند. جمع: $\sum_{i=1}^{n}(n - i + 1) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} = \Theta(n^2)$. این‌جا قاعدهٔ ضرب هم $O(n^2)$ می‌داد و این بار کران دقیق است، چون نیمی از دورها دست‌کم $n/2$ کار دارند.

۱.۱۴ الگوریتم‌های چندجمله‌ای و اهمیت آن‌ها

جست‌وجوی جامع (brute force): برای بسیاری از مسائل غیربدیهی یک الگوریتم طبیعی وجود دارد که **همهٔ** جواب‌های ممکن را امتحان می‌کند. معمولاً تعداد جواب‌های ممکن 2^N یا بیشتر است (مثلاً همهٔ زیرمجموعه‌ها)، و این در عمل غیرقابل قبول است.

تعریف · الگوریتم چندجمله‌ای

الگوریتمی **چندجمله‌ای** (poly-time) است اگر ثابت‌های $c > 0$ و $d > 0$ وجود داشته باشند که روی **هر** ورودی به اندازهٔ N، زمان اجرایش حداکثر $c\,N^d$ گام باشد؛ یا به‌طور خلاصه زمانش $O(N^d)$ باشد.

چرا؟ · چرا «چندجمله‌ای» را معادل «کارا» می‌گیریم؟

ویژگی کلیدی این است: اگر اندازهٔ ورودی دو برابر شود، زمان الگوریتم $O(N^d)$ حداکثر در ضریب ثابت 2^d ضرب می‌شود. اما زمان الگوریتم 2^N **به توان دو** می‌رسد ($2^{2N} = (2^N)^2$). یعنی خرید کامپیوتری دو برابر سریع‌تر، به الگوریتم چندجمله‌ای اجازه می‌دهد ورودی‌هایی با اندازهٔ چند برابر را حل کند، ولی به الگوریتم نمایی فقط اجازه می‌دهد یک واحد به N اضافه کند!

تعریف: الگوریتمی را **عملی** (practical) می‌دانیم اگر زمانش چندجمله‌ای باشد. **توجیه:** در عمل جواب می‌دهد! گرچه $6.02 \times 10^{23} \times N^{20}$ از نظر فنی چندجمله‌ای است، در عمل بی‌فایده است؛ اما الگوریتم‌های چندجمله‌ای که مردم طراحی می‌کنند تقریباً همیشه ثابت‌ها و توان‌های کوچکی دارند.

نکته · استثناها

- بعضی الگوریتم‌های چندجمله‌ای ثابت یا توان بزرگی دارند و در عمل بی‌فایده‌اند.
- بعضی الگوریتم‌های نمایی (یا بدتر) به‌طور گسترده استفاده می‌شوند، چون ورودی‌های بدترین حالتشان در عمل نادرند. نمونهٔ معروف، روش سیمپلکس در برنامه‌ریزی خطی است.

چرا مهم است؟ جدول زمان‌ها

جدول زیر (برگرفته از کتاب کلاینبرگ و تاردوس) زمان اجرای الگوریتم‌هایی با مرتبه‌های مختلف را روی پردازنده‌ای با سرعت یک میلیون دستور در ثانیه نشان می‌دهد. «خیلی طولانی» یعنی بیش از 10^{25} سال.

	n	$n\log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n=10$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۴ ثانیه
$n=30$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۱۸ دقیقه	10^{25} سال
$n=50$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۱۱ دقیقه	۳۶ سال	خیلی طولانی
$n=100$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۱ ثانیه	۱۲,۸۹۲ سال	10^{17} سال	خیلی طولانی
$n=1000$	< ۱ ثانیه	< ۱ ثانیه	۱ ثانیه	۱۸ دقیقه	خیلی طولانی	خیلی طولانی	خیلی طولانی
$n=10^4$	< ۱ ثانیه	< ۱ ثانیه	۲ دقیقه	۱۲ روز	خیلی طولانی	خیلی طولانی	خیلی طولانی
$n=10^5$	< ۱ ثانیه	۲ ثانیه	۳ ساعت	۳۲ سال	خیلی طولانی	خیلی طولانی	خیلی طولانی
$n=10^6$	۱ ثانیه	۲۰ ثانیه	۱۲ روز	۳۱,۷۱۰ سال	خیلی طولانی	خیلی طولانی	خیلی طولانی

شهود · از جدول چه بفهمیم؟

ستون‌های چندجمله‌ای «آرام» بدتر می‌شوند: از ۱۰ به یک میلیون، الگوریتم خطی از کمتر از ثانیه به ۱ ثانیه می‌رسد. ستون‌های نمایی «انفجاری» بدتر می‌شوند: الگوریتم 2^n با رفتن از $n=30$ به $n=50$ از ۱۸ دقیقه به ۳۶ سال می‌رسد. به همین دلیل مرز «چندجمله‌ای / نمایی» مهم‌ترین مرز در طراحی الگوریتم است.

1.15 مثال اول: اعداد فیبوناچی

تعریف · دنبالهٔ فیبوناچی

$$F_n = F_{n-1} + F_{n-2}, \qquad F_1 = F_2 = 1.$$

مسئله: برای n داده‌شده، F_n را حساب کنید. مثلاً ده جملهٔ اول: $1, 1, 2, 3, 5, 8, 13, 21, 34, 55$؛ پس $F_{10} = 55$.

با سه الگوریتم برای این مسئلهٔ ساده آشنا می‌شویم. هر سه درست‌اند، اما زمانشان زمین تا آسمان فرق می‌کند.

الگوریتم ۱: بازگشتی ساده

```
FIB1(n)
```

```
1  if n ⩽ 2
2      return 1
3  else
4      return FIB1(n-1) + FIB1(n-2)
```

این الگوریتم ترجمهٔ مستقیم تعریف است. زمانش در رابطهٔ بازگشتی زیر صدق می‌کند (c هزینهٔ ثابت هر فراخوانی است):

$$T(n) = T(n-1) + T(n-2) + c.$$

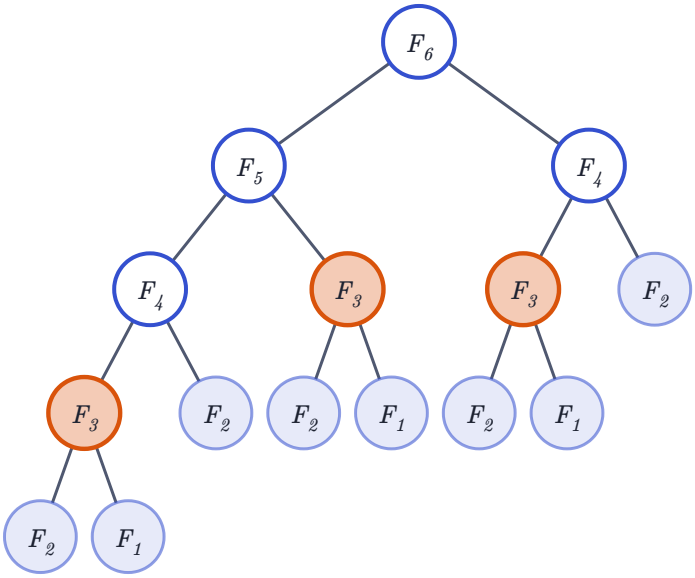
این رابطه تقریباً همان رابطهٔ فیبوناچی است! پس انتظار داریم $T(n)$ تقریباً مثل خود F_n رشد کند. در واقع با استقرا می‌توان نشان داد $T(n) \geq F_n$، و می‌دانیم $F_n \approx \varphi^n/\sqrt{5}$ که در آن $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ **نسبت طلایی** است. پس:

$$T(n) = \Theta(\varphi^n), \qquad \varphi = \frac{1+\sqrt{5}}{2}.$$

چرا؟ φ از کجا آمد؟

حدس می‌زنیم جواب رابطهٔ $F_n = F_{n-1} + F_{n-2}$ به شکل x^n باشد. با جای‌گذاری: $x^n = x^{n-1} + x^{n-2}$؛ با تقسیم بر x^{n-2} به $x^2 = x + 1$ می‌رسیم که ریشهٔ مثبتش $x = \frac{1+\sqrt{5}}{2} = \varphi$ است. جملهٔ ثابت c در رابطهٔ T فقط ضریب را عوض می‌کند، نه پایهٔ نمایی را.

یک استدلال سادهٔ دیگر برای نمایی بودن: چون T صعودی است، $T(n) \geq 2\,T(n-2)$؛ با تکرار این نابرابری $T(n) \geq 2^{n/2} \approx 1.41^n$ به دست می‌آید. از طرف دیگر $T(n) \leq 2\,T(n-1)$ نتیجه می‌دهد $T(n) = O(2^n)$. جواب دقیق φ^n بین این دو است.



شکل ۱-۵ درخت فراخوانی‌های $\mathrm{FIB1}(6)$: پانزده فراخوانی برای محاسبهٔ عدد ۸!
زیردرخت F_3 (نارنجی) سه بار از نو حساب می‌شود. علت کندی همین دوباره‌کاری است.

الگوریتم ۲: حلقهٔ ساده

```
FIB2(n)
```

```
1  a = b = 1
2  for i = 3 to n
3      b = a + b
4      a = b - a
5  return b
```

ایده: هر جمله را فقط یک بار حساب کن و دو جملهٔ آخر را نگه دار. **ناوردا:** پس از دور iام، $b = F_i$ و $a = F_{i-1}$. اگر پیش از دور i، $a = F_{i-2}$ و $b = F_{i-1}$ باشد، خط سوم b را $F_{i-2} + F_{i-1} = F_i$ می‌کند و خط چهارم a را $F_i - F_{i-2} = F_{i-1}$. (این ترفند تفریق، نیاز به متغیر کمکی را از بین می‌برد.) حلقه $n-2$ بار و هر بار $O(1)$ کار انجام می‌دهد، پس:

$$T(n) = O(n).$$

نکته · پیوند با برنامه‌ریزی پویا

تفاوت الگوریتم ۱ و ۲ دقیقاً ایدهٔ **برنامه‌ریزی پویا** است: الگوریتم ۱ زیرمسئله‌های تکراری را بارها حل می‌کند، الگوریتم ۲ هر زیرمسئله را یک بار حل و نتیجه را نگه می‌دارد.

الگوریتم ۳: توان ماتریس

از این مشاهده استفاده می‌کنیم که گام فیبوناچی یک ضرب ماتریسی است:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} \quad \Longrightarrow \quad \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F_2 \\ F_1 \end{pmatrix}.$$

(سطر اول: $F_{n+1} = F_n + F_{n-1}$، سطر دوم: $F_n = F_n$. با تکرار $n-1$ بار از (F_2, F_1) شروع می‌کنیم.) پس کافی است توان $n-1$ام ماتریس $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ را حساب کنیم. ترفند کلیدی **به توان رساندن با مجذورسازی پیاپی** است: برای محاسبهٔ a^n فقط $a, a^2, a^4, a^8, \ldots$ را حساب می‌کنیم (هر کدام مربع قبلی) و آن‌هایی را که در نمایش دودویی n لازم‌اند در هم ضرب می‌کنیم.

مثال · محاسبهٔ a^{13}

$13 = (1101)_2 = 8 + 4 + 1$، پس $a^{13} = a^8 \cdot a^4 \cdot a^1$. سه مجذورسازی (a^2, a^4, a^8) و دو ضرب اضافه کافی است: ۵ ضرب به‌جای ۱۲ ضرب. در حالت کلی حداکثر $2\log_2 n$ ضرب لازم است.

POWER(M, k) ▷ M^k with O(log k) multiplications

```
if k = 0
    return I                        // identity matrix
H = POWER(M, floor(k/2))
H = H * H
if k is odd
    H = H * M
return H
```

هر ضرب ماتریس 2×2 فقط ۸ ضرب و ۴ جمع است، یعنی $O(1)$. تعداد فراخوانی‌ها $\log k$ است، پس:

$$T(n) = O(\log n).$$

مقایسهٔ زمان‌ها

n	۱۰	۲۰	۳۰	۴۰	۵۰	۱۰۰
الگوریتم ۱	۴۴µs	۵۳۰۰µs	۰٫۵ ثانیه	۶۴ ثانیه	۲٫۲ ساعت	**۷ میلیون سال**
الگوریتم ۲	۴µs	۷µs	۱۰µs	۱۴µs	۱۸µs	**۴۵µs**
الگوریتم ۳	۱۱µs	۱۲µs	۱۳µs	۱۵µs	۱۷µs	**۲۱µs**

نکته · «۷ میلیون سال» از کجا آمده؟

در جدول، با هر ۱۰ واحد افزایش n، زمان الگوریتم ۱ حدود $\varphi^{10} \approx 123$ برابر شده است (مثلاً از ۶۴ ثانیه به ۲٫۲ ساعت). اگر همین روند را از $n = 50$ تا $n = 100$ ادامه دهیم (یعنی ضرب در $\varphi^{50} \approx 2.8 \times 10^{10}$)، به حدود ۷ میلیون سال می‌رسیم. تخمین خام‌تر $\varphi^{100}/10^8$ ثانیه (با فرض 10^8 عمل در ثانیه و یک عمل برای هر فراخوانی) حدود ۲۵۰ هزار سال می‌دهد. عدد دقیق به هزینهٔ هر فراخوانی بستگی دارد، اما نتیجه یکی است: **الگوریتم ۱ برای $n = 100$ هرگز تمام نمی‌شود.**

دقت کنید که برای nهای کوچک (مثلاً ۱۰) الگوریتم ۳ از الگوریتم ۲ کندتر است، چون ضرب ماتریس ثابت بزرگ‌تری دارد. این همان پدیدهٔ «تقاطع» است که در مثال $30n + 8$ دیدیم.

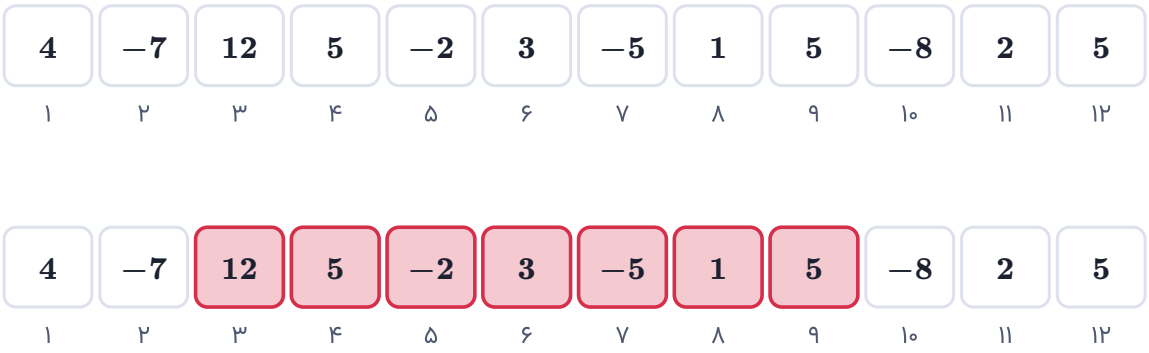
تکمیلی (فراتر از اسلاید) · فرض هزینهٔ واحد این‌جا کجا می‌لنگد؟

F_n تقریباً $0.694n$ بیت دارد (چون $\log_2 \varphi \approx 0.694$). برای nهای خیلی بزرگ، جمع و ضرب چنین اعدادی دیگر $O(1)$ نیست. با شمارش دقیق عملیات بیتی، الگوریتم ۲ در واقع $O(n^2)$ عمل بیتی انجام می‌دهد و الگوریتم ۳ به سرعت الگوریتم ضرب اعداد بزرگ (فصل ۲) وابسته می‌شود. اما ترتیب کلی «۱ خیلی کند، ۲ و ۳ سریع» تغییری نمی‌کند.

۱.۱۶ مثال دوم: زیرآرایه با بیشترین مجموع

تعریف · مسئلهٔ زیرآرایهٔ بیشینه

آرایهٔ یک‌بعدی $A[1..n]$ از اعداد (مثبت و منفی) داده شده است. زیرآرایهٔ **پیوسته‌ای** $A[i..j]$ را پیدا کنید که مجموع عناصرش بیشینه باشد. فرض می‌کنیم مجموع زیرآرایهٔ تهی صفر است؛ پس جواب هرگز منفی نیست.



شکل ۱-۶ جواب برای این آرایه زیرآرایهٔ $A[3..9]$ است: $12 + 5 - 2 + 3 - 5 + 1 + 5 = 19$. دقت کنید جواب عناصر منفی (-2 و -5) را هم در بر دارد، چون آن‌ها دو بخش پرسود را به هم وصل می‌کنند.

چرا؟ · چرا فرض زیرآرایهٔ تهی؟

اگر همهٔ اعداد منفی باشند، هر زیرآرایهٔ ناتهی مجموع منفی دارد. با این فرض، جواب در این حالت صفر (برداشتن هیچ چیز) است و دیگر لازم نیست حالت خاص تعریف کنیم. در شبه‌کدها این فرض با مقدار اولیهٔ `sol = 0` پیاده شده است.

الگوریتم ۱: جست‌وجوی جامع $O(n^3)$

MAX-SUB-1(A)

```
1  sol = 0
2  for i = 1 to n                  // start
3      for j = i to n              // end
4          sum = 0
5          for k = i to j
6              sum = sum + A[k]
7          if sum > sol then
8              sol = sum
9  return sol
```

همهٔ جفت‌های (i, j) را امتحان می‌کنیم و برای هر کدام مجموع $A[i..j]$ را از نو حساب می‌کنیم. تعداد اجرای خط جمع:

$$\sum_{i=1}^{n}\sum_{j=i}^{n}(j-i+1)=\sum_{L=1}^{n}L\,(n-L+1)=\frac{n(n+1)(n+2)}{6}=\Theta(n^3).$$

(در جمع وسط، $L=j-i+1$ طول زیرآرایه است و دقیقاً $n-L+1$ زیرآرایه به طول L وجود دارد.)

الگوریتم ۲: مجموع پیشوندی $O(n^2)$

شهود · کجا دوباره‌کاری می‌کنیم؟

الگوریتم ۱ مجموع $A[i..j]$ را از صفر حساب می‌کند، در حالی که مجموع $A[i..j-1]$ را همین الان داشت! اگر مجموع‌ها را یک بار پیش‌محاسبه کنیم، هر مجموع بازه‌ای در $O(1)$ به دست می‌آید.

مشاهده: فرض کنید $S[i]=A[1]+\cdots+A[i]$ (مجموع پیشوندی) و $S[0]=0$. آنگاه:

$$A[i]+A[i+1]+\cdots+A[j]=S[j]-S[i-1].$$

چون $S[j]$ مجموع تا j است و $S[i-1]$ همان بخش اضافهٔ اول را دارد که با تفریق حذف می‌شود.

PRE-PROCESSING ▷ O(n)

```
1  S[0] = 0
2  for i = 1 to n
3      S[i] = S[i-1] + A[i]
```

MAX-SUB-2(A) ▷ O(n^2)

```
1  sol = 0
2  for i = 1 to n
3      for j = i to n
4          if S[j] - S[i-1] > sol then
5              sol = S[j] - S[i-1]
6  return sol
```

پیش‌پردازش $O(n)$ و دو حلقهٔ تودرتو $\frac{n(n+1)}{2}$ بار بدنهٔ $O(1)$ را اجرا می‌کنند؛ پس $T(n) = O(n^2)$.

نکته · ادامهٔ داستان

در فصل ۲ همین مسئله را با تقسیم و حل در $O(n \log n)$ حل می‌کنیم. و در کادر زیر، الگوریتمی خطی می‌بینیم.

تکمیلی (فراتر از اسلاید) · الگوریتم کادان: $O(n)$

فرض کنید cur بیشترین مجموعِ زیرآرایه‌ای باشد که **دقیقاً** در خانهٔ فعلی تمام می‌شود. با آمدن $A[i]$، یا زیرآرایهٔ قبلی را ادامه می‌دهیم ($cur + A[i]$) یا اگر منفی شد، از نو شروع می‌کنیم (صفر). جواب بیشترین مقداری است که cur به خود می‌گیرد.

KADANE(A)

```
1  best = 0
2  cur = 0
3  for i = 1 to n
4      cur = max(0, cur + A[i])
5      best = max(best, cur)
6  return best
```

روی آرایهٔ مثال، مقادیر cur به ترتیب $4, 0, 12, 17, 15, 18, 13, 14, \mathbf{19}, 11, 13, 18$ است و جواب ۱۹ به دست می‌آید.

۱.۱۷ مروری بر زمان‌های اجرای رایج

برای هر مرتبهٔ زمانی رایج، یک الگوریتم نمونه می‌بینیم تا با «شکل» هر کدام آشنا شوید.

زمان خطی $O(n)$

زمان متناسب با اندازهٔ ورودی است. معمولاً یعنی ورودی را یک (یا چند) بار پیمایش کرده‌ایم و برای هر عنصر کار ثابتی انجام داده‌ایم.

محاسبهٔ بیشینه: بیشینهٔ n عدد $a_1, \ldots, a_n$.

```
MAX(a1..an)
```

```
1 max = a1
2 for i = 2 to n
3     if a[i] > max
4         max = a[i]
```

ادغام: دو لیست مرتب $A = a_1, \ldots, a_n$ و $B = b_1, \ldots, b_n$ را به یک لیست مرتب تبدیل کنید.

```
MERGE(A, B)
```

```
1 i = 1, j = 1
2 while both lists are nonempty
3     if a[i] ⩽ b[j]
4         append a[i] to output; i = i + 1
5     else
6         append b[j] to output; j = j + 1
7 append remainder of nonempty list to output
```

قضیه · زمان ادغام

ادغام دو لیست به اندازهٔ n زمان $O(n)$ می‌برد.

پس از هر مقایسه، دقیقاً یک عنصر به خروجی اضافه می‌شود. خروجی در نهایت $2n$ عنصر دارد، پس حداکثر $2n$ مقایسه انجام می‌شود و هر کدام $O(1)$ کار دارد.

■

زمان $O(n \log n)$

این زمان (که به آن «خطی‌لگاریتمی» هم می‌گویند) به‌طور طبیعی در الگوریتم‌های تقسیم و حل ظاهر می‌شود: $\log n$ سطح تقسیم و در هر سطح $O(n)$ کار.

- **مرتب‌سازی:** مرتب‌سازی ادغامی و هرمی (heapsort) با $O(n \log n)$ مقایسه مرتب می‌کنند.
- **بزرگ‌ترین بازهٔ خالی:** n برچسب زمانی $x_1, \ldots, x_n$ داریم که در آن‌ها نسخه‌هایی از یک فایل به سرور رسیده‌اند. طولانی‌ترین بازه‌ای که هیچ نسخه‌ای نرسیده کدام است؟ **راه‌حل** $O(n \log n)$**:** برچسب‌ها را مرتب کن ($O(n \log n)$)، سپس لیست مرتب را یک بار پیمایش کن و بیشترین فاصلهٔ دو برچسب متوالی را بیاب ($O(n)$).

زمان درجه دو $O(n^2)$

شکل معمول: **همهٔ جفت‌ها** را بررسی کن.

نزدیک‌ترین جفت نقاط: n نقطه $(x_1, y_1), \ldots, (x_n, y_n)$ در صفحه داده شده؛ نزدیک‌ترین دو نقطه را پیدا کنید. راه‌حل $O(n^2)$: همهٔ جفت‌ها را امتحان کن.

CLOSEST-PAIR-BRUTE

```
min = (x1 - x2)^2 + (y1 - y2)^2
for i = 1 to n
    for j = i+1 to n
        d = (x[i] - x[j])^2 + (y[i] - y[j])^2    // no square root needed
        if d < min
            min = d
```

چرا؟ · چرا جذر نگرفتیم؟

تابع جذر اکیداً صعودی است: $d_1 < d_2 \iff \sqrt{d_1} < \sqrt{d_2}$. پس جفتی که **مربع** فاصله‌اش کمینه است، همان جفتی است که فاصله‌اش کمینه است. با مقایسهٔ مربع‌ها از محاسبهٔ گران و غیردقیقِ جذر صرف‌نظر می‌کنیم.

نکته: به نظر می‌رسد $\Omega(n^2)$ اجتناب‌ناپذیر باشد، اما این توهمی بیش نیست: الگوریتمی تقسیم و حل برای این مسئله در $O(n \log n)$ وجود دارد.

زمان مکعبی $O(n^3)$

شکل معمول: **همهٔ سه‌تایی‌ها** را بررسی کن.

مجزا بودن مجموعه‌ها: n مجموعهٔ $S_1, \ldots, S_n$ داده شده که هر کدام زیرمجموعهٔ $\{1, 2, \ldots, n\}$ است. آیا جفتی از آن‌ها وجود دارد که اشتراکشان تهی باشد؟

SET-DISJOINTNESS

```
foreach set S_i
    foreach other set S_j
        foreach element p of S_i
            determine whether p also belongs to S_j
        if no element of S_i belongs to S_j
            report that S_i and S_j are disjoint
```

n^2 جفت مجموعه، هر کدام حداکثر n عنصر؛ اگر هر مجموعه با یک آرایهٔ بولی به طول n نمایش داده شود، «آیا $p \in S_j$؟» در $O(1)$ پاسخ داده می‌شود. پس زمان $O(n^3)$ است.

زمان چندجمله‌ای $O(n^k)$

مجموعهٔ مستقل به اندازهٔ k: گرافی داده شده؛ آیا k رأس وجود دارد که هیچ دوتایی‌شان با یال به هم وصل نباشند؟ (k یک ثابت است.)

INDEPENDENT-SET-k

```
foreach subset S of k nodes
    check whether S is an independent set
    if S is an independent set
        report S is an independent set
```

- بررسی مستقل بودن S: همهٔ $\binom{k}{2}$ جفت را چک کن، یعنی $O(k^2)$.
- تعداد زیرمجموعه‌های kعضوی: $\binom{n}{k} = \dfrac{n(n-1)(n-2)\cdots(n-k+1)}{k(k-1)(k-2)\cdots 1} \le \dfrac{n^k}{k!}$.
- جمع کل: $O(k^2\, n^k/k!) = O(n^k)$، چون k ثابت است.

نکته · چندجمله‌ای ولی غیرعملی

برای $k = 17$ این الگوریتم از نظر فنی چندجمله‌ای است ($O(n^{17})$)، اما برای گرافی با فقط ۱۰۰۰ رأس به حدود $10^{51}/17! \approx 3 \times 10^{36}$ زیرمجموعه نیاز دارد. این همان استثنایی است که در بخش الگوریتم‌های چندجمله‌ای گفتیم.

زمان نمایی

مجموعهٔ مستقل بیشینه: بزرگ‌ترین مجموعهٔ مستقل گراف را پیدا کنید. اکنون k ثابت نیست؛ باید همهٔ زیرمجموعه‌ها را امتحان کنیم.

MAX-INDEPENDENT-SET

```
S* = empty set
foreach subset S of nodes
    check whether S is an independent set
    if S is largest independent set seen so far
        update S* = S
```

2^n زیرمجموعه و برای هر کدام بررسی $O(n^2)$: کل زمان $O(n^2\,2^n)$. (تا امروز هیچ الگوریتم چندجمله‌ای برای این مسئله شناخته نشده است.)

تکمیلی (فراتر از اسلاید) · زمان زیرخطی $O(\log n)$

الگوریتم‌هایی هم هستند که حتی کل ورودی را نمی‌خوانند. نمونهٔ کلاسیک **جست‌وجوی دودویی** در آرایهٔ مرتب است: با هر مقایسه نیمی از آرایه کنار گذاشته می‌شود، پس پس از $\log_2 n$ مقایسه یک خانه باقی می‌ماند. هر جا دیدید اندازهٔ مسئله در هر گام نصف (یا کسری ثابت) می‌شود، به $\log n$ فکر کنید.

جمع‌بندی · فصل ۱ در یک نگاه

- زمان اجرا = تعداد عملیات پایه، به‌عنوان تابعی از اندازهٔ ورودی؛ معمولاً در بدترین حالت.
- O کران بالا، Ω کران پایین، Θ کران محکم؛ همه با «ضریب ثابت» و «از جایی به بعد».
- چندجمله‌ای‌ها با بالاترین توان، لگاریتم‌ها بی‌پایه، و نمایی‌ها همیشه بالاتر از چندجمله‌ای‌ها.
- ایدهٔ بهتر بر سخت‌افزار بهتر غلبه می‌کند: فیبوناچی از $\Theta(\varphi^n)$ به $O(\log n)$، زیرآرایهٔ بیشینه از $O(n^3)$ به $O(n)$.

۱.۱۸ خودآزمایی فصل ۱

خودآزمایی

۱. آیا $2^{n+1} = O(2^n)$؟ آیا $2^{2n} = O(2^n)$؟

پاسخ: اولی بله: $2^{n+1} = 2 \cdot 2^n$، پس $c = 2$ کار می‌کند. دومی نه: $2^{2n}/2^n = 2^n$ به بی‌نهایت میل می‌کند و هیچ ثابت cای نمی‌تواند آن را کران‌دار کند.

۲. این توابع را به ترتیب رشد مرتب کنید: $n^2,\ 2^n,\ n!,\ \sqrt{n},\ 2^{\log_2 n},\ \log^2 n,\ n^{1.5},\ n\log n$.

پاسخ: $\log^2 n \prec \sqrt{n} \prec 2^{\log_2 n} = n \prec n\log n \prec n^{1.5} \prec n^2 \prec 2^n \prec n!$. دقت کنید $2^{\log_2 n}$ همان n است.

3. زمان این قطعه را با Θ بیان کنید: `for i = 1 to n: j = i; while j > 1: j = j / 2`

پاسخ: حلقهٔ داخلی برای هر i حدود $\log_2 i$ بار اجرا می‌شود. جمع: $\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n)$ (چون نیمهٔ بالایی جمله‌ها هر کدام دست‌کم $\log(n/2)$ هستند).

4. مرتب‌سازی درجی روی آرایهٔ $1\ 2\ 3\ 4\ 5\ 6$ چند بار خط `A[i+1] = A[i]` را اجرا می‌کند؟

پاسخ: در بدترین حالت برای هر j، $j-1$ جابه‌جایی داریم: $1+2+3+4+5 = 15$.

5. نشان دهید برای توابع نامنفی f, g: $\max(f,g) = \Theta(f+g)$.

پاسخ: $\max(f,g) \le f+g$ (پس $c_2 = 1$) و $f+g \le 2\max(f,g)$، یعنی $\max(f,g) \ge \frac{1}{2}(f+g)$ (پس $c_1 = \frac{1}{2}$).

6. چرا جملهٔ «زمان این الگوریتم دست‌کم $O(n^2)$ است» بی‌معناست؟

پاسخ: O کران بالاست. «دست‌کم یک کران بالا» هیچ قیدی ایجاد نمی‌کند. برای کران پایین باید گفت «$\Omega(n^2)$ است».

فصل ۲

تقسیم و حل

مسئلهٔ بزرگ را بشکن، تکه‌ها را بازگشتی حل کن و جواب‌ها را به هم بچسبان. این ایدهٔ ساده، مرتب‌سازی را از n^2 به $n \log n$ و ضرب اعداد را از n^2 به $n^{1.585}$ می‌رساند.

مرتب‌سازی ادغامی · رابطه‌های بازگشتی · قضیهٔ اصلی · زیرآرایهٔ بیشینه · ضرب اعداد صحیح · حدس و استقرا

۲.۱ ایدهٔ تقسیم و حل

تعریف · تقسیم و حل

الگوریتم تقسیم و حل (divide and conquer) سه گام دارد:

۱. **تقسیم (divide):** مسئله را به چند زیرمسئلهٔ کوچک‌تر از همان نوع بشکن.

۲. **حل (conquer):** هر زیرمسئله را به‌صورت **بازگشتی** حل کن. اگر زیرمسئله به‌اندازهٔ کافی کوچک بود، مستقیم حلش کن (حالت پایه).

۳. **ترکیب (combine):** جواب زیرمسئله‌ها را ترکیب کن تا جواب مسئلهٔ اصلی به دست آید.

رایج‌ترین شکل: مسئلهٔ اندازهٔ n را به دو بخش مساوی به اندازهٔ $n/2$ بشکن، هر دو را بازگشتی حل کن و دو جواب را در **زمان خطی** ترکیب کن. **نتیجه:** جایی که روش جامع n^2 زمان می‌برد، تقسیم و حل اغلب به $n \log n$ می‌رسد.

چرا؟ · چرا شکستن مسئله اصلاً سود دارد؟

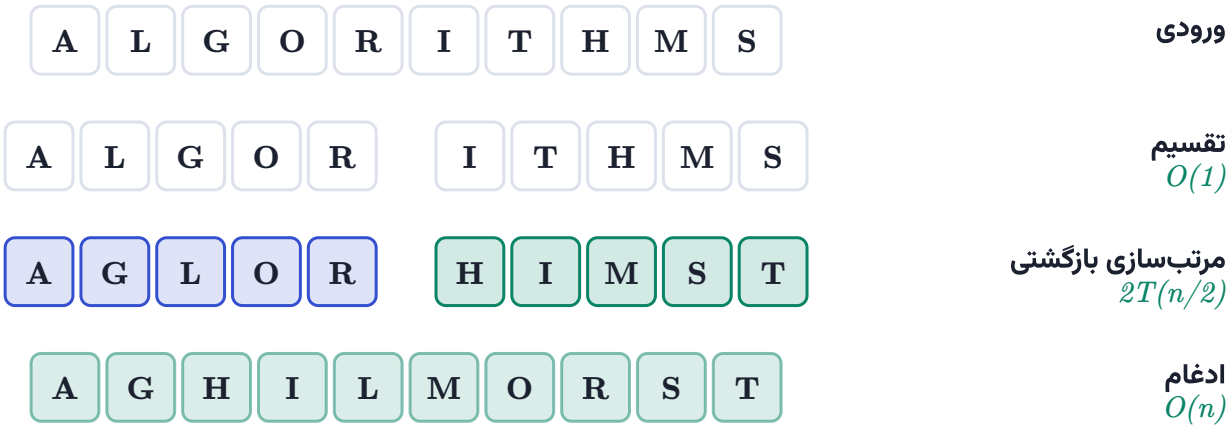
فرض کنید روش مستقیم n^2 کار می‌کند. اگر مسئله را به دو نیمه بشکنیم و هر نیمه را با **همان** روش مستقیم حل کنیم، هزینه $2 \cdot (n/2)^2 = n^2/2$ می‌شود: همین الان نصف شد! حالا اگر ترکیب ارزان (خطی) باشد و این کار را بازگشتی تکرار کنیم، صرفه‌جویی در هر سطح روی هم انباشته می‌شود. راز کار در این است که هزینهٔ روش مستقیم **فوق‌خطی** است، پس مسئله‌های کوچک «به نسبت» ارزان‌ترند.

۲.۲ مرتب‌سازی ادغامی

مرتب‌سازی: n عنصر داده شده؛ آن‌ها را به ترتیب صعودی بچینید. کاربردهای مرتب‌سازی بی‌شمارند: مرتب کردن فهرست نام‌ها، سازمان‌دهی کتابخانهٔ موسیقی، یافتن میانه، یافتن نزدیک‌ترین جفت، جست‌وجوی دودویی در پایگاه داده، یافتن تکراری‌ها در فهرست پستی و ... . بسیاری از مسائل وقتی داده مرتب باشد ساده می‌شوند.

الگوریتم · مرتب‌سازی ادغامی (Mergesort)

1. آرایه را به دو نیمه تقسیم کن.
2. هر نیمه را بازگشتی مرتب کن.
3. دو نیمهٔ مرتب را **ادغام** کن تا کل آرایه مرتب شود.



شکل ۲-۱ مرتب‌سازی ادغامی روی حروف ALGORITHMS. تقسیم فقط پیدا کردن وسط است ($O(1)$)، دو مرتب‌سازی بازگشتی $2T(n/2)$ و ادغام $O(n)$ زمان می‌برند.

ادغام: قلب الگوریتم

ادغام: دو لیست از پیش مرتب را به یک لیست مرتب تبدیل کن. چطور کارآمد ادغام کنیم؟ با تعداد **خطی** مقایسه و یک **آرایهٔ موقت**: دو اشاره‌گر روی ابتدای دو لیست می‌گذاریم. در هر گام، کوچک‌ترین عنصرِ باقی‌مانده حتماً یکی از دو عنصرِ زیر اشاره‌گرهاست (چون هر لیست مرتب است و کوچک‌ترین عنصرش اولش است). آن دو را مقایسه می‌کنیم، کوچک‌تر را به خروجی می‌فرستیم و اشاره‌گرش را جلو می‌بریم.

مثال · ادغام AGLOR و HIMST

step	L (i)	R (j)	compare	output so far
1	**A**GLOR	**H**IMST	A ≤ H	A
2	A**G**LOR	**H**IMST	G ≤ H	A G
3	AG**L**OR	**H**IMST	L > H	A G H
4	AG**L**OR	H**I**MST	L > I	A G H I
5	AG**L**OR	HI**M**ST	L ≤ M	A G H I L
6	AGL**O**R	HI**M**ST	O > M	A G H I L M
7	AGL**O**R	HIM**S**T	O ≤ S	A G H I L M O
8	AGLO**R**	HIM**S**T	R ≤ S	A G H I L M O R
9	(empty)	HIM**ST**	—	A G H I L M O R S T

هشت مقایسه برای ده عنصر. وقتی یک لیست تمام شد، بقیهٔ لیست دیگر بدون مقایسه به انتها چسبانده می‌شود.

```
MERGESORT(A[1..n])
```

```
1  if n = 1 then return A
2  m = floor(n/2)
3  L = MERGESORT(A[1..m])
4  R = MERGESORT(A[m+1..n])
5  i = 1, j = 1
6  for k = 1 to n
7      if i ⩽ m and j ⩽ n-m then          // both lists nonempty
8          if L[i] ⩽ R[j] then B[k] = L[i], i = i+1
9          else                B[k] = R[j], j = j+1
10     else if i > m then B[k] = R[j], j = j+1    // L is exhausted
11     else               B[k] = L[i], i = i+1    // R is exhausted
12 return B
```

نکته · دو جزئیات کوچک

- نیمهٔ راست $\lceil n/2 \rceil = n - m$ عنصر دارد، نه $\lfloor n/2 \rfloor$. در شبه‌کد اسلاید شرط $j \le n/2$ آمده که فقط وقتی n زوج است درست است؛ این‌جا $j \le n - m$ نوشته‌ایم تا برای n فرد هم درست باشد.
- استفاده از $\le$ (به‌جای $<$) در مقایسهٔ $L[i] \le R[j]$ باعث می‌شود از دو عنصر مساوی، آنکه در نیمهٔ چپ بوده زودتر بیاید. به این ویژگی **پایداری** (stability) می‌گویند.

۲.۳ رابطهٔ بازگشتی و درخت بازگشت

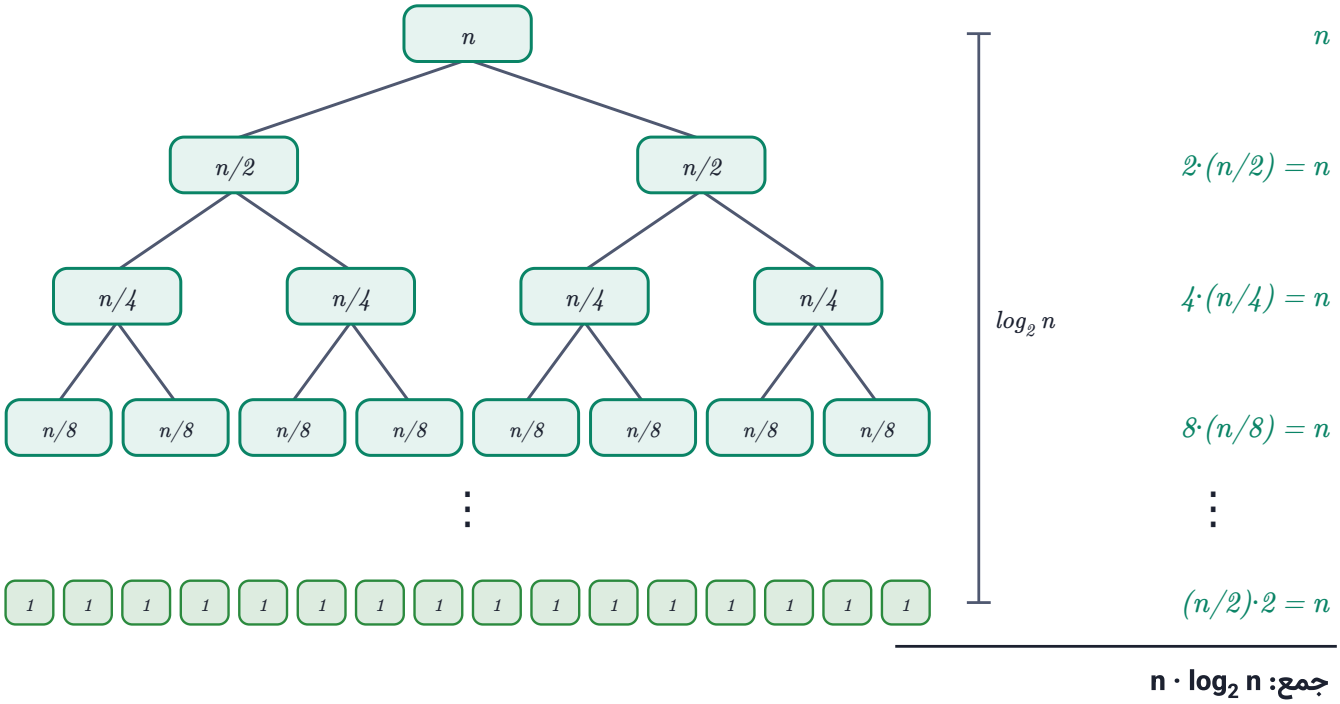
تعریف · رابطهٔ بازگشتی مرتب‌سازی ادغامی

$T(n)$ = تعداد مقایسه‌ها برای مرتب‌سازی ادغامی روی ورودی به اندازهٔ n.

$$T(n) = \begin{cases} 0 & n \le 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{نیمهٔ چپ}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{نیمهٔ راست}} + \underbrace{n}_{\text{ادغام}} & \text{در غیر این صورت} \end{cases}$$

جواب: $T(n) = O(n \log_2 n)$.

این رابطه را با فرض اینکه n توانی از ۲ است حل می‌کنیم (تا کف و سقف حذف شوند): $T(n) = 2T(n/2) + n$ و $T(1) = 0$.
ابزار اول ما **درخت بازگشت** است: هر گره یک فراخوانی است و داخلش هزینه‌ای را می‌نویسیم که **خود آن فراخوانی** (بدون فرزندانش) انجام می‌دهد.



شکل ۲-۲ درخت بازگشت $T(n) = 2T(n/2) + n$. در سطح kام 2^k گره داریم که هر کدام $n/2^k$ کار می‌کنند؛ پس هر سطح دقیقاً n کار دارد.

اثبات · اثبات با درخت بازگشت

هزینهٔ هر سطح: سطح k (ریشه سطح ۰ است) 2^k زیرمسئله به اندازهٔ $n/2^k$ دارد و ادغام هر کدام $n/2^k$ کار می‌برد: $2^k \cdot \frac{n}{2^k} = n$.

تعداد سطح‌ها: اندازه در هر سطح نصف می‌شود و از n به ۱ می‌رسد؛ پس $\log_2 n$ سطح هزینه‌دار داریم (برگ‌ها، یعنی زیرمسئله‌های یک‌عنصری، هزینه‌ای ندارند).

جمع: $T(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \text{ بار}} = n \log_2 n$.

■

تکمیلی (فراتر از اسلاید) · اثبات دوم: تلسکوپی

طرفین $T(n) = 2T(n/2) + n$ را بر n تقسیم کنید:

$$\frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1 = \frac{T(n/4)}{n/4} + 1 + 1 = \cdots = \frac{T(1)}{1} + \underbrace{1 + \cdots + 1}_{\log_2 n} = \log_2 n.$$

پس $T(n) = n \log_2 n$. این ترفند (تقسیم بر n تا رابطه به شکل «جمع ساده» درآید) برای رابطه‌هایی با ضریب و تقسیم برابر خیلی کارآمد است.

نکته · اگر n توانی از ۲ نباشد؟

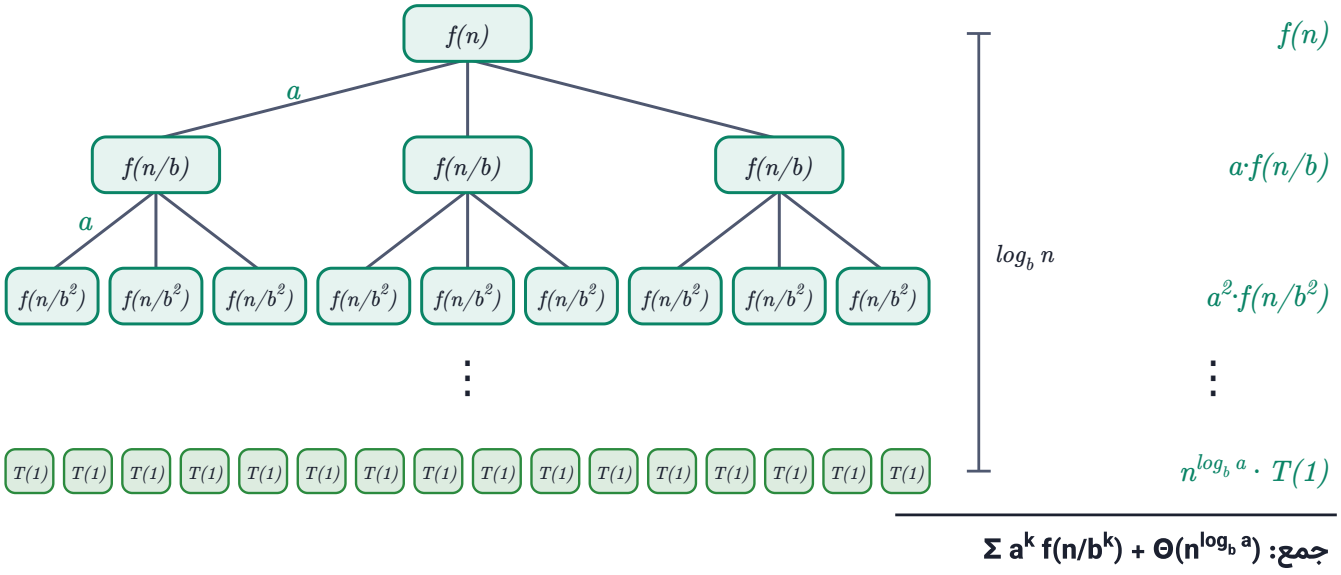
T صعودی است (آرایهٔ بزرگ‌تر مقایسهٔ کمتری نمی‌خواهد). اگر $2^{k-1} < n \le 2^k$، آنگاه $T(n) \le T(2^k) = k\,2^k \le 2n(\log_2 n + 1) = O(n \log n)$. یعنی گرد کردن به توان بعدی ۲ فقط ضریب ثابت را عوض می‌کند. به همین دلیل در تحلیل مجانبی معمولاً کف و سقف را نادیده می‌گیریم.

۲.۴ درخت بازگشت در حالت کلی

حالا رابطهٔ کلی تقسیم و حل را در نظر بگیرید:

$$T(n) = \begin{cases} 0 & n \le 1 \\ \underbrace{a\,T(n/b)}_{\text{زیرمسئله‌ها}} + \underbrace{f(n)}_{\text{تقسیم و ترکیب}} & \text{در غیر این صورت} \end{cases}$$

می‌توانید آن را تابع بازگشتی‌ای تصور کنید که خودش را a بار، هر بار روی ورودی به اندازهٔ n/b، صدا می‌زند و خروجی‌ها را در زمان $f(n)$ ترکیب می‌کند.



شکل ۲-۳ درخت بازگشت $T(n) = aT(n/b) + f(n)$. هر گره a فرزند دارد و اندازه در هر سطح بر b تقسیم می‌شود.

- **سطح kام** a^k گره دارد، هر کدام با ورودی n/b^k و هزینهٔ $f(n/b^k)$. هزینهٔ کل سطح: $a^k f(n/b^k)$.
- **عمق درخت** $\log_b n$ است، چون پس از $\log_b n$ بار تقسیم بر b به اندازهٔ ۱ می‌رسیم.
- **تعداد برگ‌ها** $a^{\log_b n} = n^{\log_b a}$ است.

چرا؟ · چرا $a^{\log_b n} = n^{\log_b a}$؟

از تعریف لگاریتم، $a = b^{\log_b a}$. پس

$$a^{\log_b n} = \left(b^{\log_b a}\right)^{\log_b n} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a}.$$

(دو توان را جابه‌جا کردیم، چون $(x^p)^q = (x^q)^p$.) این اتحاد مهم است: تعداد برگ‌ها را به شکل یک **توان از** n درمی‌آورد تا بتوانیم با $f(n)$ مقایسه‌اش کنیم.

با جمع زدن سطح‌ها:

$$T(n) = \sum_{k=0}^{\log_b n - 1} a^k f\left(\frac{n}{b^k}\right) + \Theta\left(n^{\log_b a}\right).$$

۲.۵ قضیهٔ اصلی

جمع بالا را می‌شود یک بار برای همیشه حساب کرد. نتیجه، **قضیهٔ اصلی** (master theorem) است. ایدهٔ آن یک «کشمکش» است: کشمکش میان **تعداد برگ‌ها** ($n^{\log_b a}$) و **هزینهٔ ریشه** ($f(n)$). هر کدام بزرگ‌تر باشد، برنده است.

تعریف · به‌طور چندجمله‌ای بزرگ‌تر

$g(n)$ به‌طور چندجمله‌ای از $h(n)$ بزرگ‌تر است، اگر و فقط اگر برای عددی مثل $x > 0$: $\frac{g(n)}{h(n)} = \Omega(n^x)$.

مثلاً n^2 به‌طور چندجمله‌ای از $n^{1.5}$ بزرگ‌تر است (نسبت $n^{0.5}$)، ولی $n \log n$ به‌طور چندجمله‌ای از n بزرگ‌تر **نیست** (نسبت $\log n$ از هر n^x کندتر رشد می‌کند).

قضیه · قضیهٔ اصلی

برای $T(n) = aT(n/b) + f(n)$ با $a \geq 1$ و $b > 1$:

1. اگر $f(n) = \Theta(n^{\log_b a})$، آنگاه $T(n) = \Theta(n^{\log_b a} \log n)$. **مساوی**
2. اگر $f(n)$ به‌طور چندجمله‌ای از $n^{\log_b a}$ بزرگ‌تر باشد، آنگاه $T(n) = \Theta(f(n))$. **ریشه برنده**
3. اگر $n^{\log_b a}$ به‌طور چندجمله‌ای از $f(n)$ بزرگ‌تر باشد، آنگاه $T(n) = \Theta(n^{\log_b a})$. **برگ‌ها برنده**



شکل ۲-۴ هر نوار، هزینهٔ یک سطح درخت بازگشت است (ریشه بالا). وقتی هزینه‌ها یک دنبالهٔ هندسی‌اند، جمعشان توسط بزرگ‌ترین جمله تعیین می‌شود: یا ردیف برگ‌ها، یا ریشه. در حالت مساوی، همهٔ $\log n$ سطح سهم برابر دارند.

نکته · شرط نظم در حالت «ریشه برنده»

در صورت دقیق قضیه (کتاب CLRS)، حالت دوم یک شرط اضافه هم دارد: برای ثابتی مثل $c < 1$ و nهای بزرگ، $af(n/b) \le cf(n)$. یعنی هزینهٔ سطح‌ها واقعاً رو به پایین کم شود. برای همهٔ توابع چندجمله‌ای مثل $f(n) = n^d$ این شرط خودبه‌خود برقرار است، پس در عمل نگرانش نباشید.

اشتباه رایج · قضیهٔ اصلی همه چیز را پوشش نمی‌دهد

اگر $f(n)$ و $n^{\log_b a}$ «به‌طور چندجمله‌ای» فاصله نداشته باشند ولی برابر هم نباشند، هیچ حالتی اعمال نمی‌شود. مثال: $T(n) = 2T(n/2) + n\log n$. اینجا $n^{\log_2 2} = n$ و $f(n)/n = \log n$ که چندجمله‌ای نیست. با درخت بازگشت: سطح k هزینهٔ $2^k \cdot \frac{n}{2^k}\log\frac{n}{2^k} = n(\log n - k)$ دارد، و جمع روی $k = 0..\log n - 1$ برابر $n \cdot \frac{\log n(\log n+1)}{2} = \Theta(n\log^2 n)$ است.

حالت خاص پرکاربرد: $f(n) = O(n^d)$

در بیشتر الگوریتم‌ها هزینهٔ ترکیب یک چندجمله‌ای است. در این حالت قضیه شکل ساده‌ای دارد:

قضیه · قضیهٔ اصلی برای $f(n) = O(n^d)$

$$T(n) = a\,T\!\left(\frac{n}{b}\right) + O(n^d) \quad\Longrightarrow\quad T(n) = \begin{cases} O(n^d \log n) & \log_b a = d \\ O(n^d) & \log_b a < d \\ O(n^{\log_b a}) & \log_b a > d \end{cases}$$

اثبات · چرا این سه حالت؟ (سری هندسی)

هزینهٔ سطح k حداکثر $a^k \cdot c\left(\frac{n}{b^k}\right)^d = c\,n^d\left(\frac{a}{b^d}\right)^k$ است. پس کل هزینه یک سری هندسی با قدرنسبت $r = a/b^d$ است:

$$T(n) \le c\,n^d \sum_{k=0}^{\log_b n} r^k.$$

- $r < 1$ (یعنی $a < b^d$، یعنی $\log_b a < d$): سری نزولی است و جمعش حداکثر $\frac{1}{1-r}$، یک ثابت. پس $T(n) = O(n^d)$: **ریشه غالب است.**
- $r = 1$ (یعنی $\log_b a = d$): همهٔ $\log_b n + 1$ جمله برابر ۱ هستند. پس $T(n) = O(n^d \log n)$.
- $r > 1$ (یعنی $\log_b a > d$): سری صعودی است و جمعش هم‌مرتبهٔ آخرین جمله است: $r^{\log_b n} = \frac{a^{\log_b n}}{(b^{\log_b n})^d} = \frac{n^{\log_b a}}{n^d}$. ضرب در n^d: $T(n) = O(n^{\log_b a})$: **برگ‌ها غالب‌اند.**

∎

مثال · چهار مثال اسلاید

رابطه	$a,\ b,\ d$	مقایسه	جواب
$T(n) = 4T(n/2) + O(n)$	$4, 2, 1$	$\log_2 4 = 2 > 1$	$O(n^2)$
$T(n) = 3T(n/2) + O(n)$	$3, 2, 1$	$\log_2 3 \approx 1.585 > 1$	$O(n^{\log_2 3}) \approx O(n^{1.6})$
$T(n) = 2T(n/2) + O(n)$	$2, 2, 1$	$\log_2 2 = 1 = d$	$O(n \log n)$
$T(n) = T(n/2) + O(n)$	$1, 2, 1$	$\log_2 1 = 0 < 1$	$O(n)$

چند مثال دیگر برای تمرین:

- جست‌وجوی دودویی $T(n) = T(n/2) + O(1)$: $a = 1, b = 2, d = 0$؛ $\log_2 1 = 0 = d$، پس $O(\log n)$.
- ضرب ماتریس استراسن $T(n) = 7T(n/2) + O(n^2)$: $\log_2 7 \approx 2.81 > 2$، پس $O(n^{2.81})$.
- $T(n) = 2T(n/4) + O(\sqrt{n})$: $\log_4 2 = \frac{1}{2} = d$، پس $O(\sqrt{n} \log n)$.

شهود · نسبت a/b

مجموع اندازهٔ ورودی‌هایی که به زیرمسئله‌ها داده می‌شود $a \cdot \frac{n}{b} = \frac{a}{b}\,n$ است. اگر $a/b > 1$، در هر سطح «حجم کار» بیشتر می‌شود و برگ‌ها سنگین‌اند؛ اگر کوچک‌تر باشد، حجم کار در هر سطح کم می‌شود. پس هر چه a/b کوچک‌تر باشد، زمان اجرا بهتر است. ترفند کاراتسوبا (پایین‌تر) دقیقاً همین است: کم کردن a از ۴ به ۳.

۲.۶ اگر آرایه را به بیش از دو بخش تقسیم کنیم؟

سؤال طبیعی: اگر نصف کردن خوب است، شاید سه‌تایی یا ده‌تایی کردن بهتر باشد؟ فرض کنید آرایه را به a بخش تقسیم کنیم. در مرحلهٔ ادغام، برای انتخاب هر عنصر خروجی باید **کمینهٔ** a **عدد** (عناصر جلوی a لیست) را پیدا کنیم که $a-1$ مقایسه است. اسلاید این را تقریباً a گرفته است:

$$T(n) = a\,T(n/a) + a\,n.$$

در هر سطح $a \cdot n$ کار و $\log_a n$ سطح داریم، پس

$$T(n) = a\,n \log_a n = \frac{a}{\ln a} \cdot n \ln n.$$

چرا؟ · کدام a بهترین است؟

باید ضریب را کمینه کنیم. با هزینهٔ **دقیق** ادغام ($a-1$ مقایسه برای هر عنصر)، ضریب $\frac{a-1}{\ln a}$ می‌شود: برای $a=2$ برابر 1.44، برای $a=3$ برابر 1.82 و برای $a=4$ برابر 2.16. این تابع برای $a \geq 2$ اکیداً صعودی است، پس $a=2$ **بهترین انتخاب است**، همان‌طور که اسلاید می‌گوید.

(اگر با تقریب خامِ a مقایسه حساب کنیم، ضریب $\frac{a}{\ln a}$ در $a = e \approx 2.7$ کمینه است و مقدارش برای $a=2$ و $a=3$ تقریباً برابر است؛ این تقریب برای مقایسهٔ دقیق مناسب نیست.)

تکمیلی (فراتر از اسلاید) · ادغام aتایی با هرم

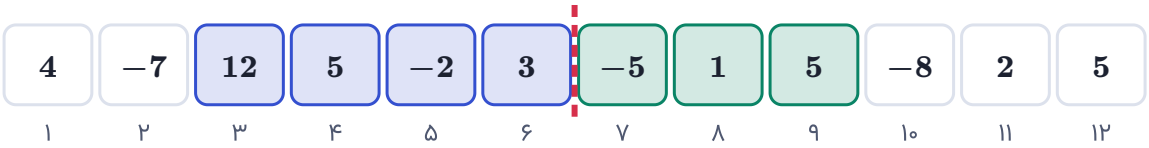
اگر کمینهٔ a عنصر را به‌جای جست‌وجوی خطی با یک هرم کمینه بیابیم، هر عنصر $O(\log a)$ هزینه دارد و کل زمان $n \log a \cdot \log_a n = n \log n$ می‌شود. یعنی با ساختار دادهٔ مناسب، تعداد بخش‌ها روی مرتبه اثری ندارد. این ایده در مرتب‌سازی فایل‌های بزرگ روی دیسک (مرتب‌سازی خارجی) کاربرد دارد.

۲.۷ زیرآرایهٔ بیشینه با تقسیم و حل

مسئلهٔ فصل ۱ را دوباره در نظر بگیرید: زیرآرایهٔ پیوسته‌ای با بیشترین مجموع. تا این‌جا الگوریتم $O(n^2)$ با مجموع پیشوندی داشتیم. حالا با تقسیم و حل به $O(n \log n)$ می‌رسیم.

راهبرد کلی: آرایه را به دو زیرآرایهٔ هم‌اندازه تقسیم کن. جواب بهینه یکی از این دو حالت را دارد:

- **حالت ۱:** جواب کاملاً درون یکی از دو نیمه است. آن را بازگشتی پیدا می‌کنیم.
- **حالت ۲:** جواب از خط وسط **عبور می‌کند**. این را مستقیم و در زمان خطی پیدا می‌کنیم.



شکل ۲-۵ خط قرمز مرز دو نیمه است. هر زیرآرایهٔ قطع‌کننده از یک **پسوند** نیمهٔ چپ (آبی) و یک **پیشوند** نیمهٔ راست (سبز) تشکیل شده است.

چرا؟ · چرا حالت ۲ در زمان خطی حل می‌شود؟

زیرآرایهٔ $A[i..j]$ که $i \le m < j$ (و m وسط است) را می‌توان به دو قسمت مستقل شکست: $A[i..m]$ که **حتماً به m ختم می‌شود** و $A[m+1..j]$ که **حتماً از $m+1$ شروع می‌شود**. مجموع کل = مجموع قسمت چپ + مجموع قسمت راست، و انتخاب i هیچ اثری روی انتخاب بهترین j ندارد. پس کافی است بهترین پسوند چپ ($n/2$ گزینه) و بهترین پیشوند راست ($n/2$ گزینه) را جداگانه پیدا کنیم و جمع بزنیم. به‌جای $n^2/4$ جفت، فقط n حالت بررسی می‌شود.

```
MCS(l, r) ▷ max contiguous sum in A[l..r], S = prefix sums
```

```
1  if l = r then return max(0, A[l])
2  m = floor((l + r) / 2)
3  sol = max(MCS(l, m), MCS(m+1, r))          // case 1
4  Lsol = 0
5  for i = m downto l                          // best suffix of left half
6      if S[m] - S[i-1] > Lsol then
7          Lsol = S[m] - S[i-1]
8  Rsol = 0
9  for i = m+1 to r                            // best prefix of right half
10     if S[i] - S[m] > Rsol then
11         Rsol = S[i] - S[m]
12 return max(sol, Lsol + Rsol)                // case 2
```

نکته · تصحیح یک اندیس

مجموع $A[m+1..i]$ برابر $S[i]-S[m]$ است (نه $S[i]-S[m-1]$ که در اسلاید آمده؛ آن عبارت $A[m]$ را هم می‌شمارد). پیشوند S یک بار در ابتدا با $O(n)$ ساخته می‌شود.

مثال · اجرا روی آرایهٔ مثال

نیمهٔ چپ $4, -7, 12, 5, -2, 3$ و نیمهٔ راست $-5, 1, 5, -8, 2, 5$.

- **حالت ۱ (بازگشتی):** بهترین جواب نیمهٔ چپ $12+5-2+3=18$ و بهترین جواب نیمهٔ راست $5+2=7$ است.

- **پسوندهای چپ** (از خانهٔ ۶ به عقب): $3,\ 1,\ 6,\ \mathbf{18},\ 11,\ 15$. پس $Lsol = 18$.
- **پیشوندهای راست** (از خانهٔ ۷ به جلو): $-5,\ -4,\ \mathbf{1},\ -7,\ -5,\ 0$. پس $Rsol = 1$.
- **جواب:** $\max(18, 7, 18+1) = 19$ ✓.

زمان اجرا: دو فراخوانی بازگشتی روی نیمه‌ها و دو حلقهٔ خطی:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n).$$

۲.۸ جمع و ضرب اعداد صحیح

حالا به مسئله‌ای می‌رسیم که فرض «هزینهٔ واحد» در آن معنا ندارد: کار با اعداد خیلی بزرگ (مثلاً هزاران رقمی در رمزنگاری). اندازهٔ ورودی تعداد **بیت‌ها** (n) است و «عمل پایه» یعنی یک عمل روی بیت‌ها.

جمع

جمع: دو عدد صحیح n بیتی a و b داده شده؛ $a+b$ را حساب کنید. روش دبستانی (ستونی با رقم نقلی) $\Theta(n)$ عمل بیتی انجام می‌دهد.

```
carry:   1 1 1 1 1 1 0 1
           1 1 0 1 0 1 0 1      (213)
       +   0 1 1 1 1 1 0 1      (125)
       ----------------------
         1 0 1 0 1 0 0 1 0      (338)
```

قضیه · بهینه بودن جمع دبستانی

هر الگوریتم جمع باید دست‌کم همهٔ $2n$ بیت ورودی را بخواند (تغییر هر بیت، جواب را تغییر می‌دهد)، پس $\Omega(n)$ زمان لازم است. روش دبستانی $O(n)$ است، پس **بهینه** است.

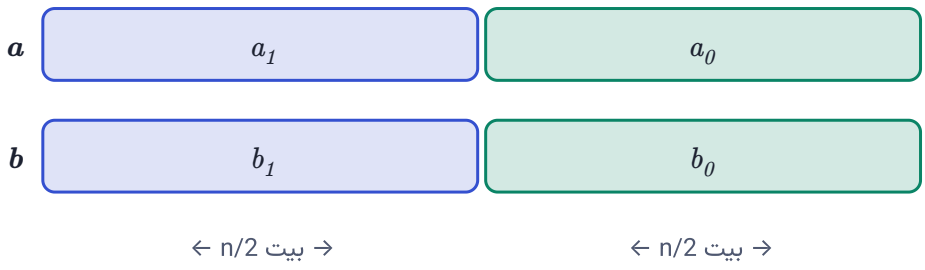
ضرب

ضرب: دو عدد صحیح n بیتی a و b داده شده؛ $a \times b$ را حساب کنید. روش دبستانی $\Theta(n^2)$ عمل بیتی انجام می‌دهد: n حاصل‌ضرب جزئی که هر کدام n بیت دارد، و جمع آن‌ها.

```
              1 1 0 1 0 1 0 1      (213)
            x 0 1 1 1 1 1 0 1      (125)
            ------------------
              1 1 0 1 0 1 0 1
            0 0 0 0 0 0 0 0
          1 1 0 1 0 1 0 1
        1 1 0 1 0 1 0 1
      1 1 0 1 0 1 0 1
    1 1 0 1 0 1 0 1
  1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
----------------------------
0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 1    (26625)
```

پرسش: آیا ضرب دبستانی هم بهینه است؟ برخلاف جمع، این‌جا کران پایین ساده فقط $\Omega(n)$ است و بین n و n^2 فاصلهٔ زیادی هست. تقسیم و حل نشان می‌دهد پاسخ **منفی** است.

گرم کردن: تقسیم و حل ساده



شکل ۲-۶ هر عدد n بیتی را به نیمهٔ پرارزش و نیمهٔ کم‌ارزش (هر کدام $n/2$ بیت) تقسیم می‌کنیم.

برای ضرب دو عدد n بیتی، هر کدام را دو نیم می‌کنیم:

$$a = 2^{n/2}\,a_1 + a_0, \qquad b = 2^{n/2}\,b_1 + b_0.$$

$$ab = \left(2^{n/2}a_1 + a_0\right)\left(2^{n/2}b_1 + b_0\right) = 2^{n}\,a_1b_1 + 2^{n/2}\,(a_1b_0 + a_0b_1) + a_0b_0.$$

ضرب در $2^{n/2}$ یا 2^n فقط یعنی **شیفت به چپ** (اضافه کردن صفر به انتها) که $O(n)$ است. پس چهار ضرب $n/2$ بیتی بازگشتی داریم، به‌علاوهٔ چند جمع و شیفت خطی:

$$T(n) = \underbrace{4T(n/2)}_{\text{فراخوانی‌های بازگشتی}} + \underbrace{\Theta(n)}_{\text{جمع و شیفت}} \implies T(n) = \Theta(n^2).$$

مثال · $a = 10001101,\ b = 11100001$

$a_1 = 1000 = 8$، $a_0 = 1101 = 13$، $b_1 = 1110 = 14$، $b_0 = 0001 = 1$. (یعنی $a = 141$، $b = 225$.)

$$ab = 2^8 \cdot (8 \cdot 14) + 2^4 (8 \cdot 1 + 13 \cdot 14) + 13 \cdot 1 = 28672 + 3040 + 13 = 31725. \checkmark$$

چرا؟ · چرا تقسیم و حل این‌جا سودی نداشت؟

با قضیهٔ اصلی: $a = 4$، $b = 2$، $d = 1$ و $\log_2 4 = 2 > 1$؛ برگ‌ها برنده‌اند و $n^{\log_2 4} = n^2$. چهار زیرمسئله با نصف اندازه، دقیقاً همان کار ضرب دبستانی است که به شکل دیگری چیده شده. برای بهبود باید **تعداد زیرمسئله‌ها را کم کنیم.**

۲.۹ ضرب کاراتسوبا

شهود · ترفند گاوس

ما در واقع به a_1b_0 و a_0b_1 جداگانه نیازی نداریم؛ فقط **جمعشان** را می‌خواهیم. حاصل‌ضرب $(a_1 + a_0)(b_1 + b_0)$ را باز کنید: $a_1b_1 + a_1b_0 + a_0b_1 + a_0b_0$. همهٔ چهار جمله در آن هست! اگر a_1b_1 و a_0b_0 را (که به هر حال لازم داریم) کم کنیم، جملهٔ وسط با **یک** ضرب به دست می‌آید.

$$a_1b_0 + a_0b_1 = (a_1 + a_0)(b_1 + b_0) - a_1b_1 - a_0b_0.$$

پس با سه حاصل‌ضرب کار تمام است:

$$ab = 2^n \underbrace{a_1b_1}_{P_1} + 2^{n/2}\Big(\underbrace{(a_1 + a_0)(b_1 + b_0)}_{P_2} - \underbrace{a_1b_1}_{P_1} - \underbrace{a_0b_0}_{P_3}\Big) + \underbrace{a_0b_0}_{P_3}.$$

الگوریتم · ضرب کاراتسوبا

1. دو عدد $n/2$ بیتی را جمع کن: $a_1 + a_0$ و $b_1 + b_0$.
2. سه ضرب $n/2$ بیتی را بازگشتی انجام بده: P_1, P_2, P_3.
3. با جمع، تفریق و شیفت جواب را بساز.

```
KARATSUBA(a, b, n)
1  if n = 1 then return a * b
2  split a = a1 * 2^(n/2) + a0  and  b = b1 * 2^(n/2) + b0
3  P1 = KARATSUBA(a1, b1, n/2)
4  P3 = KARATSUBA(a0, b0, n/2)
5  P2 = KARATSUBA(a1 + a0, b1 + b0, n/2 + 1)
6  return P1 * 2^n + (P2 - P1 - P3) * 2^(n/2) + P3      // shifts and adds: O(n)
```

زمان اجرا:

$$T(n) \le \underbrace{T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + T(1 + \lceil n/2 \rceil)}_{\text{فراخوانی‌های بازگشتی}} + \underbrace{\Theta(n)}_{\text{جمع، تفریق، شیفت}} \implies T(n) = O(n^{\log_2 3}) = O(n^{1.585}).$$

چرا؟ · آن «۱+» در $T(1+\lceil n/2\rceil)$ از کجا آمد و چرا مهم نیست؟

جمع دو عدد $n/2$ بیتی ممکن است یک رقم نقلی تولید کند، پس $a_1 + a_0$ تا $n/2+1$ بیت دارد. این یک بیت اضافه در هر سطح، اندازه را فقط به اندازهٔ یک ثابت تغییر می‌دهد و مرتبهٔ جواب را عوض نمی‌کند (می‌توان نشان داد $T(n) \le 3T(n/2+1) + cn$ همان جواب $O(n^{\log_2 3})$ را دارد). با قضیهٔ اصلی: $a = 3, b = 2, d = 1$ و $\log_2 3 \approx 1.585 > 1$.

مثال · کاراتسوبا روی همان مثال

$P_1 = 8 \cdot 14 = 112$، $P_3 = 13 \cdot 1 = 13$، $P_2 = (8+13)(14+1) = 21 \cdot 15 = 315$.

جملهٔ وسط: $P_2 - P_1 - P_3 = 315 - 112 - 13 = 190$ (درست برابر $8 \cdot 1 + 13 \cdot 14 = 190$).

$$ab = 112 \cdot 2^8 + 190 \cdot 2^4 + 13 = 28672 + 3040 + 13 = 31725. \checkmark$$

همین ایده در مبنای ده هم کار می‌کند. برای 1234×5678 با تقسیم به دو رقمی‌ها: $P_1 = 12 \cdot 56 = 672$، $P_3 = 34 \cdot 78 = 2652$، $P_2 = 46 \cdot 134 = 6164$ و جملهٔ وسط $6164 - 672 - 2652 = 2840$:

$$1234 \times 5678 = 672 \cdot 10^4 + 2840 \cdot 10^2 + 2652 = 7006652.$$

n (تعداد بیت)	دبستانی n^2	کاراتسوبا $n^{1.585}$	نسبت
10^3	10^6	$\approx 5.7 \times 10^4$	≈ 18
10^6	10^{12}	$\approx 3.2 \times 10^9$	≈ 310

تکمیلی (فراتر از اسلاید) · ادامهٔ داستان ضرب

کتابخانه‌های اعداد بزرگ (مثلاً در پایتون و GMP) برای اعداد متوسط از کاراتسوبا استفاده می‌کنند؛ برای اعداد کوچک، روش دبستانی به‌خاطر ثابت کمترش سریع‌تر است. الگوریتم‌های سریع‌تری هم وجود دارند: تقسیم به سه بخش (Toom-Cook)، روش‌های مبتنی بر تبدیل فوریهٔ سریع (Schönhage–Strassen) و در سال ۲۰۱۹ الگوریتم هاروی و فان‌درهوون با زمان $O(n \log n)$.

۲.۱۰ روش حدس و استقرا

قضیهٔ اصلی فقط رابطه‌هایی به شکل $aT(n/b) + f(n)$ را حل می‌کند. برای رابطه‌های دیگر (مثلاً دو زیرمسئلهٔ نابرابر) ابزار عمومی‌تری لازم است: **حدس و استقرا** (که به آن «روش جای‌گذاری» یا substitution method هم می‌گویند).

۱ **حدس:** جوابی برای رابطه حدس بزن (با باز کردن رابطه، درخت بازگشت، یا تجربه).

۲ **اثبات:** با استقرای ریاضی ثابت کن حدست درست است.

۳ **پاک‌نویس:** وانمود کن هرگز گام ۱ را انجام نداده‌ای و فقط قضیه و اثباتِ گام ۲ را تمیز بنویس.

مثال اول: $T(n) = 2T(n/2) + n$

با $T(0) = 0$ و $T(1) = 1$. قضیهٔ اصلی می‌گوید $T(n) = O(n \log n)$. این را با حدس و استقرا ثابت می‌کنیم (فرض: n توانی از ۲).

گام ۱: حدس با باز کردن رابطه

$$
\begin{aligned}
T(n) &= 2\,T(n/2) + n \\
&= 2\left(2\,T(n/4) + \tfrac{n}{2}\right) + n = 4\,T(n/4) + 2n && \text{باز کردن و ساده‌سازی} \\
&= 4\left(2\,T(n/8) + \tfrac{n}{4}\right) + 2n = 8\,T(n/8) + 3n && \text{باز کردن دوباره} \\
&\;\;\vdots \\
&= 2^t\,T(n/2^t) + t \cdot n. && \text{حدس الگو}
\end{aligned}
$$

با قرار دادن $t = \log n$ (تا $n/2^t = 1$ شود): $T(n) = n\,T(1) + n \log n = n(\log n + 1)$.

گام ۲: اثبات حدس با استقرا

فرض استقرا: $T(n) = n(\log n + 1)$.

پایه ($n = 1$): $T(1) = 1 = 1 \cdot (\log 1 + 1)$ ✓.

گام: فرض کنید حکم برای همهٔ $1 \le n < k$ برقرار است. آنگاه

$$T(k) = 2\,T(k/2) + k = 2 \cdot \frac{k}{2}\left(\log \frac{k}{2} + 1\right) + k = k\,(\log k - 1 + 1) + k = k(\log k + 1).$$

(خط اول تعریف T، تساوی دوم فرض استقرا برای $k/2 < k$، و بقیه ساده‌سازی است.) پس حکم برای $n = k$ هم برقرار است.

نتیجه: برای همهٔ $n \geq 1$ (توان ۲)، $T(n) = n(\log n + 1) = O(n \log n)$.

■

مثال دوم: وقتی ثابت را نمی‌دانیم

$$T(n) = 2\,T(n/2) + 32\,n, \qquad T(2) = 2.$$

حدس: $O(n \log n)$. اما حدس دقیقی دربارهٔ **شکل کامل** جواب نداریم؛ ضریب پیشرو چیست؟ آیا باز هم می‌توانیم گام ۲ را انجام دهیم؟ اول یک اثبات **غلط** ببینیم:

اشتباه رایج · این اثبات کجایش غلط است؟

فرض استقرا: $T(n) = O(n \log n)$. **پایه:** $T(2) = 2 = O(1) = O(2 \log 2)$.

گام: فرض کنید برای $n < k$، $T(n) = O(n \log n)$. آنگاه $T(k) = 2T(k/2) + 32k = 2 \cdot O\left(\frac{k}{2} \log \frac{k}{2}\right) + 32k =$ $O(k \log k)$. پس حکم برای k برقرار است.

با **همین استدلال** می‌توان «ثابت کرد» $T(n) = O(n)$: فرض کن $T(n) = O(n)$ برای $n < k$؛ آنگاه $T(k) = 2 \cdot O(k/2) + 32k = O(k)$. اما این **نادرست** است!

مشکل: جملهٔ «$T(n) = O(n)$ برای $n < k$» بی‌معناست. O دربارهٔ رفتار تابع **وقتی n به بی‌نهایت می‌رود** حرف می‌زند و برای تعداد متناهی n همیشه درست است. تعریف O یک ثابت c واحد برای **همهٔ** nها می‌خواهد. در اثبات غلط، اگر $T(n) \leq c\,n$ را فرض کنیم، به $T(k) \leq (c + 32)\,k$ می‌رسیم: ثابت در هر گام بزرگ‌تر می‌شود، و پس از $\log n$ گام به $(c + 32 \log n)\,n$ می‌رسد که دیگر ثابت نیست.

جمع‌بندی · قانون طلایی

در فرض استقرا هرگز O ننویسید. ثابت را **صریح** بنویسید ($T(n) \leq C \cdot g(n)$) و ثابت کنید **همان** C در گام استقرا حفظ می‌شود.

گام ۲ درست: ثابت را باز بگذار و در آخر پیدایش کن

حدس: $T(n) \leq C \cdot n \log n$ برای ثابتی مثل C که هنوز نمی‌دانیم.

پایه: $T(2) = 2 \le C \cdot 2\log 2 = 2C$، که برای $C \ge 1$ برقرار است.

گام: فرض کنید برای $n < k$ برقرار باشد. آنگاه

$$\begin{aligned} T(k) &= 2\,T(k/2) + 32k \;\le\; 2C\,\frac{k}{2}\log\frac{k}{2} + 32k \\ &= Ck(\log k - 1) + 32k \;=\; Ck\log k + (32 - C)\,k \\ &\le Ck\log k \qquad C \ge 32 \text{ به شرط} \end{aligned}$$

پس با $C = 32$ (که شرط پایه را هم دارد) حکم برقرار است.

■

گام ۳: پاک‌نویس

$$.T(n) = O(n\log n)$$

فرض استقرا: $T(n) \le 32\,n\log n$. **پایه:** $T(2) = 2 \le 32 \cdot 2\log 2$. ✓

گام: برای $n < k$ فرض را داریم. $T(k) = 2T(k/2) + 32k \le 2 \cdot 32 \cdot \frac{k}{2}\log\frac{k}{2} + 32k = 32k(\log k - 1) + 32k = 32\,k\log k$.

نتیجه: برای همهٔ $n \ge 2$، $T(n) \le 32\,n\log n$. طبق تعریف O با $n_0 = 2$ و $c = 32$، $T(n) = O(n\log n)$.

■

مثال سوم: دو زیرمسئلهٔ نابرابر

$$T(n) \le T\!\left(\frac{n}{5}\right) + T\!\left(\frac{7n}{10}\right) + n \quad (n > 10), \qquad T(n) = 1 \quad (1 \le n \le 10).$$

این رابطه (که در فصل ۳، الگوریتم انتخاب قطعی، دوباره می‌بینیم) شکل $aT(n/b)$ ندارد، پس قضیهٔ اصلی کاربرد ندارد. حدس می‌زنیم $O(n)$ باشد (چون $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$؛ کل کار در هر سطح کم می‌شود).

اثبات · گام ۲: C را در طول اثبات پیدا می‌کنیم

فرض استقرا: $T(n) \le C\,n$. **پایه:** $T(n) = 1 \le Cn$ برای $1 \le n \le 10$؛ کافی است $C \ge 1$.

گام: برای $k > 10$ و فرض برقرار برای $1 \le n < k$:

$$T(k) \le k + C \cdot \frac{k}{5} + C \cdot \frac{7k}{10} = k + \frac{9C}{10}\,k \;\overset{?}{\le}\; C\,k.$$

نامساوی آخر یعنی $k \le \frac{C}{10}k$، یعنی $C \ge 10$. پس $C = 10$ کار می‌کند: $T(k) \le k + 2k + 7k = 10k$.

نتیجه: برای همهٔ $n \ge 1$، $T(n) \le 10n$؛ یعنی $T(n) = O(n)$ با $n_0 = 1$ و $c = 10$.

■

نکته · کف و سقف

در عمل $n/5$ و $7n/10$ باید گرد شوند. چون T صعودی است و ما فقط نامساوی $\le$ می‌خواهیم، با کمی دقت (و احتمالاً تغییر جزئی ثابت‌ها) همین اثبات کار می‌کند. در این درس کف و سقف را نادیده می‌گیریم مگر خلافش خواسته شود.

حدس نادرست

$$T(n) \le T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n \quad (n > 3), \qquad T(n) = 1 \quad (1 \le n \le 3).$$

دوباره $O(n)$ را حدس می‌زنیم و فرض استقرا را $T(n) \le Cn$ می‌گیریم:

$$T(k) \le k + C \cdot \frac{k}{3} + C \cdot \frac{2k}{3} = k + C\,k \overset{?}{\le} C\,k.$$

این نامساوی برای **هیچ** Cای برقرار نیست (همیشه k اضافه داریم). نتیجه: یا حدس ما غلط است، یا باید اثبات را تغییر دهیم. این‌جا **حدس غلط است**: چون $\frac{1}{3} + \frac{2}{3} = 1$، هر سطح درخت بازگشت کل n کار دارد و عمق درخت $\Theta(\log n)$ است، پس جواب $\Theta(n \log n)$ است.

تکمیلی (فراتر از اسلاید) · اثبات $O(n \log n)$ برای همین رابطه

فرض استقرا: $T(n) \le C\,n \log n$. آنگاه

$$T(k) \le k + C\frac{k}{3}\log\frac{k}{3} + C\frac{2k}{3}\log\frac{2k}{3} = Ck\log k + k\Big(1 - C\underbrace{\big(\tfrac{1}{3}\log 3 + \tfrac{2}{3}\log\tfrac{3}{2}\big)}_{\approx 0.918}\Big).$$

عبارت داخل پرانتز بزرگ برای $C \ge 1/0.918 \approx 1.09$ نامثبت است، پس با $C = 2$ (و بررسی پایه) حکم ثابت می‌شود.

قضیه · رابطه‌های دو شاخه‌ای

$$T(n) \le T(an) + T(bn) + O(n) \implies T(n) = \begin{cases} O(n) & a + b < 1 \\ O(n \log n) & a + b = 1 \end{cases}$$

شهود با درخت بازگشت: اگر ریشه cn کار کند، دو فرزندش $c(an)+c(bn)=(a+b)\,cn$ کار می‌کنند، نوه‌ها $(a+b)^2cn$ و ... . اگر $a+b<1$، این یک سری هندسی نزولی است با جمع حداکثر $\frac{cn}{1-(a+b)}=O(n)$. اگر $a+b=1$، هر سطح حداکثر cn کار دارد و عمق درخت $O(\log n)$ است.

اثبات استقرایی حالت اول: با فرض $T(n)\le Cn$: $T(k)\le Cak+Cbk+ck=Ck(a+b)+ck\le Ck$ اگر $C\ge \frac{c}{1-a-b}$.

حالت دوم: مثل اثبات کادر قبل؛ با فرض $T(n)\le Cn\log n$ به شرط $C\ge c/H$ می‌رسیم که $H=a\log\frac{1}{a}+b\log\frac{1}{b}>0$.

■

مثال آخر: تقویت فرض استقرا

$$T(n)\le 2\,T\!\left(\frac{n}{2}\right)+1\quad(n\ge 2),\qquad T(1)=1.$$

حدس: $O(n)$. فرض استقرا: $T(n)\le Cn$. پایه: $1\le C$، پس $C\ge 1$. گام:

$$T(k)\le 2\,T(k/2)+1\le 2C\cdot\frac{k}{2}+1=Ck+1\overset{?}{\le}Ck.$$

برای هیچ Cای برقرار نیست! آیا حدس غلط است؟ **نه!** با باز کردن رابطه می‌بینیم $T(n)=1+2+4+\cdots+n=2n-1$ که واقعاً $O(n)$ است. مشکل در حدس نیست؛ **فرض استقرا ضعیف است**: آن «+۱» اضافه در هر گام جمع می‌شود و فرض Cn جایی برای جذبش ندارد.

اثبات · فرض قوی‌تر: یک جملهٔ مرتبه‌پایین کم کن

فرض استقرا: $T(n)\le Cn+B$ (که B می‌تواند منفی باشد). **پایه:** $T(1)\le C+B\ge 1$.

گام:

$$T(k)\le 2\left(C\frac{k}{2}+B\right)+1=Ck+2B+1\overset{?}{\le}Ck+B\iff B\le -1.$$

با $B=-1$، پایه $C\ge 2$ می‌خواهد. پس **حدس جدید** $T(n)\le 2n-1$ با استقرا ثابت می‌شود.

■

شهود · تناقضِ ظاهری

$T(n) \le 2n$ را **نمی‌توان** مستقیم با استقرا ثابت کرد، اما $T(n) \le 2n - 1$ را که **قوی‌تر** است، می‌توان! دلیلش این است که در گام استقرا، فرض استقرا هم قوی‌تر شده است: وقتی بیشتر فرض می‌کنیم، می‌توانیم بیشتر نتیجه بگیریم. هر وقت گام استقرا «تقریباً» درست درآمد و فقط یک جملهٔ مرتبه‌پایین اضافه داشت، فرض را با کم کردن یک جملهٔ مرتبه‌پایین تقویت کنید.

جمع‌بندی · فصل ۲ در یک نگاه

- تقسیم و حل = تقسیم + حل بازگشتی + ترکیب. زمانش با یک رابطهٔ بازگشتی توصیف می‌شود.
- درخت بازگشت: هزینهٔ هر سطح را حساب و سطح‌ها را جمع کن. برگ‌ها $n^{\log_b a}$ تا هستند.
- قضیهٔ اصلی: مقایسهٔ $f(n)$ با $n^{\log_b a}$. در حالت n^d: مقایسهٔ d با $\log_b a$.
- مرتب‌سازی ادغامی و زیرآرایهٔ بیشینه $O(n \log n)$؛ ضرب کاراتسوبا $O(n^{1.585})$ با کم کردن تعداد زیرمسئله‌ها از ۴ به ۳.
- حدس و استقرا: ثابت‌ها را صریح بنویس، C را در طول اثبات پیدا کن، و اگر لازم شد فرض را با کم کردن جملهٔ مرتبه‌پایین تقویت کن.

۲.۱۱ خودآزمایی فصل ۲

خودآزمایی

۱. $T(n) = 9T(n/3) + n^2$ را حل کنید.

پاسخ: $a = 9, b = 3, d = 2$؛ $\log_3 9 = 2 = d$، پس $\Theta(n^2 \log n)$.

۲. $T(n) = 2T(n/2) + n^2$ را حل کنید.

پاسخ: $\log_2 2 = 1 < 2 = d$، ریشه برنده است: $\Theta(n^2)$.

۳. $T(n) = 16T(n/4) + n$ را حل کنید.

پاسخ: $\log_4 16 = 2 > 1$، برگ‌ها برنده‌اند: $\Theta(n^2)$.

۴. اگر در ضرب تقسیم و حل بتوانیم با ۵ ضرب یک‌سوم‌اندازه کار را انجام دهیم ($T(n) = 5T(n/3) + O(n)$)، زمان چه می‌شود؟ از کاراتسوبا بهتر است؟

پاسخ: $\log_3 5 \approx 1.465 < 1.585$، پس $O(n^{1.465})$ که از کاراتسوبا بهتر است. (این ایدهٔ الگوریتم Toom-3 است.)

5. چرا اثبات «$T(n) = O(n)$ برای $T(n) = 2T(n/2) + n$» با فرض استقرای $T(n) = O(n)$ غلط است؟

پاسخ: چون ثابت پنهان در O در هر گام عوض می‌شود ($c \to c+1$). استقرا باید با ثابت صریح انجام شود: $T(n) \le cn$ نتیجه می‌دهد $T(k) \le ck + k$ که از ck بیشتر است.

6. در الگوریتم تقسیم و حل زیرآرایهٔ بیشینه، اگر به‌جای مجموع پیشوندی، مجموع‌های حالت ۲ را هر بار از نو با جمع زدن حساب کنیم، زمان چه می‌شود؟

پاسخ: اگر پسوندها را با یک جمع تجمعی از وسط به چپ (و پیشوندها از وسط به راست) حساب کنیم، باز هم $O(n)$ برای هر فراخوانی است و جواب $O(n \log n)$ می‌ماند. اما اگر هر مجموع را مستقل از صفر حساب کنیم، حالت ۲ $O(n^2)$ و کل زمان $O(n^2)$ می‌شود.

فصل ۳

الگوریتم‌های تصادفی

الگوریتمی که سکه می‌اندازد! می‌بینیم چطور کمی تصادف، الگوریتم را از دست ورودی‌های بد نجات می‌دهد و چطور با امید ریاضی و متغیرهای نشانگر زمانش را تحلیل کنیم.

مبانی احتمال · متغیر تصادفی و امید · میانهٔ تقریبی · مسئلهٔ استخدام · انتخاب · کوئیک‌سورت

۳.۱ تصادف در طراحی الگوریتم

تا این‌جا با الگوهای حریصانه، تقسیم و حل، برنامه‌ریزی پویا و شارش شبکه آشنا شدیم. الگوی پنجم **تصادفی‌سازی** است.

تعریف · الگوریتم تصادفی

الگوریتم تصادفی (randomized algorithm) الگوریتمی است که رفتارش نه فقط به ورودی، بلکه به **انتخاب‌های تصادفی**‌ای که خود الگوریتم در حین اجرا انجام می‌دهد هم بستگی دارد. پس روی یک ورودی ثابت، دو اجرای مختلف ممکن است مسیر متفاوتی بروند.

نکته · فرض مولد عدد تصادفی

فرض می‌کنیم تابع $\text{Random}(a, b)$ را داریم که برای دو عدد صحیح $a < b$، عدد صحیح r با $a \le r \le b$ را **به‌طور یکنواخت تصادفی** برمی‌گرداند (یعنی هر کدام از $b - a + 1$ مقدار با احتمال برابر). فرض می‌کنیم این تابع در $O(1)$ اجرا می‌شود؛ یا دقیق‌تر، پرتاب یک سکهٔ سالم یک واحد زمان می‌برد.

چرا؟ · چرا تصادف؟

تصادف می‌تواند به **ساده‌ترین**، **سریع‌ترین** یا حتی **تنها** الگوریتم شناخته‌شده برای یک مسئله منجر شود. نمونه‌ها: الگوریتم‌های گراف، کوئیک‌سورت، درهم‌سازی (hashing)، توازن بار و رمزنگاری.

شهود اصلی: در الگوریتم قطعی، یک «دشمن» که الگوریتم را می‌شناسد همیشه می‌تواند بدترین ورودی را بسازد (مثلاً آرایهٔ مرتب برای کوئیک‌سورتی که همیشه اولین عنصر را محور می‌گیرد). اما اگر الگوریتم انتخاب‌هایش را با سکه انجام دهد، دشمن نمی‌داند الگوریتم چه خواهد کرد و نمی‌تواند ورودیِ همیشه-بد بسازد. بدشانسی ممکن است، ولی بسیار نادر است.

۳.۲ فضای احتمال متناهی

برای تحلیل الگوریتم‌های تصادفی به زبان احتمال نیاز داریم. فقط فضاهای **متناهی** لازم‌اند و همه چیز با جمع ساده بیان می‌شود.

آزمایش فرایندی که یکی از نتایج ممکنِ مشخصی را تولید می‌کند (مثل پرتاب تاس).

فضای نمونه مجموعهٔ همهٔ نتایج ممکن آزمایش؛ با Ω نشان می‌دهیم.

پیشامد هر زیرمجموعه از فضای نمونه (مثل «عدد زوج آمد» = $\{2,4,6\}$).

پیشامد ساده هر عضو تکی $\omega \in \Omega$.

تعریف · فضای احتمال متناهی

زوج (Ω, P) که Ω یک مجموعهٔ متناهی است و P یک اندازهٔ جمع‌پذیر روی زیرمجموعه‌های Ω با $P[\Omega] = 1$. کافی است P را روی پیشامدهای ساده مشخص کنیم: تابعی $P : \Omega \to [0,1]$ با $\sum_{\omega \in \Omega} P[\omega] = 1$. آنگاه احتمال هر پیشامد A برابر است با

$$P[A] = \sum_{a \in A} P[a].$$

تعریف · توزیع یکنواخت

ساده‌ترین مثال: $P[A] = \dfrac{|A|}{|\Omega|}$ برای هر $A \subseteq \Omega$. یعنی همهٔ نتایج هم‌احتمال‌اند، مثل پرتاب تاس سالم.

مثال · چند فضای احتمال

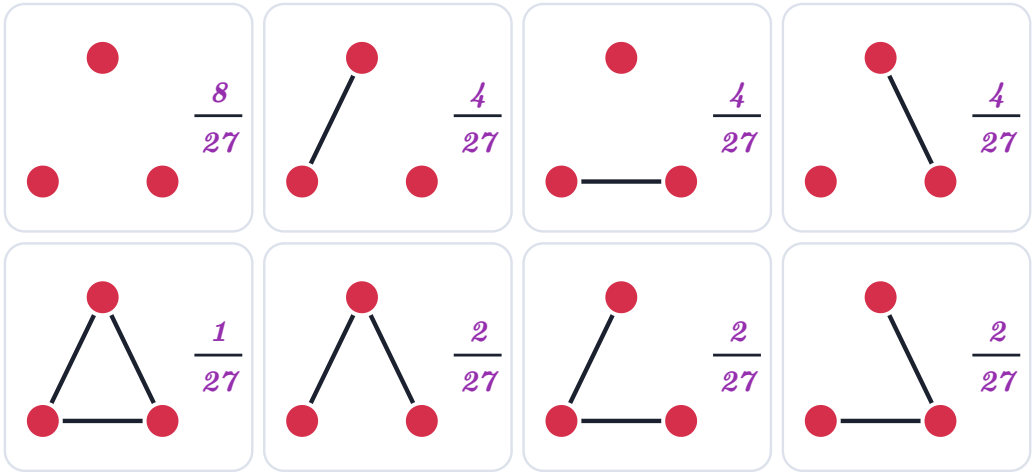
- **یک سکهٔ سالم:** $\Omega = \{H, T\}$، $P[H] = P[T] = \frac{1}{2}$. ($H$ شیر، T خط)
- **دو بار پرتاب سکه:** $\Omega = \{(H,H),(H,T),(T,H),(T,T)\}$ و هر کدام $\frac{1}{4}$.
- **یک تاس سالم:** $\Omega = \{1,\dots,6\}$ و $P[1] = \cdots = P[6] = \frac{1}{6}$.
- **تاس و سپس سکه:** $\Omega = \{(1,H),\dots,(6,H),(1,T),\dots,(6,T)\}$ و $P[(i,H)] = P[(i,T)] = \frac{1}{12}$.

فضای احتمال $G(n,p)$: گراف تصادفی

آزمایش: گرافی با n رأس می‌سازیم که در آن هر یال (از $\binom{n}{2}$ یال ممکن)، مستقل از بقیه، با احتمال p وجود دارد. فضای نمونه همهٔ گراف‌های ممکن روی n رأسِ ثابت است. احتمال یک گراف مشخص با m یال:

$$P[\text{گراف با } m \text{ یال}] = p^m\,(1-p)^{\binom{n}{2}-m}.$$

(m یالِ موجود هر کدام با احتمال p آمده‌اند و $\binom{n}{2} - m$ یالِ غایب هر کدام با احتمال $1-p$ نیامده‌اند.)



شکل ۳-۱ هشت عضو فضای نمونهٔ $G(3, \frac{1}{3})$. گراف بی‌یال $(\frac{2}{3})^3 = \frac{8}{27}$، هر گراف یک‌یالی $\frac{1}{3}(\frac{2}{3})^2 = \frac{4}{27}$، هر گراف دویالی $\frac{2}{27}$ و مثلث $\frac{1}{27}$. جمع: $8 + 3 \cdot 4 + 3 \cdot 2 + 1 = 27$. ✓

چطور این آزمایش را انجام دهیم (یک گراف تصادفی بسازیم)؟

- **روش ۱:** برای هر یال یک سکهٔ اریب بینداز که با احتمال p شیر بیاید؛ اگر شیر آمد یال را بگذار.
- **روش ۲:** همهٔ اعضای فضای نمونه را با احتمالشان فهرست کن. سپس عددی صحیح را به‌طور یکنواخت از $[1, 27]$ انتخاب کن: اگر بین ۱ تا ۸ بود گراف بی‌یال، اگر بین ۹ تا ۱۲ بود اولین گراف یک‌یالی، و به همین ترتیب.

نکته · روش ۲ همیشه ممکن است، ولی ...

روش ۲ برای هر فضای متناهی کار می‌کند، اما فهرست کردن کل فضا معمولاً نمایی است ($G(n,p)$ دارای $2^{\binom{n}{2}}$ عضو است). روش ۱ فقط $\binom{n}{2}$ سکه می‌خواهد. هنر طراحی، یافتن روش‌هایی مثل روش ۱ است.

مجموعه‌های مستقل تصادفی

گراف G داده شده است. فضای احتمالی را در نظر بگیرید که فضای نمونه‌اش مجموعه‌های مستقل G (زیرمجموعه‌ای از رأس‌ها که هیچ یالی بین دو رأسش نیست) و توزیعش یکنواخت است. یک روش ساده برای نمونه‌برداری: همهٔ مجموعه‌های مستقل را تولید و از ۱ تا m شماره‌گذاری کن، سپس عدد k را یکنواخت از $[1, m]$ بردار و مجموعهٔ شمارهٔ k را گزارش کن. این روش کارا نیست، چون m می‌تواند نمایی باشد. **روش سریع‌تری هست؟**

تکمیلی (فراتر از اسلاید) · پاسخ کوتاه

در حالت کلی این مسئله دشوار است: حتی **شمردن** مجموعه‌های مستقل یک گراف مسئله‌ای سخت است. برای خانواده‌هایی از گراف‌ها (مثلاً با بیشینهٔ درجهٔ کم) روش‌های نمونه‌برداری تقریبی مبتنی بر زنجیرهٔ مارکوف وجود دارد. این مثال نشان می‌دهد که «تعریف یک فضای احتمال» و «نمونه‌برداری کارا از آن» دو چیز کاملاً متفاوت‌اند.

۳.۳ متغیر تصادفی

تعریف · متغیر تصادفی

متغیر تصادفی X **تابعی** از فضای نمونه به اعداد حقیقی است: $X : \Omega \to \mathbb{R}$. به هر نتیجهٔ ممکنِ آزمایش یک عدد نسبت می‌دهد.

اشتباه رایج · نام گمراه‌کننده

متغیر تصادفی یک **تابع** است؛ نه متغیر است و نه تصادفی! آنچه تصادفی است ورودی آن (نتیجهٔ آزمایش) است. چون ورودی تصادفی است، خروجی هم عددی تصادفی می‌شود.

توزیع متغیر تصادفی X:

$$P[X = r] = \sum_{\omega:\ X(\omega)=r} P[\omega].$$

اگر پیشامد $A_r = \{\omega : X(\omega) = r\}$ را تعریف کنیم، آنگاه $P[X = r] = P[A_r]$. یعنی «$X = r$» نام کوتاه یک پیشامد است.

مثال · تعداد شیرها در سه پرتاب

سکهٔ سالمی سه بار پرتاب می‌شود. $X(\omega)$ تعداد شیرها در نتیجهٔ ω است:

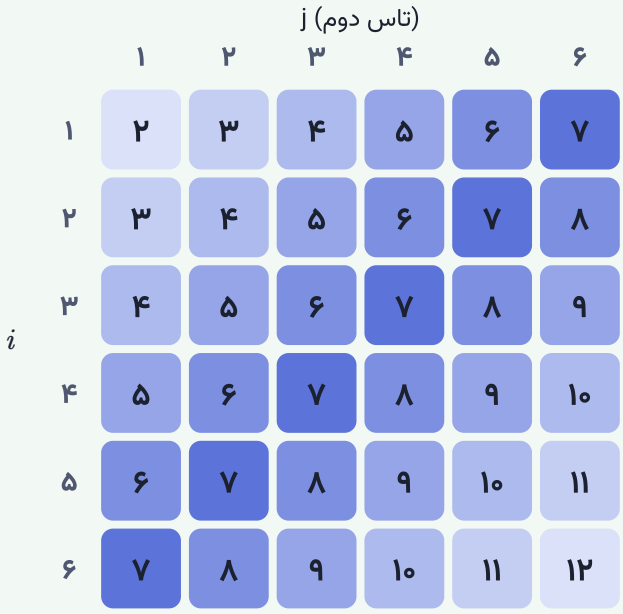
$X(HHH) = 3$؛ $X(HHT) = X(HTH) = X(THH) = 2$؛ $X(TTH) = X(THT) = X(HTT) = 1$؛ $X(TTT) = 0$.

هر کدام از ۸ نتیجه احتمال $\frac{1}{8}$ دارد، پس:

$$P[X=0] = \tfrac{1}{8},\quad P[X=1] = \tfrac{3}{8},\quad P[X=2] = \tfrac{3}{8},\quad P[X=3] = \tfrac{1}{8}.$$

مثال · مجموع دو تاس

X مجموع اعداد دو تاس است. مقادیر X برای ۳۶ نتیجهٔ ممکن (i, j) در جدول زیر آمده است. خانه‌های هم‌رنگ روی قطرهای فرعی هم‌مجموع‌اند.



شکل ۳-۲ عدد ۷ روی بلندترین قطر (۶ خانه) است، پس محتمل‌ترین مجموع است: $P[X=7]=\frac{6}{36}$. در حالت کلی $P[X=k]=\frac{6-|k-7|}{36}$ برای $2\le k\le 12$.

۳.۴ امید ریاضی و خطی بودن آن

تعریف · امید ریاضی

امید ریاضی (expected value) متغیر تصادفی X روی فضای نمونهٔ Ω:

$$E[X]=\sum_{\omega\in\Omega}P[\omega]\,X(\omega)=\sum_{r}P[X=r]\cdot r.$$

تساوی دوم با دسته‌بندی نتیجه‌ها بر اساس مقدار X به دست می‌آید. امید ریاضی یعنی میانگین وزن‌دار مقادیر، با وزن احتمال.

مثال · امید تعداد یال‌های گراف تصادفی در $G(3,\frac{1}{3})$

فرض کنید $X(\omega)$ تعداد یال‌های گراف ω باشد. با فرمول اول، روی ۸ گراف جمع می‌زنیم:

$$E[X]=\tfrac{8}{27}\cdot 0+\tfrac{4}{27}\cdot 1+\tfrac{4}{27}\cdot 1+\tfrac{4}{27}\cdot 1+\tfrac{2}{27}\cdot 2+\tfrac{2}{27}\cdot 2+\tfrac{2}{27}\cdot 2+\tfrac{1}{27}\cdot 3.$$

با گروه‌بندی بر اساس مقدار (فرمول دوم):

$$E[X] = \underbrace{\tfrac{8}{27}}_{P[X=0]} \cdot 0 + \underbrace{\tfrac{12}{27}}_{P[X=1]} \cdot 1 + \underbrace{\tfrac{6}{27}}_{P[X=2]} \cdot 2 + \underbrace{\tfrac{1}{27}}_{P[X=3]} \cdot 3 = \frac{12+12+3}{27} = 1.$$

قضیه · خطی بودن امید ریاضی

اگر $X_1, \dots, X_n$ متغیرهای تصادفی روی Ω و a, b دو عدد حقیقی باشند:

$$\text{(i)}\ \ E\Big[\sum_{i=1}^{n} X_i\Big] = \sum_{i=1}^{n} E[X_i], \qquad \text{(ii)}\ \ E[aX+b] = a\,E[X] + b.$$

(i) برای دو متغیر (و با استقرا برای n متغیر):

$$E[X_1 + X_2] = \sum_{\omega} P[\omega]\big(X_1(\omega) + X_2(\omega)\big) = \sum_{\omega} P[\omega]X_1(\omega) + \sum_{\omega} P[\omega]X_2(\omega) = E[X_1] + E[X_2].$$

$$E[aX+b] = \textstyle\sum_{\omega} P[\omega]\big(aX(\omega)+b\big) = a\sum_{\omega} P[\omega]X(\omega) + b\underbrace{\sum_{\omega} P[\omega]}_{=1} = aE[X] + b. \text{ (ii)}$$

■

اشتباه رایج · به استقلال نیازی نیست!

در اثبات بالا هیچ جا از استقلال X_1 و X_2 استفاده نکردیم. **خطی بودن امید برای هر متغیرهای تصادفی‌ای برقرار است، حتی کاملاً وابسته.** همین ویژگی آن را به قدرتمندترین ابزار تحلیل الگوریتم‌های تصادفی تبدیل می‌کند. (در مقابل، $E[XY] = E[X]E[Y]$ فقط برای متغیرهای مستقل درست است.)

مثال · امید مجموع دو تاس

راه سخت: ۳۶ نتیجه را با مقدار X و احتمالشان فهرست کن و جمع بزن.

راه ساده: $X_1(i,j) = i$ (عدد تاس اول) و $X_2(i,j) = j$ (عدد تاس دوم) را تعریف کن. آنگاه $X = X_1 + X_2$ و

$$E[X_1] = E[X_2] = \frac{1+2+3+4+5+6}{6} = \frac{7}{2} \quad \Longrightarrow \quad E[X] = E[X_1] + E[X_2] = 7.$$

همین روش برای مثال گراف: $X = \sum_e X_e$ که $X_e = 1$ اگر یال e باشد. $E[X_e] = p$، پس $E[X] = \binom{n}{2}p = 3 \cdot \frac{1}{3} = 1$.
بدون فهرست کردن ۸ گراف!

۳.۵ متغیر نشانگر و جایگشت تصادفی

تعریف · متغیر تصادفی نشانگر

برای پیشامد A، متغیر نشانگر (indicator random variable) I_A را تعریف می‌کنیم:

$$I_A(\omega) = \begin{cases} 1 & \omega \in A \\ 0 & \omega \notin A \end{cases} \qquad \Longrightarrow \qquad E[I_A] = 1 \cdot P[A] + 0 \cdot P[\bar{A}] = P[A].$$

شهود · دستور پخت تحلیل با متغیر نشانگر

1. کمیتی را که می‌خواهی (مثلاً تعداد مقایسه‌ها) به صورت **جمع** چند متغیر نشانگر بنویس: $X = \sum_i X_i$.
2. برای هر i، فقط **احتمال** پیشامد مربوط را حساب کن: $E[X_i] = P[A_i]$.
3. با خطی بودن امید جمع بزن: $E[X] = \sum_i P[A_i]$.

این دستور را در ادامه برای نقاط ثابت جایگشت، مسئلهٔ استخدام و کوئیک‌سورت به کار می‌بریم.

تولید جایگشت تصادفی

مسئله: برای n داده‌شده، جایگشت تصادفی σ از $1, 2, \ldots, n$ را با **توزیع یکنواخت** تولید کنید (هر کدام از $n!$ جایگشت با احتمال $1/n!$).

راه ساده (ناکارا): همهٔ $n!$ جایگشت را تولید و شماره‌گذاری کن، سپس عدد k را یکنواخت انتخاب کن. **راه کاراتر:** $\sigma(1)$ (چپ‌ترین عنصر) را یکنواخت از $\{1, \ldots, n\}$ انتخاب کن، سپس $\sigma(2)$ را یکنواخت از $\{1, \ldots, n\} - \{\sigma(1)\}$، و به همین ترتیب. چطور این را برنامه‌نویسی کنیم؟ با جابه‌جایی درجا:

```
RANDOM-PERMUTATION(A[1..n]) ▷ Fisher-Yates shuffle
```

```
1  for i = 1 to n
2      r = Random(i, n)
3      swap(A[i], A[r])
```

در دور iام، خانه‌های $A[i..n]$ عناصرِ «هنوز انتخاب‌نشده» هستند. یکی از آن‌ها را یکنواخت برمی‌داریم و در خانهٔ i می‌گذاریم. این دقیقاً همان «راه کاراتر» است و $O(n)$ زمان می‌برد.

اثبات · چرا یکنواخت است؟

جایگشت مشخص π را در نظر بگیرید. برای اینکه خروجی π شود، در دور ۱ باید عنصر درست از بین n عنصر انتخاب شود (احتمال $\frac{1}{n}$)، در دور ۲ از بین $n-1$ عنصر باقی‌مانده (احتمال $\frac{1}{n-1}$)، و ... . این انتخاب‌ها مستقل‌اند، پس

$$P[\pi = \text{خروجی}] = \frac{1}{n} \cdot \frac{1}{n-1} \cdots \frac{1}{1} = \frac{1}{n!}.$$

■

خودآزمایی · پرسش اسلاید

اگر $\text{Random}(i, n)$ را با $\text{Random}(1, n)$ جایگزین کنیم، الگوریتم هنوز درست کار می‌کند؟

پاسخ: خیر. در آن صورت الگوریتم n انتخاب مستقل از n گزینه دارد، پس n^n مسیر اجرای هم‌احتمال وجود دارد. برای یکنواخت بودن، هر جایگشت باید از تعداد برابری از این مسیرها حاصل شود، یعنی $n!$ باید n^n را بشمارد. برای $n = 3$: 27 مسیر و 6 جایگشت، اما $27/6$ صحیح نیست. پس بعضی جایگشت‌ها محتمل‌ترند. (بررسی دقیق نشان می‌دهد $\langle 1,3,2\rangle$، $\langle 2,1,3\rangle$ و $\langle 2,3,1\rangle$ هر کدام $\frac{5}{27}$ و بقیه $\frac{4}{27}$ احتمال دارند.)

امید تعداد نقاط ثابت

مثال · چند عدد سر جای خودشان می‌مانند؟

σ جایگشتی تصادفی (یکنواخت) است و X تعداد iهایی است که $\sigma(i) = i$. نشان دهید $E[X] = 1$.

حل: پیشامد A_i را «$\sigma(i) = i$» و X_i را متغیر نشانگرش بگیرید. آنگاه $X = X_1 + \cdots + X_n$. چون $\sigma(i)$ با احتمال برابر هر یک از n مقدار را می‌گیرد، $E[X_i] = P[A_i] = \frac{1}{n}$. با خطی بودن امید:

$$E[X] = E[X_1] + \cdots + E[X_n] = \underbrace{\frac{1}{n} + \cdots + \frac{1}{n}}_{n} = 1.$$

جالب است که جواب به n بستگی ندارد. توجه کنید X_iها مستقل نیستند (اگر $n-1$ عدد سر جایشان باشند، آخری هم هست)، اما خطی بودن امید به استقلال نیازی ندارد.

۳.۶ میانهٔ تقریبی تصادفی

ورودی مجموعهٔ S از n عدد. برای سادگی فرض می‌کنیم همه متمایزند.

رتبه رتبهٔ عدد x در S برابر ۱ به‌اضافهٔ تعداد عناصری از S است که از x کوچک‌ترند. (کوچک‌ترین عنصر رتبهٔ ۱ دارد.)

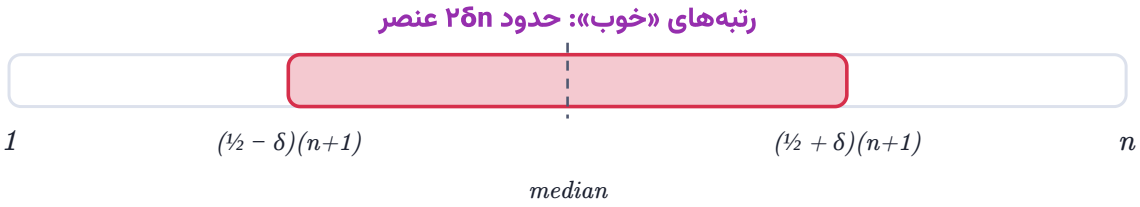
میانه عنصری با رتبهٔ $\lfloor (n+1)/2 \rfloor$.

تعریف · میانهٔ δ-تقریبی

برای ثابت دادهشدهٔ $0 \le \delta \le \frac{1}{2}$، عنصری با رتبهٔ k که

$$\left(\tfrac{1}{2} - \delta\right)(n+1) \;\le\; k \;\le\; \left(\tfrac{1}{2} + \delta\right)(n+1).$$

یعنی عنصری که رتبه‌اش حداکثر $\delta(n+1)$ از وسط فاصله دارد. با $\delta = 0$ همان میانهٔ دقیق و با $\delta = \frac{1}{2}$ هر عنصری قبول است.



شکل ۳-۳ عناصر به ترتیب رتبه. بازهٔ قرمز رتبه‌های قابل قبول است که حدود 2δ کل عناصر را در بر می‌گیرد. شکل برای $\delta = \frac{1}{4}$ کشیده شده است.

مسئله: یک میانهٔ δ-تقریبی گزارش کنید.

الگوریتم ۱: یک تیر در تاریکی

```
APPROX-MEDIAN-1(S, δ)
```

```
1  r = Random(1, n)
2  x* = S[r]
3  k = 1
4  for i = 1 to n                          // compute rank of x*
5      if S[i] < x* then
6          k = k + 1
7  if (1/2 - δ)(n+1) ⩽ k ⩽ (1/2 + δ)(n+1) then
8      return x*
9  else
10     return "error"
```

زمان اجرا: $O(n)$ (یک پیمایش برای محاسبهٔ رتبه). **احتمال موفقیت:** x^* یکنواخت انتخاب شده و رتبه‌اش هم یکنواخت در $\{1, \dots, n\}$ است. تعداد رتبه‌های خوب حدود $2\delta(n+1)$ است، پس:

$$P[\text{موفقیت}] = \frac{\left(\frac{1}{2} + \delta\right)(n+1) - \left(\frac{1}{2} - \delta\right)(n+1)}{n} \approx 2\delta.$$

برای $\delta = \frac{1}{4}$ احتمال موفقیت حدود $\frac{1}{2}$ است. برای $\delta = \frac{1}{10}$ (یعنی دنبال عنصری نزدیک‌تر به میانه هستیم) احتمال موفقیت به حدود $\frac{1}{5}$ کاهش می‌یابد. هر چه دقت بیشتری بخواهیم، شانس کمتری داریم.

الگوریتم ۲: چند بار امتحان کن

```
APPROX-MEDIAN-2(S, δ, c)
```

```
1  j = 1
2  repeat
3      result = APPROX-MEDIAN-1(S, δ)
4      j = j + 1
5  until (result ≠ error) or (j = c + 1)
6  return result
```

حداکثر c بار الگوریتم ۱ را صدا می‌زنیم. **زمان اجرا:** $O(cn)$. **احتمال موفقیت:** الگوریتم فقط وقتی شکست می‌خورد که هر c تلاش مستقل شکست بخورند، و هر تلاش با احتمال $1 - 2\delta$ شکست می‌خورد:

$$P[\text{موفقیت}] = 1 - (1 - 2\delta)^c.$$

مثال · تقویت احتمال موفقیت

- $\delta = \frac{1}{4},\ c = 10$: $1 - (\frac{1}{2})^{10} = 1 - \frac{1}{1024} \approx 99.9\%$.
- $\delta = \frac{1}{10},\ c = 10$: $1 - (0.8)^{10} \approx 1 - 0.107 = 89.3\%$.

احتمال شکست به‌صورت **نمایی** با c کم می‌شود. این ترفند، «تقویت» (amplification) نام دارد: یک الگوریتم با شانس موفقیت متوسط را چند بار تکرار کن تا شانس شکست ناچیز شود.

الگوریتم ۳: تا موفقیت ادامه بده

```
APPROX-MEDIAN-3(S, δ)
```

```
1  repeat
2      result = APPROX-MEDIAN-1(S, δ)
3  until result ≠ error
4  return result
```

احتمال موفقیت: ۱ (هرگز جواب غلط نمی‌دهد). اما زمان اجرا دیگر قطعی نیست و یک متغیر تصادفی است. امید آن:

$$E[\text{زمان}] = E[\text{تعداد فراخوانی‌ها}] \cdot O(n) = \frac{1}{2\delta} \cdot O(n) = O\left(\frac{n}{\delta}\right).$$

چرا؟ · چرا امید تعداد تلاش‌ها $1/(2\delta)$ است؟

اگر هر تلاش مستقلاً با احتمال p موفق شود، تعداد تلاش‌ها تا اولین موفقیت Y یک متغیر **هندسی** است. با احتمال p همان تلاش اول موفق است؛ وگرنه یک تلاش هدر رفته و دوباره در همان وضعیت اولیه‌ایم. پس $E[Y] = p \cdot 1 + (1-p)(1 + E[Y])$، که با حل آن $E[Y] = \frac{1}{p}$ به دست می‌آید. با $p = 2\delta$: $E[Y] = \frac{1}{2\delta}$. مثلاً اگر شانس موفقیت $\frac{1}{2}$ باشد، به‌طور میانگین ۲ تلاش لازم است، و اگر $\frac{1}{5}$ باشد، ۵ تلاش.

نکته · «امید زمان اجرا» یعنی چه؟

وقتی از «امید زمان اجرا» حرف می‌زنیم، منظورمان **امید زمان اجرا در بدترین حالت** است: برای ورودی‌های مختلف ممکن است امید زمان متفاوت باشد، و ما بدترینش را گزارش می‌کنیم. (در APPROX-MEDIAN-3 این امید برای همهٔ ورودی‌ها یکی است.)

۳.۷ زمان اجرای الگوریتم‌های قطعی و تصادفی

تعریف · الگوریتم قطعی

$T(X)$ زمان روی ورودی X است (یک عدد ثابت).

- $T_{\text{worst}}(n) = \max_{|X|=n} T(X)$
- $T_{\text{best}}(n) = \min_{|X|=n} T(X)$
- $T_{\text{avg}}(n) = \sum_{|x|=n} T(x) \cdot P[X = x]$

تعریف · الگوریتم تصادفی

$T(X)$ روی ورودی ثابت X یک **متغیر تصادفی** است (به سکه‌ها بستگی دارد).

- $T_{\text{worst-exp}}(n) = \max_{|X|=n} E[T(X)]$

اشتباه رایج · «حالت میانگین» با «امید زمان» فرق دارد

حالت میانگین میانگین‌گیری روی **ورودی‌ها** است و به فرضی دربارهٔ توزیع ورودی نیاز دارد؛ اگر ورودی‌های واقعی بدتر از فرض ما باشند، تحلیل بی‌اعتبار است. **امید زمان** میانگین‌گیری روی **سکه‌های خود الگوریتم** است و برای **هر** ورودی برقرار است. ما توزیع سکه‌ها را کنترل می‌کنیم، نه توزیع ورودی را. به همین دلیل تضمینِ الگوریتم تصادفی محکم‌تر است.

۳.۸ مونت‌کارلو در برابر لاس‌وگاس

تعریف · الگوریتم مونت‌کارلو

تضمین می‌شود در زمان چندجمله‌ای اجرا شود؛ **احتمالاً** جواب درست می‌دهد. مثال: APPROX-MEDIAN-1.

تعریف · الگوریتم لاس‌وگاس

تضمین می‌شود جواب درست بدهد؛ **احتمالاً** در زمان چندجمله‌ای اجرا می‌شود. مثال: APPROX-MEDIAN-3.

	زمان اجرا	درستی
لاس‌وگاس	احتمالاتی	**قطعی**
مونت‌کارلو	**قطعی**	احتمالاتی

نکته: APPROX-MEDIAN-2 ترکیبی است: انتخاب‌های تصادفی هم روی زمان و هم روی درستی اثر دارند. گاهی به چنین الگوریتمی هم مونت‌کارلو می‌گویند (چون زمانش حداکثر $O(cn)$ تضمین شده است).

شهود · نام‌ها از کجا آمده‌اند؟

هر دو نام شهرهای معروف قمارند. یادآوری ساده: در **لاس‌وگاس** همیشه با جواب درست بیرون می‌آیی ولی معلوم نیست چقدر طول بکشد؛ در **مونت‌کارلو** زمانت محدود است ولی ممکن است ببازی.

تکمیلی (فراتر از اسلاید) · تبدیل این دو به هم

- **لاس‌وگاس ← مونت‌کارلو:** الگوریتم لاس‌وگاس را پس از مثلاً دو برابر امید زمانش متوقف کن. طبق نامساوی مارکوف، احتمال اینکه تا آن موقع تمام نشده باشد حداکثر $\frac{1}{2}$ است.
- **مونت‌کارلو ← لاس‌وگاس:** اگر بتوانیم درستی جواب را سریع بررسی کنیم، الگوریتم مونت‌کارلو را تا رسیدن به جواب درست تکرار می‌کنیم. دقیقاً همان کاری که APPROX-MEDIAN-3 کرد (بررسی رتبه در خود الگوریتم ۱ انجام می‌شد).

۳.۹ مسئلهٔ استخدام

تعریف · صورت مسئله

برای پروژه‌ای به بهترین دستیار ممکن نیاز دارید. یک بنگاه کاریابی قول داده هر روز یک متقاضی برای مصاحبه بفرستد. چون واقعاً بهترین دستیار را می‌خواهید، این راهبرد را انتخاب می‌کنید: **هر وقت متقاضی‌ای که مصاحبه کردید از دستیار فعلی بهتر بود، دستیار فعلی را اخراج و متقاضی جدید را استخدام کنید.** هر تعویض دستیار هزینهٔ f دارد. هزینهٔ کل چقدر است؟

```
HIRE-ASSISTANT(candidates 1..n)
```

```
1  current = nil
2  for i = 1 to n
3      interview candidate i
4      if candidate i is better than current then
5          current = candidate i              // pay f
```

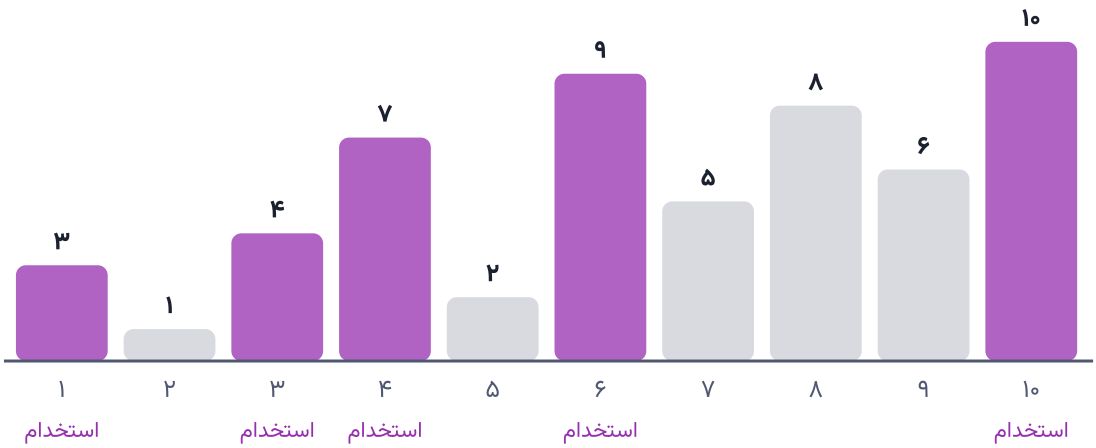
بدترین حالت: اگر متقاضی‌ها به ترتیب **صعودی** کیفیت بیایند، هر کدام از قبلی بهتر است و هر n نفر استخدام می‌شوند: هزینهٔ $f \cdot n$. اگر بنگاه کاریابی بدجنس باشد، دقیقاً همین ترتیب را می‌فرستد!

الگوریتم تصادفی

```
RANDOMIZED-HIRE-ASSISTANT(candidates 1..n)
```

```
1  compute a random permutation of the candidates
2  current = nil
3  for i = 1 to n
4      interview candidate i
5      if candidate i is better than current then
6          current = candidate i              // pay f
```

تنها تغییر: پیش از شروع، ترتیب متقاضی‌ها را با یک جایگشت تصادفی به هم می‌ریزیم. هزینه در بدترین حالت هنوز $f \cdot n$ است (ممکن است جایگشت تصادفی بدشانسانه صعودی دربیاید)، اما این اتفاق احتمال $\frac{1}{n!}$ دارد. **امید هزینه** بسیار کمتر است.



شکل ۳-۴ یک ترتیب تصادفی از ۱۰ متقاضی (ارتفاع = کیفیت). فقط «رکوردشکن‌ها» (ستون‌های رنگی) استخدام می‌شوند: ۵ استخدام به‌جای ۱۰.

اثبات · تحلیل با متغیر نشانگر

X_i را متغیر نشانگری بگیرید که ۱ است اگر و فقط اگر متقاضی iام از همهٔ متقاضی‌های $1, 2, \dots, i-1$ بهتر باشد (یعنی استخدام شود).

احتمال: در جایگشت تصادفی، i متقاضی اول یک زیرمجموعهٔ تصادفی‌اند که ترتیبشان هم تصادفی است. بهترینِ این i نفر با احتمال برابر در هر یک از i جایگاه قرار دارد، پس احتمال اینکه دقیقاً در جایگاه iام (آخرین) باشد $\frac{1}{i}$ است: $P[X_i = 1] = \frac{1}{i}$.

$$E[\text{هزینه}] = E\Big[\sum_{i} f \cdot X_i\Big] = f \sum_{i=1}^{n} E[X_i] = f \sum_{i=1}^{n} P[X_i = 1] = f \sum_{i=1}^{n} \frac{1}{i} \approx f \ln n.$$

∎

چرا؟ · چرا $\sum 1/i \approx \ln n$؟

جمع $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$ را **عدد هارمونیک** می‌نامند. با مقایسه با انتگرال (هر جملهٔ $\frac{1}{i}$ مساحت مستطیلی است که زیر یا روی منحنی $\frac{1}{x}$ قرار دارد):

$$\ln(n+1) = \int_{1}^{n+1} \frac{dx}{x} \;\le\; H_n \;\le\; 1 + \int_{1}^{n} \frac{dx}{x} = 1 + \ln n.$$

پس $H_n = \ln n + O(1)$. برای ۱۰۰۰ متقاضی، به‌جای ۱۰۰۰ استخدام فقط حدود ۷٫۵ استخدام انتظار داریم!

۳.۱۰ انتخاب تصادفی

تعریف · مسئلهٔ انتخاب

مجموعهٔ S از n عنصر متمایز و عدد صحیح i داده شده؛ عنصری با رتبهٔ i را در S پیدا کنید. ($i=1$: کمینه، $i=n$: بیشینه، $i=\lfloor (n+1)/2 \rfloor$: میانه.)

راه بدیهی: مرتب کن و خانهٔ iام را بردار: $O(n \log n)$. می‌خواهیم در $O(n)$ انجامش دهیم.

```
SELECTION(S, i) ▷ randomized (quickselect)
1  if |S| = 1 then return the only element of S
2  choose a splitter a_j in S uniformly at random
3  foreach a in S
4      if a < a_j then put a in S-
5      else if a > a_j then put a in S+
6  k = |S-|
7  if k = i-1 then return a_j                 // the splitter has rank i
8  else if k > i-1 then return SELECTION(S-, i)
9  else return SELECTION(S+, i-k-1)
```

ایده: یک عنصر تصادفی a_j، «جداکننده» یا «محور») انتخاب کن و بقیه را به کوچک‌ترها (S^-) و بزرگ‌ترها (S^+) تقسیم کن. رتبهٔ a_j دقیقاً $k+1$ است. اگر همان i بود، تمام؛ اگر i کوچک‌تر بود جواب در S^- است؛ وگرنه جواب در S^+ است، ولی چون $k+1$ عنصر کوچک‌تر را کنار گذاشته‌ایم، در S^+ دنبال رتبهٔ $i-k-1$ می‌گردیم. برخلاف کوئیک‌سورت، فقط روی **یک** طرف بازگشت می‌کنیم.

تحلیل

- **بهترین حالت:** اگر جداکننده همیشه میانه باشد، $T(n)=O(n)+T(n/2)$، پس $\Theta(n)$ مقایسه.
- **بدترین حالت:** اگر جداکننده همیشه کوچک‌ترین عنصر باشد، $T(n)=O(n)+T(n-1)$، پس $\Theta(n^2)$ مقایسه.

تصادفی‌سازی با انتخاب تصادفی جداکننده، از بدترین حالت محافظت می‌کند.

شهود · جداکنندهٔ «به‌اندازهٔ کافی خوب»

لازم نیست جداکننده دقیقاً میانه باشد. اگر همیشه عنصری را انتخاب کنیم که از ۲۵٪ عناصر بزرگ‌تر و از ۲۵٪ عناصر کوچک‌تر است، اندازهٔ مسئله در هر گام حداقل به $\frac{3}{4}$ کاهش می‌یابد: $T(n) \le cn + T(3n/4)$، که با سری هندسی $cn(1+\frac{3}{4}+\frac{9}{16}+\dots) = 4cn = O(n)$ می‌شود. و نیمی از عناصر (رتبه‌های بین $n/4$ و $3n/4$) چنین ویژگی‌ای دارند!

تحلیل دقیق: هر عنصر با احتمال $\frac{1}{n}$ جداکننده است. اگر جداکننده رتبهٔ j داشته باشد، در بدترین حالت روی طرف بزرگ‌تر، با اندازهٔ $\max(j-1, n-j)$، بازگشت می‌کنیم:

$$T_{\exp}(n) \le O(n) + \sum_{j=1}^{n} \frac{1}{n}\, T_{\exp}\big(\max(j-1, n-j)\big).$$

می‌توان نشان داد $T_{\exp}(n) = O(n)$. اما یک روش ساده‌تر هم هست:

اثبات · روش ساده‌تر

با احتمال $\frac{1}{2}$ جداکننده «خوب» است (رتبه‌اش بین $n/4$ و $3n/4$)، و در این صورت روی حداکثر $\frac{3n}{4}$ عنصر بازگشت می‌کنیم. با احتمال $\frac{1}{2}$ «بد» است و در بدترین حالت روی $n-1$ عنصر بازگشت می‌کنیم. چون T صعودی است:

$$T_{\exp}(n) \le cn + \tfrac{1}{2}\,T_{\exp}(3n/4) + \tfrac{1}{2}\,T_{\exp}(n-1) \;\le\; cn + \tfrac{1}{2}\,T_{\exp}(3n/4) + \tfrac{1}{2}\,T_{\exp}(n).$$

$\frac{1}{2}T_{\exp}(n)$ را به سمت چپ ببرید و در ۲ ضرب کنید:

$$T_{\exp}(n) \le 2cn + T_{\exp}(3n/4) \le 2cn\left(1 + \tfrac{3}{4} + \left(\tfrac{3}{4}\right)^2 + \dots\right) = 2cn \cdot 4 = 8cn = O(n).$$

(برای دقت کامل، همین را با استقرا روی فرض $T_{\exp}(n) \le 8cn$ ثابت کنید: $cn + \frac{1}{2}\cdot 8c\frac{3n}{4} + \frac{1}{2}\cdot 8cn = cn + 3cn + 4cn = 8cn$.)

■

۳.۱۱ انتخاب قطعی: میانهٔ میانه‌ها

آیا می‌توان بدون تصادف هم در $O(n)$ انتخاب کرد؟ بله! ترفند این است که جداکنندهٔ خوب را به‌جای شانس، با یک بازگشت دیگر **تضمین** کنیم.

```
SELECT(S, i) ▷ deterministic, median of medians
```

```
1  if |S| = 1 then return the only element of S
2  divide S into groups of 5:  S[5t+1..5t+5]
3  foreach group t
4      B[t] = median of that group                  // O(1) per group
5  x = SELECT(B, ceil(|B| / 2))                     // median of the medians
6  foreach a in S
7      if a < x then put a in S-
8      else if a > x then put a in S+
9  k = |S-|
10 if k = i-1 then return x
11 else if k > i-1 then return SELECT(S-, i)
12 else return SELECT(S+, i-k-1)
```

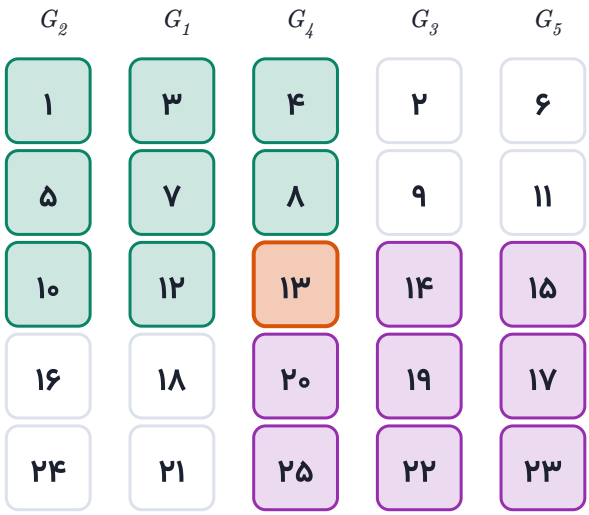
تنها تفاوت با نسخهٔ تصادفی، روش انتخاب جداکننده است: x میانهٔ میانه‌های گروه‌های پنج‌تایی است. ادعا: x همیشه جداکنندهٔ «به‌اندازهٔ کافی خوب» است.

اثبات با تصویر

G_1	G_2	G_3	G_4	G_5
۱۲	۵	۲۲	۸	۱۷
۳	۲۴	۹	۲۵	۶
۲۱	۱	۱۴	۱۳	۲۳
۷	۱۶	۲	۲۰	۱۱
۱۸	۱۰	۱۹	۴	۱۵

شکل ۳-۵۲۵ عدد در ۵ گروه پنج‌تایی ($m = \lceil n/5 \rceil = 5$ گروه).

حالا در ذهنمان (الگوریتم واقعاً این کار را نمی‌کند) هر گروه را مرتب می‌کنیم تا میانه‌اش در ردیف وسط قرار گیرد، و سپس گروه‌ها را بر اساس میانه‌شان از چپ به راست مرتب می‌کنیم. میانه‌ها $12, 10, 14, 13, 15$ هستند و میانهٔ آن‌ها $x = 13$ است: محور ما.



شکل ۳-۶ نارنجی: محور (۱۳). سبز: عناصری که **قطعاً** از محور کوچک‌ترند (بالا و چپِ محور). بنفش: عناصری که **قطعاً** بزرگ‌ترند (پایین و راستِ محور).

چرا؟ · چرا ناحیهٔ سبز حتماً از محور کوچک‌تر است؟

ستون‌های سمت چپِ ستونِ محور، میانه‌هایی کوچک‌تر از x دارند. در هر یک از این ستون‌ها، سه عنصرِ بالایی (میانه و دو عنصر بالای آن) از میانهٔ آن ستون کوچک‌تر یا مساوی‌اند، پس از x کوچک‌ترند. در ستون خود محور هم دو عنصر بالای x از آن کوچک‌ترند. این‌ها ناحیهٔ «بالا-چپ» را می‌سازند. ناحیهٔ «پایین-راست» به همین ترتیب بزرگ‌تر است.

چند عنصر قطعاً از محور کوچک‌ترند؟ حدود نیمی از m ستون ($\lceil m/2 \rceil$ ستون، شامل ستون محور) سمت چپ قرار دارند و هر کدام ۳ عنصر در ناحیهٔ سبز دارند؛ البته یکی از این ستون‌ها خود ستون محور است (که محور را نباید شمرد) و ممکن است یکی هم گروه «باقی‌مانده» با کمتر از ۵ عنصر باشد. پس دست‌کم

$$3\left(\left\lceil \frac{m}{2} \right\rceil - 2\right)$$

عنصر از محور کوچک‌ترند. در نتیجه حداکثر این تعداد عنصر از محور بزرگ‌ترند (با $m = \lceil n/5 \rceil \geq n/5$):

$$|S^+| \leq n - 1 - 3\left(\left\lceil \frac{m}{2} \right\rceil - 2\right) \leq n + 5 - \frac{3m}{2} \leq n + 5 - \frac{3n}{10} = \frac{7n}{10} + 5.$$

لم · اندازهٔ دو طرف

اگر S^- و S^+ مانند الگوریتم SELECT بالا باشند، آنگاه $|S^-| \leq \frac{7n}{10} + 5$ و $|S^+| \leq \frac{7n}{10} + 5$. (اثبات طرف دیگر دقیقاً مشابه است.)

زمان اجرا: گروه‌بندی و یافتن میانهٔ گروه‌ها $O(n)$، بازگشت برای میانهٔ میانه‌ها $T(n/5)$، تقسیم $O(n)$، و بازگشت روی یک طرف حداکثر $T(7n/10 + 5)$:

$$T(n) \le T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10} + 5\right) + O(n) \implies T(n) = O(n).$$

این دقیقاً «مثال سوم» روش حدس و استقرا در فصل ۲ است: $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$.

چرا؟ · چرا گروه‌های پنج‌تایی؟ سه‌تایی نمی‌شد؟

با گروه‌های سه‌تایی، تقریباً $\frac{n}{3}$ عنصر قطعاً کوچک‌ترند، پس بازگشت دوم روی حداکثر حدود $\frac{2n}{3}$ عنصر است و رابطه $T(n) \le T(n/3) + T(2n/3) + O(n)$ می‌شود. حالا $\frac{1}{3} + \frac{2}{3} = 1$ و جواب $O(n \log n)$ است: دیگر خطی نیست! با گروه‌های پنج‌تایی جمع کسرها $\frac{9}{10}$ است که اکیداً کمتر از ۱ است. گروه‌های هفت‌تایی و بزرگ‌تر هم کار می‌کنند، اما ثابت‌ها بدتر می‌شوند.

۳.۱۲ کوئیک‌سورت تصادفی

```
RANDOMIZED-QUICKSORT(S)
```

```
1  if |S| = 0 then return
2  choose a splitter a_i in S uniformly at random
3  foreach a in S
4      if a < a_i then put a in S-
5      else if a > a_i then put a in S+
6  RANDOMIZED-QUICKSORT(S-)
7  output a_i
8  RANDOMIZED-QUICKSORT(S+)
```

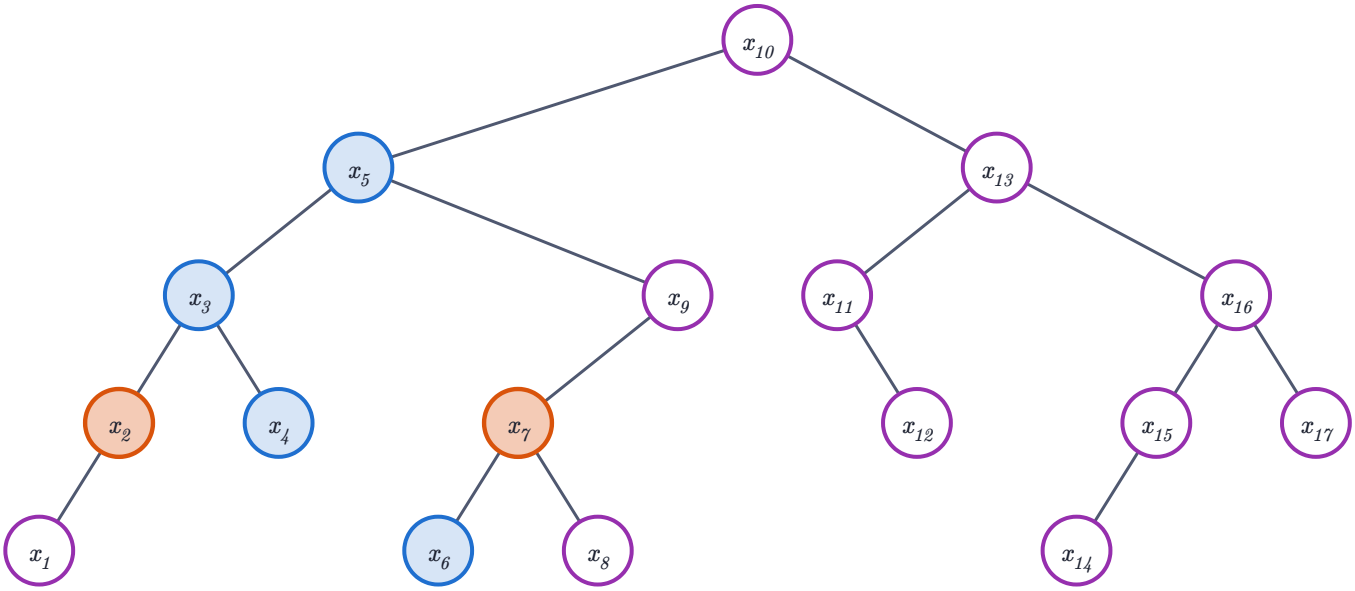
همان ایدهٔ انتخاب تصادفی، با این تفاوت که روی **هر دو** طرف بازگشت می‌کنیم.

- **بهترین حالت:** جداکننده همیشه میانه: $T(n) = 2T(n/2) + O(n)$، پس $\Theta(n \log n)$ مقایسه.
- **بدترین حالت:** جداکننده همیشه کوچک‌ترین: $T(n) = T(n-1) + O(n)$، پس $\Theta(n^2)$ مقایسه.
- **شهود:** اگر همیشه جداکننده‌ای بین صدک ۲۵ و ۷۵ انتخاب کنیم، $T(n) \le T(n/4) + T(3n/4) + O(n)$ که طبق قضیهٔ دو شاخه‌ای فصل ۲، $\Theta(n \log n)$ است.

نمادگذاری: عناصر را به ترتیب مرتب‌شده نام‌گذاری می‌کنیم: $x_1 < x_2 < \cdots < x_n$. (توجه: x_i یعنی iامین کوچک‌ترین عنصر، نه عنصر خانهٔ iام ورودی.)

نمایش جداکننده‌ها با درخت جست‌وجوی دودویی

اجرای کوئیک‌سورت را می‌توان با یک درخت نشان داد: ریشه اولین جداکننده است؛ زیردرخت چپ اجرای بازگشتی روی S^- و زیردرخت راست اجرای روی S^+. این درخت یک **درخت جست‌وجوی دودویی** است (همهٔ عناصر زیردرخت چپ کوچک‌ترند).



شکل ۳-۷ درخت جداکننده‌ها برای ۱۷ عنصر که اولین جداکنندهٔ تصادفی x_{10} بوده است. عناصر به ترتیب از چپ به راست چیده شده‌اند. x_2 و x_7 (نارنجی) هرگز با هم مقایسه نمی‌شوند: عناصر آبی (x_3 تا x_6) بین آن دو قرار دارند و یکی از آن‌ها، x_5، پیش از هر دو به‌عنوان جداکننده انتخاب شده و آن‌ها را از هم جدا کرده است.

نکته · مشاهدهٔ کلیدی

هر عنصر فقط با **اجداد و نوادگانش** در این درخت مقایسه می‌شود: وقتی a جداکننده است، با همهٔ عناصر زیردرختش مقایسه می‌شود و پس از آن دیگر در هیچ مقایسه‌ای شرکت نمی‌کند. مثلاً x_2 و x_7 فقط وقتی مقایسه می‌شوند که پایین‌ترین جد مشترکشان (lca) خود x_2 یا x_7 باشد؛ اگر جد مشترک x_3, x_4, x_5 یا x_6 باشد، مقایسه نمی‌شوند.

قضیه · احتمال مقایسه

$$P\big[x_i \text{ و } x_j \text{ مقایسه می‌شوند}\big] = \frac{2}{j-i+1} \qquad (i<j).$$

- مجموعهٔ $S_{ij} = \{x_i, x_{i+1}, \dots, x_j\}$ را در نظر بگیرید ($j-i+1$ عنصر).
- تا وقتی جداکنندهٔ انتخاب‌شده عضو S_{ij} نباشد، همهٔ اعضای S_{ij} کنار هم و در یک زیرمسئله می‌مانند (یا همه از جداکننده کوچک‌ترند یا همه بزرگ‌تر) و هیچ مقایسه‌ای بین x_i و x_j انجام نمی‌شود.
- این ادامه دارد تا وقتی که بالاخره یکی از عناصر S_{ij} به‌عنوان جداکننده انتخاب شود.
- اگر آن عنصر x_i یا x_j باشد، این دو با هم مقایسه می‌شوند. وگرنه (یک x_k با $i<k<j$)، x_i به S^- و x_j به S^+ می‌رود و **هرگز** مقایسه نمی‌شوند.
- چون جداکننده یکنواخت انتخاب می‌شود، هر عضو S_{ij} احتمال برابر دارد که اولین جداکننده از S_{ij} باشد. ۲ حالت از $j-i+1$ حالت مطلوب است، پس احتمال $\frac{2}{j-i+1}$ است.

■

قضیه · امید تعداد مقایسه‌های کوئیک‌سورت تصادفی

امید تعداد مقایسه‌ها $O(n \log n)$ است.

$X_{ij} = 1$ اگر x_i و x_j مقایسه شوند و ۰ در غیر این صورت. هر جفت حداکثر یک بار مقایسه می‌شود (پس از مقایسه، جداکننده کنار می‌رود)، پس $X = \sum_{i<j} X_{ij}$ تعداد کل مقایسه‌هاست و

$$E[X] = \sum_{1 \le i < j \le n} E[X_{ij}] = \sum_{1 \le i < j \le n} \frac{2}{j - i + 1} = 2 \sum_{i=1}^{n} \sum_{d=2}^{n-i+1} \frac{1}{d} \;\le\; 2n \sum_{d=2}^{n} \frac{1}{d} \;\le\; 2n \int_{1}^{n} \frac{dx}{x} = 2n \ln n.$$

(در جمع وسط، $d = j - i + 1$ را جایگزین کردیم؛ برای هر i، d از ۲ تا $n - i + 1$ تغییر می‌کند و ما آن را با n کران زدیم.) پس $E[X] \le 2n \ln n \approx 1.39\, n \log_2 n = O(n \log n)$.

■

نکته · تمرکز حول امید

امید کم به‌تنهایی کافی نیست؛ باید بدانیم احتمال اینکه زمان خیلی بیشتر از امید شود کم است. با محاسبهٔ واریانس و **نامساوی چبیشف** ($P[|X - \mu| \ge k\sigma] \le \frac{1}{k^2}$) می‌توان نشان داد مثلاً برای n = یک میلیون، احتمال اینکه کوئیک‌سورت تصادفی کمتر از $4n \ln n$ مقایسه انجام دهد دست‌کم 99.94% است.

رویکرد دیگر: محور با میانهٔ تقریبی

به‌جای اینکه محور را یکنواخت تصادفی برداریم، می‌توانیم اصرار کنیم که محور «خوب» باشد. راه ساده: با APPROX-MEDIAN-3 یک میانهٔ $\frac{1}{4}$-تقریبی پیدا کنیم (امید زمان $O(n/\delta) = O(n)$). چنین محوری هر دو طرف را حداکثر $\frac{3}{4}n$ می‌کند. امید زمان:

$$E[\text{زمان APPROX-MEDIAN-3}] + E[\text{فراخوانی‌های بازگشتی}] = O(n) + E[\text{فراخوانی‌های بازگشتی}]$$

$$T_{\exp}(n) \le O(n) + T_{\exp}(3n/4) + T_{\exp}(n/4) \quad \Longrightarrow \quad T_{\exp}(n) = O(n \log n).$$

(دوباره قضیهٔ دو شاخه‌ای فصل ۲ با $\frac{3}{4} + \frac{1}{4} = 1$.)

جمع‌بندی · فصل ۳ در یک نگاه

- متغیر تصادفی یک تابع است؛ امید ریاضی میانگین وزن‌دار آن.
- خطی بودن امید بدون نیاز به استقلال؛ با متغیر نشانگر: $E[I_A] = P[A]$.
- مونت‌کارلو: زمان قطعی، درستی احتمالاتی. لاس‌وگاس: درستی قطعی، زمان احتمالاتی. تکرار، احتمال شکست را نمایی کم می‌کند.
- استخدام تصادفی: امید هزینه $f H_n \approx f \ln n$ به‌جای fn.
- انتخاب: تصادفی با امید $O(n)$؛ قطعی با میانهٔ میانه‌ها $O(n)$ در بدترین حالت.
- کوئیک‌سورت تصادفی: $P[\text{مقایسه } x_i, x_j] = \frac{2}{j-i+1}$، و امید مقایسه‌ها $\leq 2n \ln n$.

۳.۱۳ خودآزمایی فصل ۳

خودآزمایی

1. سکه‌ای سالم را ۱۰ بار پرتاب می‌کنیم. امید تعداد دفعاتی که دو پرتاب متوالی هر دو شیر باشند چقدر است؟

پاسخ: برای $i = 1..9$، X_i نشانگر «پرتاب i و $i+1$ هر دو شیر» است: $E[X_i] = \frac{1}{4}$. با خطی بودن امید: $9 \cdot \frac{1}{4} = 2.25$. ($X_i$ها مستقل نیستند، ولی مهم نیست.)

2. در کوئیک‌سورت تصادفی، احتمال اینکه کوچک‌ترین و بزرگ‌ترین عنصر با هم مقایسه شوند چقدر است؟

پاسخ: $i = 1, j = n$: احتمال $\frac{2}{n}$. فقط وقتی مقایسه می‌شوند که یکی از این دو، اولین جداکنندهٔ کل آرایه باشد.

3. در APPROX-MEDIAN-2 با $\delta = \frac{1}{4}$، حداقل چند تکرار لازم است تا احتمال شکست کمتر از یک در میلیون شود؟

پاسخ: $(\frac{1}{2})^c < 10^{-6}$ یعنی $c > 6 \log_2 10 \approx 19.9$؛ پس $c = 20$.

4. چرا الگوریتم انتخاب قطعی با گروه‌های سه‌تایی خطی نیست؟

پاسخ: رابطه $T(n) \leq T(n/3) + T(2n/3) + O(n)$ می‌شود که جمع کسرهایش ۱ است و جواب $\Theta(n \log n)$ دارد.

5. آیا الگوریتم کوئیک‌سورت تصادفی مونت‌کارلو است یا لاس‌وگاس؟

پاسخ: لاس‌وگاس: خروجی همیشه درست مرتب است و فقط زمان اجرا تصادفی است.

فصل ۴

تحلیل سرشکنی

گاهی یک عمل گران است، ولی گران بودنش آن‌قدر نادر است که اگر هزینه را میان همهٔ عمل‌ها «سرشکن» کنیم، سهم هر عمل ناچیز می‌شود. این فصل سه روش برای اثبات دقیق همین شهود می‌آموزد.

پشته با multipop | شمارندهٔ دودویی | روش انبوهه | روش حساب‌داری | روش پتانسیل

۴.۱ سه نگاه به هزینه

تا این‌جا دو نوع تحلیل دیدیم:

- **تحلیل بدترین حالت:** بیشترین هزینهٔ **یک** عمل روی بدترین ورودی.
- **تحلیل حالت میانگین:** امید هزینه، با فرضی دربارهٔ توزیع ورودی.

تحلیل سرشکنی (amortized analysis) نوع سومی است که برای **داده‌ساختارها** طراحی شده است:

تعریف · تحلیل سرشکنی

محیط: یک داده‌ساختار و **دنباله‌ای از عملیات** روی آن.

هزینهٔ سرشکن‌شدهٔ هر عمل: متوسط هزینهٔ آن عمل در **بدترین** دنباله. به زبان دقیق: اگر $T(n)$ بیشترین هزینهٔ کل روی همهٔ دنباله‌های n عملی باشد، هزینهٔ سرشکن‌شده $T(n)/n$ است.

شهود · قلک

فرض کنید هر روز ناهار ۱ واحد خرج دارد، اما هر چند وقت یک بار باید یک کفش تازه بخرید که گران است. اگر هر روز ۲ واحد کنار بگذارید (۱ برای ناهار، ۱ در قلک)، روز خرید کفش قلک پر است و هیچ‌وقت کم نمی‌آورید. «هزینهٔ سرشکن‌شدهٔ هر روز» ۲ است، با اینکه بعضی روزها خرج واقعی خیلی بیشتر است. تحلیل سرشکنی دقیقاً همین حساب‌وکتاب است.

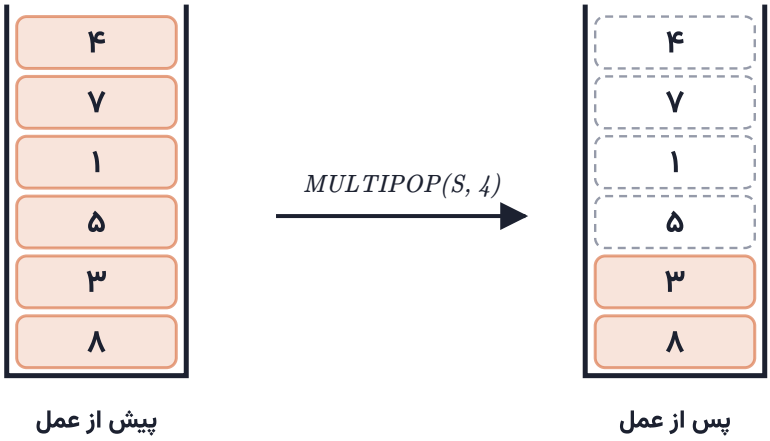
اشتباه رایج · سرشکن ≠ میانگین

در تحلیل سرشکنی **هیچ احتمالی** در کار نیست. ما میانگین را روی عملیاتِ **یک** دنباله می‌گیریم و آن دنباله را بدترین دنبالهٔ ممکن فرض می‌کنیم. پس نتیجه یک **تضمین قطعی** است: «هر دنبالهٔ n عملی حداکثر فلان‌قدر هزینه دارد». اما یک عمل **تکی** ممکن است هنوز گران باشد؛ برای سیستم‌های بی‌درنگ که هر پاسخ باید سریع باشد، این تفاوت مهم است.

۴.۲ مثال ۱: پشته با عمل multipop

پشته علاوه بر PUSH(S, x) و POP(S)، عمل سومی هم دارد: MULTIPOP(S, k) که k عنصر بالای پشته را برمی‌دارد (یا اگر کمتر از k عنصر بود، همه را).

```
MULTIPOP(S, k)
1  while not STACK-EMPTY(S) and k ≠ 0
2      POP(S)
3      k = k - 1
```



شکل ۴-۱ MULTIPOP$(S, 4)$ چهار عنصر بالایی را برمی‌دارد و ۴ واحد هزینه دارد.

هزینه‌ها در بدترین حالت

PUSH و POP $\Theta(1)$

MULTIPOP $\Theta(\min\{k, \text{SIZE}(S)\})$؛ در بدترین حالت $\Theta(n)$ چون پشته ممکن است n عنصر داشته باشد.

پس اگر دنباله‌ای از n عمل داشته باشیم و هر کدام در بدترین حالت $O(n)$ باشد، جمع کل در بدترین حالت $O(n^2)$ به دست می‌آید. این کران درست است، اما **شُل** است: نمی‌شود n عمل MULTIPOP داشت که هر کدام n عنصر بردارند، چون عنصری که برداشته شد دیگر نیست!

جمع‌بندی · هدف

نشان می‌دهیم هزینهٔ سرشکن‌شدهٔ هر عمل $O(1)$ است؛ یعنی هر دنبالهٔ n عملی روی پشتهٔ خالی، $O(n)$ هزینه دارد.

۴.۳ مثال ۲: شمارندهٔ دودویی

یک شمارندهٔ دودویی k بیتی با آرایهٔ $A[0..k-1]$ داریم ($A[0]$ کم‌ارزش‌ترین بیت است). مقدار شمارنده $\sum_{i=0}^{k-1} A[i] \cdot 2^i$ است و از صفر شروع می‌شود. تنها عمل، **افزایش** یک واحد است:

```
INCREMENT(A)
```

```
1  i = 0
2  while i < length[A] and A[i] = 1
3      A[i] = 0                    // carry: flip 1 → 0
4      i = i + 1
5  if i < length[A]
6      then A[i] = 1               // flip 0 → 1
```

همان جمع دبستانی با ۱ است: از راست، یک‌ها را صفر می‌کنیم (رقم نقلی جلو می‌رود) تا به اولین صفر برسیم و آن را یک کنیم. **هزینه** = تعداد بیت‌هایی که عوض می‌شوند.

هزینهٔ شمارنده در بدترین حالت

هر عمل افزایش در بدترین حالت $O(k)$ است (مثلاً از $0111\ldots1$ به $1000\ldots0$ همهٔ k بیت عوض می‌شوند). پس n عمل افزایش $O(nk)$. اما این‌جا هم کران شل است:

مقدار	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	هزینهٔ این افزایش	هزینهٔ کل	۲n
۰	0	0	0	0	0	0	0	0	—	۰	۰
۱	0	0	0	0	0	0	0	1	۱	۱	۲
۲	0	0	0	0	0	0	1	0	۲	۳	۴
۳	0	0	0	0	0	0	1	1	۱	۴	۶
۴	0	0	0	0	0	1	0	0	۳	۷	۸
۵	0	0	0	0	0	1	0	1	۱	۸	۱۰
۶	0	0	0	0	0	1	1	0	۲	۱۰	۱۲
۷	0	0	0	0	0	1	1	1	۱	۱۱	۱۴
۸	0	0	0	0	1	0	0	0	۴	۱۵	۱۶
۹	0	0	0	0	1	0	0	1	۱	۱۶	۱۸
۱۰	0	0	0	0	1	0	1	0	۲	۱۸	۲۰
۱۱	0	0	0	0	1	0	1	1	۱	۱۹	۲۲
۱۲	0	0	0	0	1	1	0	0	۳	۲۲	۲۴
۱۳	0	0	0	0	1	1	0	1	۱	۲۳	۲۶
۱۴	0	0	0	0	1	1	1	0	۲	۲۵	۲۸
۱۵	0	0	0	0	1	1	1	1	۱	۲۶	۳۰
۱۶	0	0	0	1	0	0	0	0	۵	۳۱	۳۲

جدول بالا ۱۶ افزایش اول یک شمارندهٔ ۸ بیتی را نشان می‌دهد. بیت‌های زرد در همان افزایش عوض شده‌اند. افزایش‌های گران (مثل ۷ ← ۸ با ۴ تغییر، یا ۱۵ ← ۱۶ با ۵ تغییر) نادرند و **هزینهٔ کل همیشه کمتر از $2n$ است** (ستون آخر را با ستون «هزینهٔ کل» مقایسه کنید).

جمع‌بندی · هدف

نشان می‌دهیم هزینهٔ هر عمل افزایش به‌صورت سرشکن‌شده $O(1)$ است.

۴.۴ روش‌های تحلیل سرشکنی

روش	ایده
انبوهه (aggregate)	هزینهٔ کل n عمل را مستقیم حساب کن و بر n تقسیم کن. همهٔ عمل‌ها هزینهٔ سرشکن یکسان می‌گیرند.
حساب‌داری (accounting)	برای هر عمل مبلغی «پرداخت» کن؛ مازاد را به‌صورت اعتبار روی اشیای داده‌ساختار ذخیره کن تا عمل‌های گران بعدی را بپردازد.
تابع پتانسیل (potential)	حالت کلی‌تر روش قبل: به‌جای اعتبار روی اشیا، یک «انرژی» برای کل داده‌ساختار تعریف کن.

۴.۵ روش انبوهه

ساده‌ترین روش: یک کران بالا $T(n)$ برای هزینهٔ کل هر دنبالهٔ n عملی پیدا کن. آنگاه هزینهٔ سرشکن هر عمل $T(n)/n$ است.

تحلیل پشته با روش انبوهه

اگر n عمل (PUSH، POP یا MULTIPOP) روی پشتهٔ ابتدا خالی داشته باشیم:

- حداکثر تعداد عناصر پشته n است (حداکثر n PUSH).
- هر عنصر داخل پشته دقیقاً **یک بار** PUSH و حداکثر **یک بار** POP می‌شود.
- یک عنصر یا مستقیماً POP می‌شود یا در یک MULTIPOP.
- هر عمل PUSH و POP هزینهٔ $O(1)$ دارد.

پس تعداد کل POPها (شامل POPهای داخل MULTIPOP) حداکثر برابر تعداد PUSHهاست که حداکثر n است. **جمع هزینه‌ها حداکثر** $2n$ است و هزینهٔ سرشکن‌شدهٔ هر عمل $\frac{2n}{n} = O(1)$.

■

چرا؟ · ترفند اصلی چه بود؟

به‌جای اینکه بپرسیم «هر **عمل** چقدر خرج دارد؟» پرسیدیم «هر **عنصر** در طول عمرش چقدر خرج ایجاد می‌کند؟». هر عنصر حداکثر ۲ واحد (یک ورود، یک خروج). تغییر زاویهٔ شمارش، قلب روش انبوهه است.

تحلیل شمارنده با روش انبوهه

دوباره زاویهٔ شمارش را عوض می‌کنیم: به‌جای شمردن تغییرات در هر افزایش، تغییرات **هر بیت** را در کل دنباله می‌شماریم.

- هر افزایش حداکثر **یک** بیت را ۱ می‌کند و تعدادی را از ۱ به ۰ تغییر می‌دهد.
- بیت ۰ با **هر** افزایش عوض می‌شود (n بار).
- بیت ۱ **یکی در میان** عوض می‌شود ($\lfloor n/2 \rfloor$ بار).
- بیت ۲ هر **۴** بار یک دفعه عوض می‌شود ($\lfloor n/4 \rfloor$ بار).
- ...
- بیت i پس از هر 2^i افزایش عوض می‌شود (در مجموع $\lfloor n/2^i \rfloor$ بار).

پس در مجموع:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n.$$

یعنی هزینهٔ سرشکن‌شدهٔ هر عمل $O(1)$ است (حداکثر ۲).

■

چرا؟ · چرا بیت i دقیقاً هر 2^i افزایش یک بار عوض می‌شود؟

بیت i وقتی عوض می‌شود که همهٔ بیت‌های 0 تا $i-1$ پیش از افزایش ۱ باشند (رقم نقلی تا بیت i برسد). این یعنی مقدار شمارنده به پیمانهٔ 2^i برابر $2^i - 1$ باشد، که در هر 2^i مقدار متوالی دقیقاً یک بار رخ می‌دهد. در جدول ببینید: ستون $A[0]$ هر بار زرد است، $A[1]$ یکی در میان، $A[2]$ هر چهار بار.

۴.۶ روش حساب‌داری

در این روش به هر عمل یک **هزینهٔ سرشکن** $\hat{c}$ (مبلغی که «می‌پردازیم») نسبت می‌دهیم که ممکن است با هزینهٔ واقعی c فرق داشته باشد. وقتی $\hat{c} > c$، مازاد را به‌صورت **اعتبار** روی اشیای مشخصی از داده‌ساختار ذخیره می‌کنیم؛ وقتی $\hat{c} < c$، کسری را از اعتبار ذخیره‌شده برمی‌داریم.

نکته · تنها شرط

اعتبار کل **هرگز نباید منفی شود** (نمی‌توان از مخزن خالی برداشت کرد). معادلش: برای هر دنباله و هر پیشوند آن، $\sum \hat{c}_i \geq \sum c_i$. اگر این برقرار باشد، جمع هزینه‌های سرشکن یک کران بالا برای جمع هزینه‌های واقعی است.

تحلیل پشته به روش حساب‌داری

- برای هر PUSH، **۲ ریال** خرج می‌کنیم.
- **۱ ریال** صرف خود عمل PUSH می‌شود.
- **۱ ریال** را روی عنصرِ داخل پشته می‌گذاریم.
- POPها همه **مجانی** خواهند بود: هزینهٔ برداشتن هر عنصر از ریالی پرداخت می‌شود که رویش گذاشته‌ایم. MULTIPOP هم چند POP است و مجانی است.

اعتبار کل همیشه برابر تعداد عناصر پشته است، پس هرگز منفی نمی‌شود. کلاً حداکثر $2n$ ریال هزینه شده است، پس هزینهٔ سرشکن‌شده $O(1)$ است.

■

تحلیل شمارنده به روش حساب‌داری

- برای هر بار تبدیل یک بیت از ۰ به ۱، **۲ ریال** هزینه می‌کنیم.
- **۱ ریال** صرف خود تبدیل ۰ به ۱ می‌شود.
- **۱ ریال** را روی همان بیت ۱ قرار می‌دهیم.
- هزینهٔ تبدیل ۱ به ۰ **مجانی** خواهد بود: ریالِ روی آن بیت را خرج می‌کنیم.

ناوردا: روی هر بیت ۱ دقیقاً یک ریال هست، پس اعتبار کل = تعداد یک‌ها ≥ 0. هر افزایش حداکثر **یک** بیت را از ۰ به ۱ می‌کند (حلقهٔ `while` فقط یک‌ها را صفر می‌کند که مجانی است)، پس هزینهٔ سرشکن هر افزایش حداکثر ۲ ریال است. n افزایش حداکثر $2n$ هزینه دارد.

■

شهود · اعتبار روی «اشیا»

در روش حساب‌داری اعتبار به چیزهای مشخصی چسبیده است: به عناصر پشته، یا به بیت‌های ۱. این کمک می‌کند به سؤال «کار گران بعدی را چه کسی می‌پردازد؟» دقیق جواب دهیم: همان شیئی که کار گران رویش انجام می‌شود.

۴.۷ روش تابع پتانسیل

به‌جای اینکه اعتبار را روی اشیای جداگانه نگه داریم، آن را به‌صورت یک عدد برای **کل** داده‌ساختار در نظر می‌گیریم؛ مثل انرژی پتانسیل در فیزیک.

D_0 داده‌ساختار اولیه

D_i داده‌ساختار پس از عمل iام

c_i هزینهٔ واقعی عمل iام

$\Phi(D_i)$ تابع پتانسیل که D_i را به یک عدد حقیقی نگاشت می‌کند

$$D_0 \xrightarrow{\text{عمل ۱}} D_1 \xrightarrow{\text{عمل ۲}} D_2 \cdots \xrightarrow{\text{عمل } n} D_n$$

تعریف · هزینهٔ سرشکن در روش پتانسیل

هزینهٔ سرشکن‌شدهٔ عمل iام:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

یعنی هزینهٔ واقعی به‌اضافهٔ **تغییر پتانسیل**. عملی که پتانسیل را بالا می‌برد گران‌تر از هزینهٔ واقعی‌اش «حساب» می‌شود (ذخیره می‌کند)، و عملی که پتانسیل را پایین می‌آورد ارزان‌تر (از ذخیره خرج می‌کند).

با جمع زدن روی همهٔ عمل‌ها، جمله‌های پتانسیل به‌صورت **تلسکوپی** حذف می‌شوند:

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} \big(c_i + \Phi(D_i) - \Phi(D_{i-1})\big) = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0).$$

(در جمع، $\Phi(D_1)$ یک بار با علامت مثبت و یک بار با علامت منفی می‌آید، و همین‌طور بقیه؛ فقط اولی و آخری می‌مانند.) اگر $\Phi(D_0) = 0$ و $\Phi(D_i) \geq 0$ برای همهٔ i:

$$\sum_{i=1}^{n} \hat{c}_i \;\geq\; \sum_{i=1}^{n} c_i.$$

یعنی $\sum \hat{c}_i$ همان **کران بالایی** برای $\sum c_i$ است که می‌خواهیم به دست آوریم.

نکته · چرا $\Phi(D_i) \geq 0$ برای همهٔ i، نه فقط $i = n$؟

چون نمی‌دانیم دنباله کجا تمام می‌شود. برای اینکه کران برای هر n (هر پیشوند دنباله) برقرار باشد، باید در هر لحظه پتانسیل از مقدار اولیه کمتر نشود.

تناظر با روش حساب‌داری

$\Phi(D_i)$ مقدار پول موجود در مخزن پس از عمل i.

$\hat{c}_i$ مقدار پولی که برای عمل i پرداخت می‌کنیم.

c_i مقدار هزینهٔ صرف‌شده برای عمل i.

- اگر $c_i < \hat{c}_i$، مابه‌التفاوت به مخزن اضافه می‌شود: $\Phi(D_i) = \Phi(D_{i-1}) + \hat{c}_i - c_i$.
- اگر $c_i > \hat{c}_i$، برای انجام عمل i لازم است به اندازهٔ $c_i - \hat{c}_i$ از مخزن برداریم تا بتوانیم این عمل را انجام دهیم. پس پول مخزن $\Phi(D_i) = \Phi(D_{i-1}) - (c_i - \hat{c}_i)$ می‌شود.

هر دو حالت همان رابطهٔ $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ است. شرط «مخزن منفی نشود» همان شرط $\Phi(D_i) \geq 0$ است.

تحلیل پشته با روش پتانسیل

تعریف: $\Phi(D_i)$ = تعداد عناصر موجود در پشته. در ابتدا $\Phi(D_0) = 0$ و همیشه $\Phi \geq 0$.

عمل PUSH: یک عنصر اضافه می‌شود: $\Phi(D_i) - \Phi(D_{i-1}) = 1$. هزینهٔ واقعی $c_i = 1$. پس $\hat{c}_i = 1 + 1 = 2$.

عمل POP: یک عنصر کم می‌شود: $\Phi(D_i) - \Phi(D_{i-1}) = -1$. هزینهٔ واقعی $c_i = 1$. پس $\hat{c}_i = 1 - 1 = 0$.

عمل MULTIPOP: اگر $k' = \min(k, s)$ عنصر برداشته شود، $c_i = k'$ و $\Delta\Phi = -k'$، پس $\hat{c}_i = 0$. (MULTIPOP هم تعدادی POP است، پس هزینهٔ سرشکنش صفر است.)

$$\sum_{i=1}^{n} c_i \;\leq\; \sum_{i=1}^{n} \hat{c}_i \;\leq\; 2n.$$

پس هزینهٔ سرشکن‌شدهٔ هر عمل $O(1)$ است.

■

تحلیل شمارنده به روش پتانسیل

تعریف تابع پتانسیل: $\Phi(D_i) = b_i$ = تعداد یک‌های A پس از عمل i. داریم $\Phi(D_0) = 0$ و $\Phi(D_i) \geq 0$.

عمل «افزایش»: فرض کنید افزایش iام، t_i بیت را از ۱ به صفر و حداکثر یک بیت صفر را به ۱ تبدیل کند. پس

$$\left.\begin{aligned} \hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ \Phi(D_i) - \Phi(D_{i-1}) &\leq -t_i + 1 \\ c_i &\leq t_i + 1 \end{aligned}\right\} \implies \hat{c}_i \leq (t_i + 1) + (1 - t_i) = 2.$$

پس هزینهٔ سرشکن‌شدهٔ هر عمل افزایش $O(1)$ است و n افزایش حداکثر $2n$ هزینه دارد.

■

نکته · چرا «حداکثر» یک بیت ۰ ← ۱؟

اگر همهٔ k بیت ۱ باشند (سرریز)، حلقه همه را صفر می‌کند و شرط $i < \text{length}[A]$ نادرست است، پس هیچ بیتی ۱ نمی‌شود. در این حالت $c_i = t_i = k$ و $\Delta\Phi = -k$، پس $\hat{c}_i = 0 \leq 2$. نامساوی‌ها هر دو حالت را پوشش می‌دهند.

تکمیلی (فراتر از اسلاید) · شمارنده‌ای که از صفر شروع نمی‌شود

اگر شمارنده با b_0 یک شروع شود، $\Phi(D_0) = b_0 \neq 0$. از تساوی تلسکوپی: $\sum c_i = \sum \hat{c}_i - \Phi(D_n) + \Phi(D_0) \leq 2n -$ $b_n + b_0 \leq 2n + k$. پس اگر $n = \Omega(k)$ (حداقل به تعداد بیت‌ها افزایش انجام دهیم)، هزینهٔ کل باز هم $O(n)$ است. این نشان می‌دهد روش پتانسیل حتی وقتی $\Phi(D_0) \neq 0$ هم مفید است.

چرا؟ · تابع پتانسیل را از کجا بیاوریم؟

قاعدهٔ سرانگشتی: پتانسیل باید «میزان بی‌نظمیِ ذخیره‌شده» را اندازه بگیرد، یعنی چیزی که بعداً عمل‌های گران را ایجاد می‌کند. در پشته، عناصر موجود همان چیزی‌اند که بعداً MULTIPOP را گران می‌کنند؛ در شمارنده، یک‌ها همان چیزی‌اند که بعداً رقم نقلی را طولانی می‌کنند. عمل گران باید پتانسیل را به همان اندازه‌ای که هزینه دارد پایین بیاورد. اغلب، اعتبارهای روش حساب‌داری را جمع بزنید؛ پتانسیل به دست می‌آید.

۴.۸ مثال تکمیلی: آرایهٔ پویا

تکمیلی (فراتر از اسلاید) · چرا این مثال؟

این مثال در اسلایدها نیامده، اما مهم‌ترین کاربرد عملی تحلیل سرشکنی است (`vector` در C++، `ArrayList` در جاوا و `list` در پایتون) و سؤال محبوب امتحان‌هاست. هر سه روش را روی آن تمرین می‌کنیم.

آرایه‌ای با ظرفیت size داریم که num خانه‌اش پر است. **درج:** اگر جا هست، عنصر را در خانهٔ بعدی بگذار (هزینهٔ ۱). اگر آرایه پر است، آرایه‌ای با **دو برابر** ظرفیت بساز، همهٔ num عنصر را کپی کن و بعد درج کن (هزینهٔ $\text{num} + 1$). از آرایهٔ خالی با ظرفیت ۱ شروع می‌کنیم.

بدترین حالت یک درج $O(n)$ است، پس کران سادهٔ n درج $O(n^2)$ است. اما:

اثبات · انبوهه

درج iام هزینهٔ i دارد اگر $i - 1$ توانی از ۲ باشد (آرایه پر است)، و ۱ در غیر این صورت. پس

$$\sum_{i=1}^{n} c_i \leq n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j < n + 2n = 3n.$$

هزینهٔ سرشکن هر درج حداکثر ۳ است.

∎

اثبات · حساب‌داری

برای هر درج **۳ ریال** بپرداز: ۱ ریال برای درج خودش، ۱ ریال روی خودش برای وقتی که در گسترش بعدی کپی می‌شود، و ۱ ریال روی یکی از عناصر **قدیمی** (نیمهٔ اول آرایه) که ریالش را در گسترش قبلی خرج کرده است. وقتی آرایه با ظرفیت s پر می‌شود، $s/2$ عنصر جدید هر کدام ۲ ریال ذخیره کرده‌اند، یعنی s ریال: دقیقاً به اندازهٔ کپی همهٔ s عنصر.

∎

اثبات · پتانسیل

$\Phi = 2 \cdot \text{num} - \text{size}$ (پس از اولین درج، آرایه همیشه دست‌کم نیمه‌پر است، پس $\Phi \geq 0$).

- **درج بدون گسترش:** $c = 1$، num یکی زیاد می‌شود: $\Delta\Phi = 2$. پس $\hat{c} = 3$.
- **درج با گسترش** ($\text{num} = \text{size} = s$): $c = s + 1$. پیش از عمل $\Phi = 2s - s = s$، پس از آن $\Phi = 2(s+1) - 2s = 2$. پس $\hat{c} = (s+1) + (2 - s) = 3$.

∎

چرا؟ · چرا دو برابر؟ چرا نه «ظرفیت + ۱۰۰»؟

اگر هر بار فقط c خانه اضافه کنیم، هر c درج یک کپی کامل داریم و هزینهٔ کپی‌ها $c + 2c + 3c + \cdots + n \approx \frac{n^2}{2c}$ است: سرشکن $\Theta(n/c) = \Theta(n)$. کلید کار **رشد ضربی** است: با ضریب رشد $r > 1$، هزینهٔ کپی‌ها یک سری هندسی است و سرشکن $O(1)$ می‌ماند (مثلاً با $r = 1.5$، حداکثر حدود $3n$ کپی).

۴.۹ مقایسهٔ سه روش

	انبوهه	حساب‌داری	پتانسیل
چه چیزی را می‌شماریم؟	هزینهٔ کل دنباله	اعتبار روی تک‌تک اشیا	یک عدد برای کل ساختار
هزینهٔ سرشکن	برای همهٔ عمل‌ها یکسان	برای هر نوع عمل جدا	برای هر نوع عمل جدا
شرط درستی	–	اعتبار هرگز منفی نشود	$\Phi(D_i) \geq \Phi(D_0)$
پشته	$\leq 2n$	PUSH: ۲، POP: ۰	Φ = اندازهٔ پشته
شمارنده	$\sum \lfloor n/2^i \rfloor < 2n$	۰ ← ۱، ۲ : ۱ ← ۰ : ۰	Φ = تعداد یک‌ها
مناسب برای	ساختارهای ساده	شهود «چه کسی می‌پردازد»	ساختارهای پیچیده (هرم فیبوناچی، درخت اسپلی)

جمع‌بندی · فصل ۴ در یک نگاه

- هزینهٔ سرشکن = هزینهٔ کل بدترین دنبالهٔ n عملی تقسیم بر n. تضمین قطعی است، بدون احتمال.
- جمع زدن «بدترین هزینهٔ هر عمل» اغلب کران شل می‌دهد ($O(n^2)$ و $O(nk)$ به‌جای $O(n)$).
- انبوهه: زاویهٔ شمارش را عوض کن (هر عنصر حداکثر یک بار POP می‌شود؛ بیت i هر 2^i بار یک بار عوض می‌شود).
- حساب‌داری: عمل ارزان بیشتر بپردازد و اعتبار روی اشیا بماند تا عمل گران را بپردازد.
- پتانسیل: $\hat{c}_i = c_i + \Delta\Phi$؛ با $\Phi(D_0) = 0$ و $\Phi \geq 0$، $\sum \hat{c}_i$ کران بالای $\sum c_i$ است.

۴.۱۰ خودآزمایی فصل ۴

خودآزمایی

1. اگر پشته عمل MULTIPUSH(S, k) هم داشته باشد (k عنصر را درج کند)، آیا هزینهٔ سرشکن همهٔ عمل‌ها هنوز $O(1)$ است؟

پاسخ: خیر. خود هر MULTIPUSH k هزینهٔ واقعی دارد؛ دنباله‌ای از n عمل MULTIPUSH هزینهٔ nk دارد، پس سرشکن $\Theta(k)$ است.

2. اگر شمارنده عمل DECREMENT هم داشته باشد، آیا هزینهٔ سرشکن $O(1)$ می‌ماند؟

پاسخ: خیر. با شروع از $0111\ldots1$، تناوب INCREMENT و DECREMENT هر بار همهٔ k بیت را عوض می‌کند؛ n عمل هزینهٔ nk دارند.

3. با پتانسیل $\Phi = 2\times$ (تعداد یک‌ها) برای شمارنده، هزینهٔ سرشکن افزایش چقدر می‌شود؟

پاسخ: $\hat{c} \leq (t+1) + 2(1-t) = 3 - t \leq 3$. باز هم $O(1)$؛ تابع پتانسیل یکتا نیست.

4. در دنباله‌ای از n عمل، عمل iام اگر i توانی از ۲ باشد هزینهٔ i و وگرنه هزینهٔ ۱ دارد. هزینهٔ سرشکن هر عمل چقدر است؟

پاسخ: انبوهه: $\sum c_i \leq n + (1 + 2 + 4 + \cdots + 2^{\lfloor \log n \rfloor}) < n + 2n = 3n$؛ پس سرشکن $O(1)$.

5. آیا هزینهٔ سرشکن $O(1)$ تضمین می‌کند که هر عمل تکی $O(1)$ باشد؟

پاسخ: خیر. فقط جمع n عمل $O(n)$ است؛ یک عمل تکی (مثل گسترش آرایهٔ پویا) می‌تواند $\Theta(n)$ باشد.

پیوست

برگهٔ خلاصه، واژه‌نامه و منابع

همهٔ فرمول‌ها و زمان‌های اجرایی که در طول جزوه به آن‌ها نیاز داشتیم، در یک جا؛ برای مرور شب امتحان.

پ.۱ فرمول‌های پرکاربرد

جمع‌ها

نام	فرمول	کجا به کار رفت
جمع حسابی	$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$	مرتب‌سازی درجی، حلقه‌های تودرتو
جمع مربعات	$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$	تحلیل‌های مکعبی
سری هندسی متناهی	$\sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1} \quad (r \neq 1)$	قضیهٔ اصلی، آرایهٔ پویا
سری هندسی نامتناهی	$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r} \quad (\lvert r\rvert < 1)$	شمارنده $(\sum 2^{-i} = 2)$، انتخاب تصادفی
عدد هارمونیک	$H_n = \sum_{i=1}^{n} \frac{1}{i} = \ln n + O(1)$	مسئلهٔ استخدام، کوئیک‌سورت
لگاریتم فاکتوریل	$\log(n!) = \Theta(n \log n)$	خودآزمایی فصل ۱

لگاریتم و توان

$\log(ab) = \log a + \log b$	$\log(a/b) = \log a - \log b$
$\log(a^k) = k \log a$	$\log_b a = \dfrac{\log_c a}{\log_c b}$
$a^{\log_b n} = n^{\log_b a}$	$2^{\log_2 n} = n$
$\log_2 3 \approx 1.585$	$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618, \quad \log_2 \varphi \approx 0.694$

احتمال

امید ریاضی	$E[X] = \sum_\omega P[\omega] X(\omega) = \sum_r r\, P[X = r]$
خطی بودن (بدون نیاز به استقلال)	$E[\sum X_i] = \sum E[X_i], \quad E[aX + b] = aE[X] + b$
متغیر نشانگر	$E[I_A] = P[A]$
تعداد تلاش تا اولین موفقیت (احتمال p)	$E[Y] = 1/p$
شکست c تلاش مستقل	$(1-p)^c$

قضیهٔ اصلی و رابطه‌های بازگشتی

رابطه	جواب	مثال
$aT(n/b) + O(n^d),\ \log_b a < d$	$O(n^d)$	$T(n/2) + n$
$aT(n/b) + O(n^d),\ \log_b a = d$	$O(n^d \log n)$	مرتب‌سازی ادغامی
$aT(n/b) + O(n^d),\ \log_b a > d$	$O(n^{\log_b a})$	کاراتسوبا
$T(an) + T(bn) + O(n),\ a + b < 1$	$O(n)$	انتخاب قطعی
$T(an) + T(bn) + O(n),\ a + b = 1$	$O(n \log n)$	کوئیک‌سورت با محور خوب
$T(n-1) + O(1)$	$O(n)$	–
$T(n-1) + O(n)$	$O(n^2)$	بدترین حالت کوئیک‌سورت
$T(n-1) + T(n-2) + O(1)$	$\Theta(\varphi^n)$	فیبوناچی بازگشتی

پ.۲ جدول الگوریتم‌های جزوه

مسئله	الگوریتم	زمان	فصل
مرتب‌سازی	درجی	$\Theta(n^2)$ بدترین، $\Theta(n)$ بهترین	۱
	ادغامی	$\Theta(n \log n)$	۲
	کوئیک‌سورت تصادفی	$O(n \log n)$ امید، $\Theta(n^2)$ بدترین	۳
فیبوناچی F_n	بازگشتی	$\Theta(\varphi^n)$	۱
	حلقه	$O(n)$	۱
	توان ماتریس	$O(\log n)$	۱
زیرآرایهٔ بیشینه	جامع	$O(n^3)$	۱
	مجموع پیشوندی	$O(n^2)$	۱
	تقسیم و حل	$O(n \log n)$	۲
	کادان	$O(n)$	۱ (تکمیلی)
ضرب دو عدد n بیتی	دبستانی	$\Theta(n^2)$	۲
	تقسیم و حل با ۴ ضرب	$\Theta(n^2)$	۲
	کاراتسوبا	$O(n^{1.585})$	۲
جمع دو عدد n بیتی	دبستانی (بهینه)	$\Theta(n)$	۲
میانهٔ δ-تقریبی	APPROX-MEDIAN-1 (مونت‌کارلو)	$O(n)$، موفقیت $\approx 2\delta$	۳
	APPROX-MEDIAN-2	$O(cn)$، موفقیت $1-(1-2\delta)^c$	۳
	APPROX-MEDIAN-3 (لاس‌وگاس)	$O(n/\delta)$ امید	۳
انتخاب (رتبهٔ i)	تصادفی	$O(n)$ امید، $\Theta(n^2)$ بدترین	۳
	میانهٔ میانه‌ها	$O(n)$ بدترین حالت	۳
استخدام	با جایگشت تصادفی	$f\,H_n \approx f \ln n$ امید هزینه	۳
جایگشت تصادفی	فیشر-ییتس	$O(n)$	۳
پشته با MULTIPOP	—	$O(1)$ سرشکن	۴

مسئله	الگوریتم	زمان	فصل
شمارندهٔ دودویی	INCREMENT	سرشکن $O(1)$، بدترین $O(k)$	۴
آرایهٔ پویا	دو برابر کردن	سرشکن $O(1)$ برای هر درج	۴ (تکمیلی)

پ۳۰ واژه‌نامهٔ فارسی–انگلیسی

فارسی	English
تحلیل مجانبی	asymptotic analysis
کران بالا / پایین / محکم	upper / lower / tight bound
بدترین حالت / حالت میانگین / بهترین حالت	worst / average / best case
ناوردای حلقه	loop invariant
جست‌وجوی جامع	brute force
زمان چندجمله‌ای	polynomial time
تقسیم و حل	divide and conquer
رابطهٔ بازگشتی	recurrence relation
درخت بازگشت	recursion tree
قضیهٔ اصلی	master theorem
روش حدس و استقرا (جای‌گذاری)	guess and induction (substitution method)
مرتب‌سازی ادغامی / درجی / سریع	mergesort / insertion sort / quicksort
زیرآرایه با بیشترین مجموع	maximum sum subarray
فضای نمونه / پیشامد	sample space / event
متغیر تصادفی / متغیر نشانگر	random variable / indicator random variable
امید ریاضی / خطی بودن امید	expected value / linearity of expectation
به‌طور یکنواخت تصادفی	uniformly at random (u.a.r.)
جایگشت تصادفی	random permutation
میانهٔ تقریبی / رتبه	approximate median / rank
جداکننده / محور	splitter / pivot
میانهٔ میانه‌ها	median of medians
تحلیل سرشکنی / هزینهٔ سرشکن‌شده	amortized analysis / amortized cost

فارسی	English
روش انبوهه / حساب‌داری / تابع پتانسیل	aggregate / accounting / potential method

پ۴۰ منابع

1. Jon Kleinberg, Éva Tardos. *Algorithm Design*. Pearson Addison-Wesley؛ بخش‌های ۲٫۱ تا ۲٫۴ (تحلیل و زمان‌های رایج)، ۵٫۱، ۵٫۲، ۵٫۴ و ۵٫۵ (تقسیم و حل) و فصل ۱۳ (الگوریتم‌های تصادفی).
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press؛ بخش ۲ (مرتب‌سازی درجی)، فصل ۴ (رابطه‌های بازگشتی و قضیهٔ اصلی)، فصل ۵ (مسئلهٔ استخدام) و فصل ۱۷ (تحلیل سرشکنی).
3. Udi Manber. *Introduction to Algorithms: A Creative Approach*. Addison-Wesley, 1989؛ بخش ۵٫۸ (زیرآرایهٔ بیشینه).
4. Mark de Berg. *Lecture notes on Advanced Algorithms* (میانهٔ تقریبی و الگوریتم‌های مونت‌کارلو و لاس‌وگاس).
5. اسلایدهای انگلیسی درس بر پایهٔ اسلایدهای Kevin Wayne (Pearson Addison-Wesley) تهیه شده‌اند.
6. اسلایدهای تحلیل سرشکنی: درس داده‌ساختارها و مبانی الگوریتم‌ها، دانشکدهٔ مهندسی کامپیوتر، محمد قدسی.

نکته · دربارهٔ این جزوه

ترتیب و محتوای اصلی این جزوه از اسلایدهای درس گرفته شده و توضیحات، اثبات‌های کامل، مثال‌های حل‌شده، شکل‌ها و خودآزمایی‌ها به آن افزوده شده است. هر جا اسلاید خطای کوچکی داشت (مثل اندیس‌های شبه‌کد ادغام و زیرآرایهٔ بیشینه) در متن تذکر داده شده است. مطالب کادرهای «تکمیلی» فراتر از اسلایدهاست.